

UNIVERSIDAD DE CASTILLA-LA MANCHA

ESCUELA SUPERIOR DE INFORMÁTICA

GRADO EN INGENIERÍA INFORMÁTICA

Ingeniería del Software

TRABAJO FIN DE GRADO

Smart Mallorca: App turismo

Óscar Domínguez Ocaña

Julio, 2020



UNIVERSIDAD DE CASTILLA-LA MANCHA

ESCUELA SUPERIOR DE INFORMÁTICA

Departamento de Tecnologías y Sistemas de Información

Ingeniería del Software

TRABAJO FIN DE GRADO

Smart Mallorca: App turismo

Autor: Óscar Domínguez Ocaña

Director (IECISA): Fernando de Cózar Ruano

Director (UCLM): Macario Polo Usaola

Julio, 2020

Smart Mallorca: App turismo
© Óscar Domínguez Ocaña, 2020

Este documento se distribuye con licencia CC BY-NC-SA 4.0. El texto completo de la licencia puede obtenerse en https://creativecommons.org/licenses/by-nc-sa/4.0/.

El escudo de Informática utilizado por este documento ha sido realizado por P. Moya, D. Villa e I. Díez, su inclusión debe respetar los derechos de autor y las licencias a las que se vea sometido. La copia y distribución de esta obra está permitida en todo el mundo, sin regalías y por cualquier medio, siempre que esta nota sea preservada. Se concede permiso para copiar y distribuir traducciones de este libro desde el español original a otro idioma, siempre que la traducción sea aprobada por el autor del libro y tanto el aviso de copyright como esta nota de permiso, sean preservados en todas las copias..



Tribunal:

Presidente: _____

Vocal: _____

Secretario: _____

Fecha de defensa: _____

Calificación: _____

Presidente	Vocal	Secretario
Fdo.:	Fdo.:	Fdo.:

A mi familia.

Resumen

El presente Trabajo de Fin de Grado (TFG) abordará el desarrollo de una aplicación móvil denominada Smart Mallorca para la Consejería de Turismo de la isla de Mallorca. La construcción de esta aplicación móvil implicará la realización de dos desarrollos paralelos tanto para Android como para iOS, este último será en el que estará inmerso el autor del presente TFG.

El desarrollo de la aplicación móvil de Smart Mallorca para iOS se llevará acabo siguiendo el patrón de arquitectura VIPER, recomendado en desarrollos de gran envergadura. Como lenguaje de programación se usará Swift, creado y enfocado por Apple para el desarrollo de aplicaciones móviles.

Esta aplicación permitirá a los usuarios que visiten la isla de Mallorca disfrutar de una experiencia en la que podrán interactuar con los recursos turísticos de la isla haciendo uso de realidad aumentada o a través de mapas personalizables.

El desarrollo del presente TFG se enmarca dentro de un convenio FORTE entre la Escuela Superior de Informatica de Ciudad Real (ESI) y la empresa Informática El Corte Inglés S.A. (IECISA). Este convenio ofrece a los estudiantes que están finalizando su etapa universitaria la oportunidad de desarrollar su TFG en un entorno real de trabajo en el que se incorporará a un equipo de desarrollo. La resolución de un problema real ha permitido al autor de este TFG aplicar de forma práctica los conocimientos adquiridos durante el desarrollo del Grado en Ingeniería Informática y, además, ha podido alcanzar nuevas destrezas en tecnologías novedosas y punteras, que son de uso frecuente en IECISA, empresa certificada en CMMI-DEV nivel 5.

Abstract

This Final Degree Project (TFG) will address the development of a mobile application called Smart Mallorca for the Tourism Department of the Island of Mallorca. For the construction of this application two parallel developments will be made for both Android and iOS. The author of this project will participate in the development of iOS.

The development of the Smart Mallorca mobile application for iOS will be carried out following the VIPER architecture pattern, recommended in large-scale developments. As a programming language Swift will be used, created and focused by Apple for the development of mobile applications.

This application will allow users who visit the island of Mallorca to enjoy an experience in which they can interact with the island's tourist resources using augmented reality or through customizable maps.

The development of this TFG is part of a FORTE agreement between the Escuela Superior de Informatica de Ciudad Real (ESI) and the company Informática El Corte Inglés S.A. (IECISA). This agreement offers students who are finishing their university stage the opportunity to develop their TFG in a real work environment in which they will join a development team. The resolution of a real problem has allowed the author of this TFG to apply in a practical way the knowledge acquired during the development of the Degree in Computer Engineering and, in addition, he has been able to achieve new skills in innovative and cutting-edge technologies, which are often used in IECISA, a company certified at CMMI-DEV level 5.

AGRADECIMIENTOS

Llegados a este momento, con este Trabajo de Fin de Grado pongo punto y final a unas de las etapas más díficles y la vez más satisfactorias de mi vida. Aunque con estas líneas no sería suficiente, me gustaría recordar y agradecer a todas las personas que han hecho que esto sea posible.

Gracias a mis padres Pascual y Ana Isabel y a mi hermano Alonso, que siempre han estado ahí cuando los he necesitado y han confiado en mi en todo momento.

Gracias a mis compañeros, a los que iniciamos esta aventura juntos y a los que se unieron, Gracias por esas horas de clase, por los buenos momentos vividos y por los que quedan por vivir.

Gracias a mis amigos, por sufrir de forma indirecta las consecuencias de las épocas más complicadas de cada curso.

Gracias a todo el equipo de Movilidad de IECISA, por su dedicación, su paciencia y su predisposición. Gracias por todos los consejos y por facilitar mi trabajo y adaptación durantes estos meses.

Gracias a mis tutores Fernando por parte de IECISA y Macario por parte de la ESI, por guiarme, ayudarme y dedicar su tiempo para tratar de obtener el mejor resultado posible.

Gracias a la Escuela Superior de Informática, por formarme y brindarme la oportunidad de finalizar esta etapa de mi vida de la mejor forma posible. Gracias a la empresa Informática El Corte Inglés, por darme la confianza y la oportunidad de llevar a cabo mi TFG en un proyecto real.

Óscar Domínguez Ocaña
Ciudad Real, 2020

LISTADO DE ACRÓNIMOS

API	:	Application Programming Interface
APP	:	Aplicación móvil
CMMI	:	Capability Maturity Model Integration
CMMI – DEV	:	CMMI for Development
ESI	:	Escuela Superior de Informática
SDK	:	Software Development Kit
TFG	:	Trabajo de Fin de Grado
IECISA	:	Informática El Corte Inglés S.A.

ÍNDICE GENERAL

Resumen **ix**

Abstract **xi**

Agradecimientos **xiii**

Listado de acrónimos **xv**

Índice de figuras **xxi**

Índice de tablas **xxv**

1 Introducción **1**
1.1 Motivación y contexto . . . 1
1.2 Estructuración del documento . . . 3

2 Objetivo **5**
2.1 Objetivo general . . . 5
2.2 Objetivos específicos . . . 5
2.3 Competencias adquiridas . . . 6
2.4 Pliego de condiciones de Smart Mallorca . . . 6
2.4.1 Requisitos generales de un canal tipo APP . . . 6
2.4.2 Requisitos funcionales agrupados por funcionalidad . . . 7
2.5 Conocimientos iniciales . . . 12

3 Estado del arte **15**
3.1 Antedecentes . . . 15
3.2 Tipos de aplicaciones móviles . . . 15
3.2.1 Aplicaciones nativas . . . 15
3.2.2 Aplicaciones web . . . 16
3.2.3 Aplicaciones híbridas . . . 16
3.2.4 Tipo de aplicación de Smart Mallorca . . . 17
3.3 Patrón de arquitectura VIPER . . . 17

4 Metodología de trabajo **21**

4.1 Flujo de trabajo en IECISA . . . 21
4.2 Scrum . . . 22
4.2.1 Equipo y roles . . . 22
4.2.2 Eventos de Scrum . . . 23
4.2.3 Artefactos de Scrum . . . 25
4.2.4 Historias de usuario . . . 25
4.2.5 Planning Poker . . . 26
4.3 Marco tecnológico . . . 27
4.3.1 Herramientas de documentación, planificación y seguimiento . . . 27
4.3.2 Herramientas de desarrollo . . . 28
4.3.3 Herramientas de gestión de la configuración . . . 29
4.3.4 Herramientas de comunicación . . . 30
4.4 Medios Hardware . . . 31

5 Resultados **33**
5.1 Inception . . . 34
5.2 Sprint 0 . . . 35
5.3 Sprint 1 . . . 35
5.3.1 Objetivo . . . 35
5.3.2 Sprint Planning . . . 36
5.3.3 Desarrollo por historias de usuario . . . 36
5.3.4 Sprint Review . . . 40
5.3.5 Sprint Retrospective . . . 41
5.4 Sprint 2 . . . 42
5.4.1 Objetivo . . . 42
5.4.2 Sprint Planning . . . 42
5.4.3 Desarrollo por historias de usuario . . . 42
5.4.4 Sprint Review . . . 46
5.4.5 Sprint Retrospective . . . 47
5.5 Sprint 3 . . . 47
5.5.1 Objetivo . . . 47
5.5.2 Sprint Planning . . . 48
5.5.3 Desarrollo por historias de usuario . . . 48
5.5.4 Sprint Review . . . 50
5.5.5 Sprint Retrospective . . . 51
5.6 Sprint 4 . . . 52
5.6.1 Objetivo . . . 52
5.6.2 Sprint Planning . . . 52
5.6.3 Desarrollo por historias de usuario . . . 52
5.6.4 Sprint Review . . . 58
5.6.5 Sprint Retrospective . . . 58
5.7 Sprint 5 . . . 59
5.7.1 Objetivo . . . 59

5.7.2 Sprint Planning . . . 60
5.7.3 Desarrollo por historias de usuario . . . 60
5.7.4 Sprint Review . . . 64
5.7.5 Sprint Retrospective . . . 65
5.8 Investigación . . . 65
5.8.1 Introducción . . . 65
5.8.2 Arquitectura VIPER . . . 66
5.8.3 Realidad Aumentada . . . 69
5.8.4 Navegación y mapas . . . 71

6 Conclusiones **75**
6.1 Conclusiones . . . 75
6.2 Cumplimiento de objetivos . . . 75
6.3 Destrezas adquiridas . . . 76
6.4 Competencias adquiridas . . . 78
6.5 Trabajo futuro . . . 78
6.6 Opinión personal . . . 79

Bibliografía **81**

A Inception Smart Mallorca App turismo **85**
A.1 Introducción . . . 85
A.2 Elevator pitch . . . 85
A.3 Project Comunnity . . . 86
A.4 Restricciones . . . 87
A.5 Riesgos . . . 88
A.6 Story Map . . . 89

B Realidad aumentada en Smart Mallorca: App turismo **91**
B.1 Introducción . . . 91
B.1.1 Características de la Realidad Aumentada . . . 92
B.1.2 Tipos de la realidad aumentada . . . 93
B.2 Realidad aumentada en Smart Mallorca . . . 93
B.3 Implementación de Realidad Aumentada en Smart Mallorca . . . 96
B.4 Demostración Realidad Aumentada . . . 100

ÍNDICE DE FIGURAS

1.1 Realidad Aumentada y mapas en Smart Mallorca 2

2.1 Competencias iniciales del autor 13

3.1 Diagrama base del patrón VIPER 17
3.2 Diagrama de clases en VIPER 18
3.3 Diagrama incluyendo flujo del patrón VIPER 19

4.1 Ciclo de vida de un proyecto en IECISA 21
4.2 Roles en Scrum 23
4.3 Eventos en Scrum 24
4.4 Transformación del Product Backlog en un incremento 25
4.5 Ejemplo de historia de usuario 26
4.6 Proceso de estimación con Planning Poker 26
4.7 Marco tecnológico de Smart Mallorca 27
4.8 Extracto de la pizarra de Smart Mallorca 28
4.9 Muestra de la colección de interfaces de Smart Mallorca 29
4.10 Muestra de la estructura de árbol en Sourcetree 30

5.1 Resumen desarrollo de cada Sprint 33
5.2 Diagrama de secuencia de esqueleto en VIPER 35
5.3 SplashScreen de Smart Mallorca 37
5.4 onBoarding de Smart Mallorca 38
5.5 Acceso de Smart Mallorca 39
5.6 Pantallas de registro en Smart Mallorca 40
5.7 Resumen retrospectiva Sprint 1 41
5.8 Menu lateral de Smart Mallorca 43
5.9 Pantalla inicial de realidad aumentada 44
5.10 Encuadre positivo de recursos turísticos 45
5.11 Información de un recurso turístico 46
5.12 Resumen retrospectiva Sprint 2 47
5.13 Composición pantalla Home 49
5.14 Estructura JSON de un recurso turístico 50
5.15 Pantalla Home Smart Mallorca 50

5.16 Resumen retrospectiva Sprint 3 . . . 51
5.17 SubHome 1 de Smart Mallorca . . . 53
5.18 Pantallas de SubHome 2 en Smart Mallorca . . . 54
5.19 Gestión de vistas en pantalla de descripción de recurso . . . 55
5.20 Vista descripción de un recurso en Smart Mallorca . . . 56
5.21 Vista de audioguías de un recurso turístico . . . 57
5.22 onBoarding de Smart Mallorca . . . 58
5.23 Resumen retrospectiva Sprint 4 . . . 59
5.24 Lector de códigos QR . . . 61
5.25 Opciones de registro en Smart Mallorca . . . 62
5.26 Ubicación actual en mapa . . . 63
5.27 Resultados de búsqueda en mapa . . . 64
5.28 Resumen retrospectiva Sprint 5 . . . 65
5.29 Proyecto esqueleto VIPER . . . 66
5.30 Protocolos de comunicación VIPER . . . 67
5.31 Módulo Router en VIPER . . . 67
5.32 Módulo View en VIPER . . . 68
5.33 Módulo Presenter en VIPER . . . 68
5.34 Módulo Interactor en VIPER . . . 69
5.35 Configuración pantalla de Realidad Aumentada . . . 69
5.36 Creación de escena SCNScene . . . 70
5.37 Creación de nodos SCNNode . . . 70
5.38 Declaración de variables AGSMapView y AGSGraphicsOverlay . . . 71
5.39 Configuración del mapa . . . 71
5.40 Llamada para creación de figuras en el mapa . . . 72
5.41 Adición de capa de gráficos al mapa . . . 72
5.42 Creación de figuras . . . 73

6.1 Competencias del autor al inicio TFG . . . 77
6.2 Competencias del autor a la finalización del TFG . . . 78

A.1 Elevator Pitch . . . 86
A.2 Project Comunnity de Smart Mallorca . . . 87
A.3 Riesgos de Smart Mallorca . . . 89

B.1 Realidad Aumentada en Pokemon Go . . . 92
B.2 Acceso a realidad aumentada en Smart Mallorca . . . 94
B.3 Proyección de recursos turísticos a través de realidad aumentada . . . 95
B.4 Función viewDidLoad en vista de Realidad Aumentada . . . 96
B.5 Barra de navegación superior en vista de Realidad Aumentada . . . 97
B.6 Configuración funcionalidad Realidad Aumentada . . . 97
B.7 Composición de vista en Realidad Aumentada . . . 98
B.8 Mapeo de información de recurso turístico en escena . . . 98
B.9 Protocolo funcionamiento Realidad Aumentada . . . 99

B.10 Gestión de toques en pantalla 99
B.11 Pantalla inicial de la demostración en Ciudad Real 100
B.12 Recurso turístico proyectado en Ciudad Real 101
B.13 Descripción del recurso 1 en Ciudad Real 102
B.14 Descripción del recurso 2 en Ciudad Real 103

ÍNDICE DE TABLAS

2.1 Requisitos generales de un canal de tipo APP 7
2.2 Requisitos de taxonomía adaptable 7
2.3 Requisitos de agenda de eventos 7
2.4 Requisitos de mi planificador 8
2.5 Requisitos de espacio personal y gestión de favoritos 8
2.6 Requisitos de buscador básico y avanzado 8
2.7 Requisitos de realidad aumentada 9
2.8 Requisitos de audio guías y video guías 9
2.9 Requisitos de geolocalización 9
2.10 Requisitos de asociación de información a los recursos turísticos 9
2.11 Requisitos de emergencias 10
2.12 Requisitos de interacción con códigos QR/NFC 10
2.13 Requisitos de versionado de clientes 11
2.14 Requisitos de notificaciones push 11
2.15 Requisitos de mapas y navegación 11
2.16 Requisitos de multiidioma 12
2.17 Requisitos de funcionamiento online y offline 12

3.1 Ventajas e incovenientes de las aplicaciones móviles nativas. 16
3.2 Ventajas e incovenientes de las aplicaciones web. 16
3.3 Ventajas e incovenientes de las aplicaciones móviles híbridas. 16

5.1 Sprint Backlog (Sprint 1) 36
5.2 Tareas de H0: Splash Screen 36
5.3 Tareas de H1: onBoarding 37
5.4 Tareas de H2: Login 38
5.5 Tareas de H3: Registro 39
5.6 Sprint Backlog (Sprint 2) 42
5.7 Tareas de H4: Menú lateral de navegación 42
5.8 Tareas de H5: Encuadre de cámara negativo (RA) 43
5.9 Tareas de H6: Encuadre de cámara positivo (RA) 44
5.10 Tareas de H7: Contenido Realidad Aumentada (RA) 46
5.11 Sprint Backlog (Sprint 3) 48

5.12 Tareas de H8: Home 48
5.13 Sprint Backlog (Sprint 4) 52
5.14 Tareas de H9: SubHome 1 52
5.15 Tareas de H10: SubHome 2 53
5.16 Tareas de H11: Recurso - Descripción 54
5.17 Sprint Backlog (Sprint 5) 60
5.18 Tareas de H12: Pantalla de escaneado QR 60
5.19 Tareas de H13: Preferencias registro 61
5.20 Tareas de H14: Mapa de recursos cercanos 62

6.1 Objetivos específicos cumplidos 76
6.2 Competencias adquiridas 79

A.1 Clasificación de riesgos y acciones a tomar 89
A.2 Historias de usuario identificadas en la *Inception* 90

CAPÍTULO 1

INTRODUCCIÓN

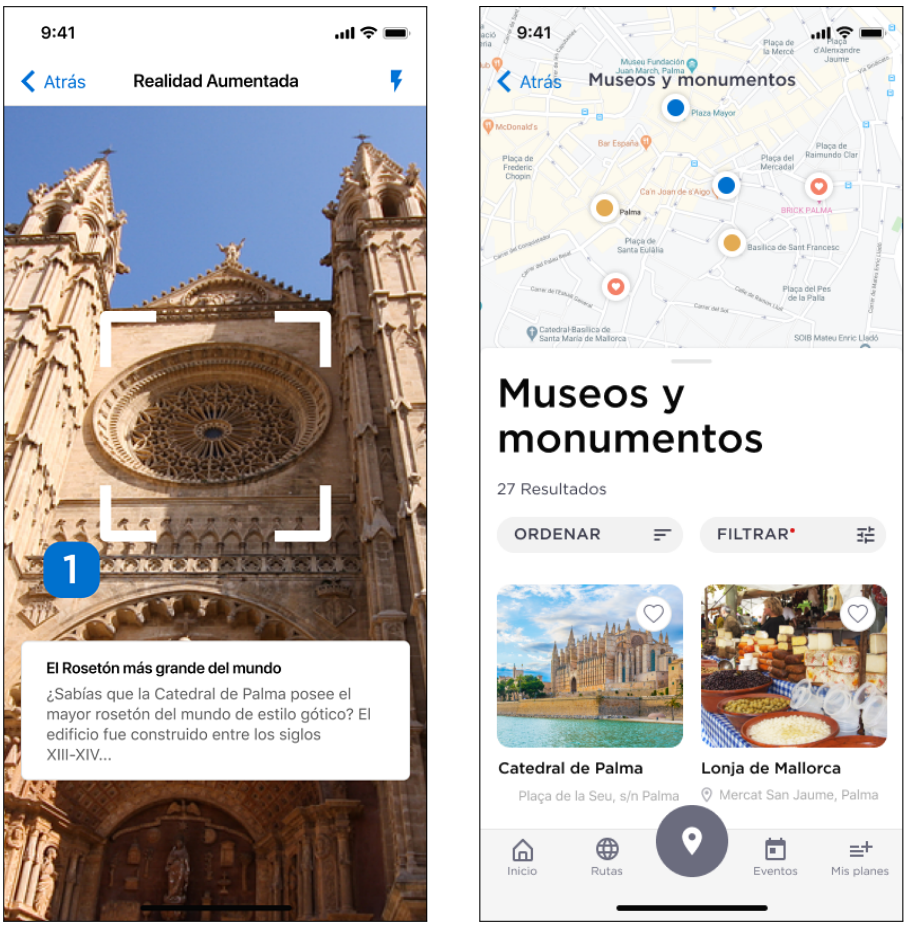
1.1 MOTIVACIÓN Y CONTEXTO

En el contexto de una constante transformación tecnológica surge este proyecto, que consiste en el desarrollo de una aplicación móvil para iOS denominada Smart Mallorca: App turismo, para el cliente Consejo Insular de Mallorca [7].

Esta aplicación permitirá reunir la información turística de la isla de Mallorca en un dispositivo móvil ofreciendo a los usuarios una experencia novedosa en el ámbito del turismo. Con la aplicación desarrollada los usuarios podrán planificar y planear sus visitas turísticas a la isla de Mallorca con antelación y además, cuando se encuentren en ella podrán interactuar con los recursos turísticos de diferentes formas. Una de ellas, y posiblemente la mas novedosa, es la Realidad Aumentada, con ella los usuarios podrá obtener información de los recursos turísticos desde la pantalla de sus dispositivos móviles.

El desarrollo de este TFG surge a raíz de un convenio FORTE [8] firmado entre la Escuela Superior de Informática [9] e Informática El Corte Inglés S.A. [15] con instalaciones en Miguelturra (Ciudad Real). Este convenio pretende mejorar las competencias profesionales de aquellos que lo cursan, favoreciendo así la empleabilidad. El autor de este TFG se incorpora en un proyecto real, formando parte de un equipo de desarrollo ágil.

En la figura B.3 se incluye un extracto de las pantallas de la aplicación en las que se puede apreciar la funcionalidad de Realidad Aumentada y la localización de recursos turísticos en mapas.



(a) Realidad Aumentada en Smart Mallorca **(b)** Mapas en Smart Mallorca

Figura 1.1: Realidad Aumentada y mapas en Smart Mallorca

Para la construcción de esta aplicación se trabajará con diferentes tecnologías, como puede ser geolocalización, realidad aumentada o navegación a través de mapas interactivos. Por tanto, la aplicación requerirá una perfecta orquestación entre las distintas tecnologías. Para ello la aplicación estará relacionada con los siguientes sistemas y tecnologías:

- **Backend Smart Mallorca**: Servicios web usados para cuestiones de acceso al sistema y recuperación de información correspondientes con los recursos turísticos de Mallorca.

- **Twitter**[22]: Se usará la API de Twitter para gestionar el acceso al sistema y así poder obtener información del usuario.

- **Facebook**[10]: Se usará el SDK de Facebook para controlar el acceso a la aplicación, además de obtener información del usuario para personalizar su perfil en Smart Mallorca.

- **ArcGIS**[2]: Se manejerá el SDK de ArGIS para Swift, el cúal permite la gestión y personalizacón de mapas y rutas.

- **ARKit**[3]: Se usará el SDK de ARKit para el soporte a la Realidad Aumentada y la representación del contenido a través de la cámara de los dispositivos móviles.

1.2 ESTRUCTURACIÓN DEL DOCUMENTO

El presente documento se estructura de la siguiente forma:

- En el **capítulo 2** se describe cuál es el objetivo principal de este TFG, además de los objetivos parciales que, conjuntamente, llevarán a la consecución del objetivo principal. Por último, se muestran los objetivos personales a conseguir y cuáles son los conocimientos iniciales de las tecnologías y herramientas a utilizar.

- En el **capítulo 3** se expone el estado del arte junto a una breve explicación de la evolución de las apliciones móviles, indicando que tipos existen y justificando su uso. Además, se presenta el patrón de arquitectura VIPER, el cúal será aplicado en el desarrollo del proyecto.

- En el **capítulo 4** se presenta el método de trabajo que se desarrolla en IECISA. Se presenta el marco de trabajo para desarrollo ágil Scrum, junto a sus características, artefactos, reuniones y roles. Además, se expone el marco tecnológico utilizado para el desarrollo de este proyecto.

- En el **capítulo 5** se presenta el desarollo de la aplicación Smart Mallorca, exponiendo cual ha sido el trabajo realizado y los resultados obtenidos en los diferentes *Sprints*. Para cada *Sprint* se detallan: objetivo del *Sprint*, planificación, historias de usuario realizadas, y resultados obtenidos. Además, se expone el trabajo de formación e investigación que ha realizado el autor.

- En el **capítulo 6** se exponen las conclusiones del autor al término de la realización del TFG y se indican las competencias adquiridas y la opinión personal del autor.

  Tras los capítulos se presenta la bibliografía consultada para realizar este TFG.

  Por último, se incluyen 2 anexos, donde se puede consultar con más detenimiento algunos aspectos del trabajo.

  - **Anexo A**: Inception de Smart Mallorca, en el que se desarrollan y se exponen los diferentes puntos de los que se compone una *Inception* o *Inception Desk*, y que mas adelante será explicado en profundidad.

  - **Anexo B**: Realidad Aumentada en Smart Mallorca. en el que se profundiza y se explica en detalle la implementación de Realidad Aumentada llevada a cabo en la aplicación Smart Mallorca.

CAPÍTULO 2

OBJETIVO

2.1 OBJETIVO GENERAL

El objetivo general de este TFG es el desarrollo de una aplicación móvil para iOS siguiendo el patrón de arquitectura VIPER y desarrollado en Swift. La finalidad de esta aplicación es facilitar a los usuarios el acceso a la información turística de la isla de Mallorca desde su dispositivo móvil, para que el turista pueda planificar su visita a la isla de Mallorca sin necesidad de visitar una oficina de turismo. De esta forma el turista se ahorraría la necesidad de obtener la información de forma presencial en una oficina física.

Además, el usuario podrá obtener información turística contextualizada en base a geolocalización y haciendo uso de Realidad Aumentada para descubrir la información de los recursos turísticos que le rodean.

2.2 OBJETIVOS ESPECÍFICOS

Para lograr el objetivo principal, este se ha dividido en una serie de objetivos parciales o específicos. Estos objetivos específicos son:

1. Desarrollo del módulo de Realidad Aumentada, que incluye la funcionalidad que permitirá a los usuarios localizar y obtener información de los recursos turísticos disponibles en la isla de Mallorca a través de Realidad Aumentada. Esta función permitirá a los usuarios localizar desde la cámara de su dispositivo móvil los recursos turísticos mas próximos a su localización geográfica y en base a la latitud, longitud y altitud de dichos recursos.

2. Desarrollar el módulo de gestión de los recursos turísticos en los mapas (búsqueda, visualización, filtrado, rutas).

3. Desarrollar el módulo que incluye las pantallas principales de la aplicación para la representación en el menú principal de los recursos turísticos, actividades turísticas, eventos, planes de viajes o expereriencias disponibles en la isla de Mallorca. Estos recursos estarán ordenados en base a tipologías de usuarios u ociotipos.

4. Desarrollar el módulo de gestión del acceso y registro a la aplicación turística.

5. Llevar a cabo en las pruebas necesarias en dispositivos físicos reales para comprobar el correcto funcionamiento de los distintos módulos.

2.3 COMPETENCIAS ADQUIRIDAS

2.4 PLIEGO DE CONDICIONES DE SMART MALLORCA

En este apartado se exponen agrupados en tablas, los requisitos y las condiciones generales expuestas por la Consejeria Insular de Mallorca para la aplicación Smart Mallorca. Para la aceptación de la aplicación por parte del cliente, es necesario el cumplimiento de todos los requisitos expuestos en las siguientes tablas.

2.4.1 Requisitos generales de un canal tipo APP

En este apartado se enumeran los requisitos generales de un canal de tipo APP (aplicación móvil) expuestos por el cliente en el pliego de condiciones y que son de obligado cumplimiento.

Identificador	Requisito	Descripción
R1	Desarrollo basado en estándares abiertos.	El canal APP deberá basar su desarrollo en estándares abiertos y de fácil mantenimiento que permitan la creación, distribución y gestión de los contenidos, que deberán ser accesibles desde dispositivos móviles, al menos, en las 2 últimas versiones de los sistemas operativos para móviles más utilizados.
R2	Descarga gratuita de la aplicación.	Permitirá la descarga gratuita en los respectivos markets (Android y Apple)
R3	Manejo sencillo y estética agradable.	El diseño del canal deberá seguir un modelo en base a carruseles u otro sistema alternativo que propicien una navegación amigable e intuitiva.
R4	Adecuación al tamaño de la ventana.	La maquetación de la APP deberá seguir criterios de elasticidad para permitir una adecuada adaptación al tamaño de la ventana del dispositivo móvil, tanto en smartphone como en tablet.
R5	Localización de forma anónima.	El canal APP permitirá obtener la localización del dispositivo móvil del usuario de forma anónima y con el consentimiento del usuario.
R6	Registro periódico de ubicación GPS.	El canal APP permitirá registrar, en un intervalo de tiempo parametrizable, la ubicación (coordenadas GPS) del dispositivo móvil del usuario y guardarlo en la base de datos de la Plataforma Smart.

Identificador	Requisito	Descripción
R7	Optimización del consumo de batería.	Permitirá definir los intervalos de tiempo para la transmisión de los datos obtenidos del dispositivo, con objeto de optimizar el consumo de batería.
R8	Preferencias de contenido.	El canal APP permitirá que el usuario pueda definir preferencias de contenidos.

Tabla 2.1: Requisitos generales de un canal de tipo APP

2.4.2 Requisitos funcionales agrupados por funcionalidad

En este apartado se enumeran los requisitos funcionales del canal de tipo APP (aplicación móvil) Smart Mallorca expuestos por el cliente en el pliego de condiciones y que son de obligado cumplimiento. Los requisitos se encuentran grupados en tablas que representan agrupaciones de requisitos relacionados con una funcionalidad concreta de la aplicación y que han sido identificados por el cliente.

Identificador	Requisito	Descripción
R9	Taxonomía adaptable.	La taxonomía y estructura de la información se adaptará específicamente al canal, pudiendo definirse categorización o etiquetado de los contenidos específicos para movilidad o trasladar de forma transparente los usados para la publicación web..

Tabla 2.2: Requisitos de taxonomía adaptable

Identificador	Requisito	Descripción
R10	Mostrar agenda de eventos.	El canal dispondrá de un área específica donde se mostrará la agenda de eventos.

Tabla 2.3: Requisitos de agenda de eventos

Identificador	Requisito	Descripción
R11	Crear planes personalizados.	El usuario del canal podrá crear planes personalizados, a los que podrá asignar eventos y compartir en redes sociales.
R12	Crear y editar cuadernos de viaje.	El canal permitirá que el usuario pueda crear y editar cuadernos de viaje. Los cuadernos de viaje sirven para que los turistas planifiquen su viaje.

Identificador	Requisito	Descripción
R13	Múltiples cuadernos de viaje por usuario.	Permitirá múltiples cuadernos de viaje por usuario.
R14	Compartir rutas y planes.	Permitirá que las rutas y planes registrados en el cuaderno de viaje puedan ser compartidos con otros usuarios del canal APP, por ejemplo por medio de email o en redes sociales, y que puedan ser exportados a PDF.
R15	Recursos georreferenciados.	Permitirá integrar los contenidos de información sobre mapas, utilizando la aplicación de mapas del dispositivo móvil, tipo Google Maps o similar, de manera que muestre los recursos georreferenciados.

Tabla 2.4: Requisitos de mi planificador

Identificador	Requisito	Descripción
R16	Favoritos y comentarios privados.	El canal dispondrá de un espacio personal que permitirá seleccionar contenidos como favoritos y asociarles comentarios privados.
R17	Botón favorito.	Se implementará un botón "Favorito" que permitirá almacenar dicho contenido como favorito, asociándolo, al menos, a un nombre y comentario.
R18	Acceso a favoritos.	Desde las principales vistas de la app, el usuario podrá acceder al listado de favoritos.

Tabla 2.5: Requisitos de espacio personal y gestión de favoritos

Identificador	Requisito	Descripción
R19	Buscador.	El canal dispondrá de un buscador básico y un buscador avanzado, que permitirá realizar búsquedas.
R20	Accesibilidad buscador.	Esta funcionalidad deberá ser accesible, al menos, desde todas las vistas principales mediante un botón disponible en la barra de menú.

Tabla 2.6: Requisitos de buscador básico y avanzado

Identificador	Requisito	Descripción
R21	Realidad aumentada.	Siempre que los dispositivos lo permitan y dispongan al menos de cámara y GPS, el canal dispondrá de un visor de realidad aumentada que permita acceder a toda la información de interés disponible, asociada a un punto de interés, que se encuentren en el entorno de la ubicación del usuario.

Tabla 2.7: Requisitos de realidad aumentada

Identificador	Requisito	Descripción
R22	Audio guías y video guías.	El canal permitirá utilizar audio guías y video guías asociadas a los recursos disponibles en la Plataforma Smart.

Tabla 2.8: Requisitos de audio guías y video guías

Identificador	Requisito	Descripción
R23	Permisos geolocalización.	El canal deberá geolocalizar al usuario desde el momento de inicio de la aplicación, permitiendo que el usuario pueda aceptar o denegar dicha funcionalidad de forma sencilla y permitiendo activarla o desactivarla en cualquier momento durante la sesión.
R24	Contenido en función de la posición.	Permitirá mostrar contenido relevante y cercano al dispositivo del usuario y lanzar la navegación a los diferentes contenidos geolocalizados de la forma más sencilla e intuitiva para el usuario.

Tabla 2.9: Requisitos de geolocalización

Identificador	Requisito	Descripción
R25	Asociar información a los recursos turísticos.	El canal permitirá asociar a los diferentes recursos (por ejemplo, lugares, eventos, rutas, etc) disponibles en el Gestor de Contenidos, imágenes, videos, audio y texto.

Tabla 2.10: Requisitos de asociación de información a los recursos turísticos

Identificador	Requisito	Descripción
R26	Mensaje de socorro.	El canal podrá redirigir un mensaje al servicio de emergencia 112, con información de la ubicación del dispositivo móvil del usuario y el mensaje de socorro.
R27	Notificaciones de socorro.	Para el envío de las notificaciones de socorro, el canal podrá utilizar los medios disponibles y configurados en la plataforma, que hayan sido definidos por el servicio de bomberos del Consell en coordinación con el 112..

Tabla 2.11: Requisitos de emergencias

Identificador	Requisito	Descripción
R28	Interacción con códigos QR.	El canal podrá interactuarán con los códigos QR.
R29	Interacción con códigos NFC.	El canal permitirá interactuar con dispositivos NFC (Near Field Communication) cercanos al dispositivo móvil del usuario, permitiendo el intercambio de datos y la presentación de información contextualizada asociada, en los casos en los que el terminal móvil disponga de la tecnología NFC y esté activada.
R30	Notificaciones de socorro.	Dependiendo desde dónde se escaneen los códigos QR, al menos se contemplarán los siguientes escenarios. • Si el usuario escanea un código QR desde un terminal para el que existe aplicación nativa publicada, pero no la tiene instalada, se le redirigirá para que pueda descargar la aplicación. • Si el usuario escanea un código QR desde un terminal para el que existe aplicación nativa publicada, y la aplicación está instalada, se abrirá la aplicación mostrando la información. • Si el usuario escanea un código QR desde un tipo de terminal para el que no existe aplicación nativa publicada, se le redirigirá al contenido web disponible.

Tabla 2.12: Requisitos de interacción con códigos QR/NFC

Identificador	Requisito	Descripción
R31	Versionado de clientes.	El canal permitirá mantener un registro de la interacción del usuario con los contenidos publicados en la plataforma (por ejemplo, número de consultas, activaciones, lugares consultados, etc) que permita la elaboración de métricas de uso.

Tabla 2.13: Requisitos de versionado de clientes

Identificador	Requisito	Descripción
R32	Recepción de notificaciones push.	Permitirá la recepción de notificaciones push asociadas a contenidos específicos. Si un usuario recibe una notificación push asociada a un contenido específico, cuando consulte la notificación se le mostrará el contenido específico asociado a la notificación.
R33	Recepción de notificaciones push segmentadas.	Permitirá la recepción de las notificaciones push segmentadas al menos por perfil (intereses).
R34	Integración de notificaciones push con servicios de Apple y Android.	El sistema de notificaciones push deberá estar integrado con los servicios de Apple y Google.

Tabla 2.14: Requisitos de notificaciones push

Identificador	Requisito	Descripción
R35	Recepción de notificaciones push.	Por defecto se deberá utilizar las herramientas y APIs nativas con la funcionalidad de mapas disponibles en cada ecosistema (es decir app con APIs propias de las plataformas IOS, Android o en su defecto las aplicaciones que permitan una integración completa y un alto grado de funcionalidad).

Tabla 2.15: Requisitos de mapas y navegación

Identificador	Requisito	Descripción
R36	Idioma de la aplicación.	El canal utilizará por defecto el idioma en el que se encuentre el terminal del usuario. En el caso de no estar disponible dicho idioma en la plataforma se utilizará el idioma por defecto del canal.
R37	Publicaciones en base a idiomas.	El canal permitirá la publicación de contenidos en los idiomas soportados en la Plataforma Smart conforme a los requerimientos recogidos en la Plataforma Smart Island y en Gestor de Contenidos.

Tabla 2.16: Requisitos de multiidioma

Identificador	Requisito	Descripción
R38	Caché local.	La solución incorporará un mecanismo de caché local y paquetes de contenidos iniciales, administrables desde el Gestor de Contenidos.
R39	Contenido en caso de no tener conectividad.	La aplicación deberá permitir la descarga de una serie de contenidos conforme al apartado anterior, que permita su uso incluso si el terminal no dispone de conectividad, como por ejemplo favoritos, rutas, etc.
R40	Offline para usuarios gestores.	El sistema offline para los usuarios gestores sólo será válido para aquellos contenidos informativos no relacionados con su actividad como gestores, debido a que estos deben tener en todo momento la información real y actualizada y la posibilidad de actualizar dicha información.

Tabla 2.17: Requisitos de funcionamiento online y offline

2.5 CONOCIMIENTOS INICIALES

En este apartado se va a plasmar el nivel de conocimiento que poseía el alumno sobre las competencias que ha debido adquirir para el desarrollo del TFG. En este capítulo se muestra su nivel de conocmimiento inicial antes de comenzar el desarrollo del TFG. Posteriormente, en el capítulo 6, se mostrarán las competencias adquiridas una vez terminado este TFG.

En la figura 2.1 se incluye un gráfico con las herramientas que se van a utilizar, con un nivel de gradación de inexistente, bajo, medio o alto, donde inexistente se corresponde con el desconocimiento total acerca de la herramienta o tecnología, bajo con un conocimiento o manejo bajo de la

herramienta, medio con un nivel aceptable de manejo o conocimiento y alto con un conocimiento eficaz y productivo de la herramienta. Cada herramienta se encuentra dividia en diferentes grupos en función de su utilidad dentro del desarrollo del proyecto de Smart Mallorca.

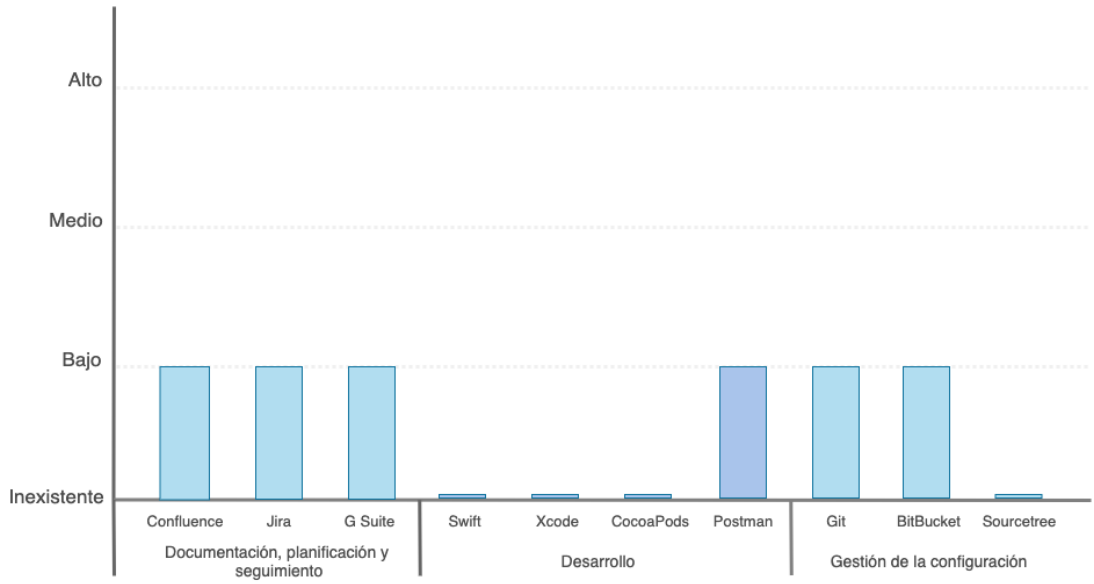


Figura 2.1: Competencias iniciales del autor al inicio del TFG

CAPÍTULO 3

ESTADO DEL ARTE

En este capítulo se pretende dar una imagen de la situación que rodea en el día de hoy al mundo de las aplicaciones móviles. Además se expondrá los tipos de apliciones móviles que existen en este momento, junto a sus ventajas e inconvenientes. Por último se procederá a explicar y justificar el uso del patrón de arquitectura para aplicaciones móviles VIPER, utilizado en el desarrollo de este proyecto.

3.1 ANTEDECENTES

Las primeras aplicaciones móviles surgen a finales de la década de los 90. La mayoría de estas aplicaciones eran muy básicas y unicamente cubrían necesidades esenciales. Las aplicaciones móviles fueron evolucionando y surgieron algunos de los juegos mas populares de la historia como pudieron ser el *Tetris* o el *Snake*.

En 2007 se produjo la entrada de Apple y Android en este mercado y se produjo un giro de 180º. Surgieron nuevas estrategias en el mundo de las apps, aparecieron los mercados digitales de aplicaciones móviles donde se permitió a los desarrolladores y a las compañias externas publicar sus aplicaciones.

En un primer momento las aplicaciones móviles fueron diseñas para facilitar y optimizar algunas aspectos de la vida cotidiana de las personas. Posteriormente, la evolución de las aplicaciones móviles se produjo hacia el ámbito del ocio y el entretenimiento. En la actualidad, existen aplicaciones que abarcan todos los ámbitos de nuestra vida, desde aplicaciones sociales o deportivas hasta aplicaciones culturales y de comercio. Se han convertido en herramientas indispensables para mejorar nuestra calidad de vida.

3.2 TIPOS DE APLICACIONES MÓVILES

3.2.1 Aplicaciones nativas

Las aplicaciones móviles nátivas son aquellas que se desarrollan específicamente para cada sistema operativo. Estas aplicaciones tiene la particularidad de que son desarrolladas en un único lenguaje dependiente de la plataforma; Objetive-C y Swift para iOS, Java o Kotlin para Android o .Net para Windows Phone.

Ventajas	Incovenientes
Mejor experiencia de usuario	Mayor coste de desarollo
Visibilidad en App Store	Diferentes herramientas en función de la plataforma
Acceso completo al dispositivo	
Actualización constante	

Tabla 3.1: Ventajas e incovenientes de las aplicaciones móviles nativas.

3.2.2 Aplicaciones web

Una aplicación web es aquella que se desarrolla en lenguajes muy conocidos para los programadores, como pueden ser Javascript, HTML y CSS. La principal ventaja de este tipo de aplicaciones es que se pueden programar independientemente del sistema operativo. Estas aplicaciones se ejecutan en el navegador web del teléfono y por tanto no es necesario que estén presentes en la tienda de aplicaciones ni tampoco requieren instalación.

Ventajas	Incovenientes
Código base reutilizable en múltiples plataformas	Requiere conexión a internet
Desarrollo sencillo y económico	Acceso muy limitado a características hardware del dispositivo
El usuario dispone siempre de la última versión	La experiencia del usuario y el tiempo de respuesta es menor que en una aplicación nativa
No es necesaria una aprobación externa para publicarse en las tiendas de aplicaciones	

Tabla 3.2: Ventajas e incovenientes de las aplicaciones web.

3.2.3 Aplicaciones híbridas

Una aplicación híbrida es una combinación de las dos anteriores. Las apps híbridas se desarrollan en lenguajes que son propios de las aplicaciones web, como pueden ser HTML, Javascript y CSS. La principal ventaja de estas aplicaciones es que, a pesar de estar desarolladas en lenguajes para aplicaciones web pueden ser distribuidas a través de las tiendas de aplicaciones móviles.

Ventajas	Incovenientes
Posibilidad de distribución en las tiendas de iOS y Android	Experiencia de usuario mas propia de aplicación web que de la aplicación móvil
Mismo código base para múltiples plataformas	Diseño visual no relacionado con el sistema operativo en el que se muestre
Desarrollo sencillo y económico	Acceso muy limitado a características hardware del dispositivo
Acceso a parte del hardware del dispositivo	
Instalación nativa pero construida con Javascript, HTML y CSS	

Tabla 3.3: Ventajas e incovenientes de las aplicaciones móviles híbridas.

3.2.4 Tipo de aplicación de Smart Mallorca

Puede parecer que una aplicación móvil híbrida puede ser una solución correcta para la mayoría de aplicaciones del mercado, pero en el caso de Smart Mallorca se procede a construir dos aplicaciones móviles nativas, una para iOS y otra para Android.

En este caso se han descartado las opciones de aplicación híbrida y web, porque no permitirían aprovechar al máximo el hardware de los dispositivos. Se ha elegido la opción de desarrollar dos aplicaciones nativas para aprovechar al máximo las características físicas que ofrecen los dispositivos móviles. Será importante explotar al máximo las características que nos ofrecen los dispositivos móviles en función del sistema operativo, como pueden ser en este caso la cámara, el giroscopio o el acelerómetro para la representación de distintos elementos a través de realidad aumentada o la geolocalización para la navegación a través de mapas.

3.3 PATRÓN DE ARQUITECTURA VIPER

Bien es sabido que la arquitectura de software es un aspeto clave a la hora de desarrollar en la industria del software. Es importante diseñar código de forma que cada pieza sea fácilmente identificable, ademas de que encaje de forma correcta y lógica con el resto de piezas.

VIPER, que es un acrónimo de las palabras *View, Interactor, Presenter, Entity y Router*, es un patrón de diseño para el desarollo de software que favorece el desarollo de código modular basado en arquitectura de código limpio. Los módulos en VIPER están orientados a protocolos y cada función de entrada y salida entre los distintos módulos se realiza a través de conjuntos específicos de reglas de comunicación.

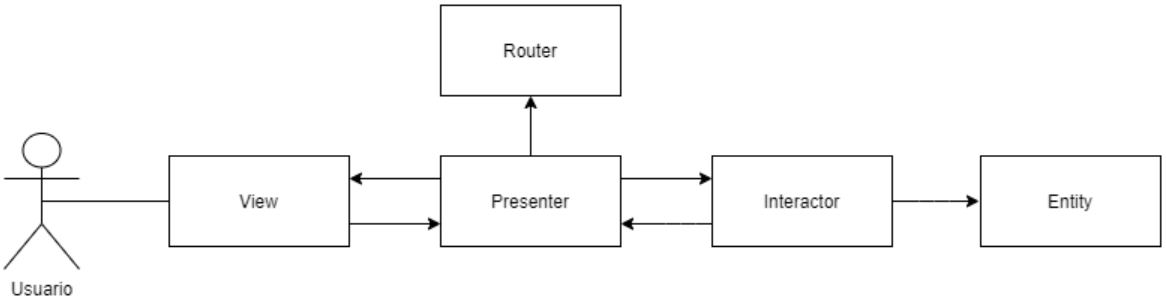


Figura 3.1: Diagrama base del patrón VIPER

- **View**: Su responsabilidad es la de enviar las acciones del usuario al *Presenter* y la de recibir y mostrar la información recbida del *Presenter*. El componente *View* es un componente pasivo, por lo tanto, nunca desencadenerá una acción si esta no ha sido provocada por el usuario o por el *Presenter*. Por otro lado, el *Presenter* conoce la existencia de la *View*, pero desconoce la existencia de los componentes propios de una vista, como pueden ser una etiqueta o un campo de ingreso de datos.

- **Interactor**: Representa un caso de uso de la aplicación. Contiene la lógica de negocios para manipular los objetos necesarios y llevar una tarea específica. El *Interactor* debe ser completamente independiente de cualquier interfaz de usuario. Obtiene los datos de la capa de conexión.

Generalmente se usará un protocolo para gestionar la comunicación entre el *Presenter* y el *Interactor*.

- **Presenter**: Contiene la lógica de la vista. El *Presenter* recibe información por parte del *Interactor* y la transforma de forma eficiente para que pueda ser mostrada en la *View*. Reacciona a las entradas del usuario y por lo tanto sabe cuando y a qué pantalla navegar.

- **Entity**: Modelo básico de objetos. Son manipulados por el *Interactor*.

- **Router**: Contiene la navegación lógica. También puede recibir el nombre de *Wireframe*. El *Router* es el responsable de la navegación y la transmisión de datos entre vistas. Este componente tiene la responsabilidad de crear el módulo e inicializar los módulos de la vista a la que se pretende navegar.

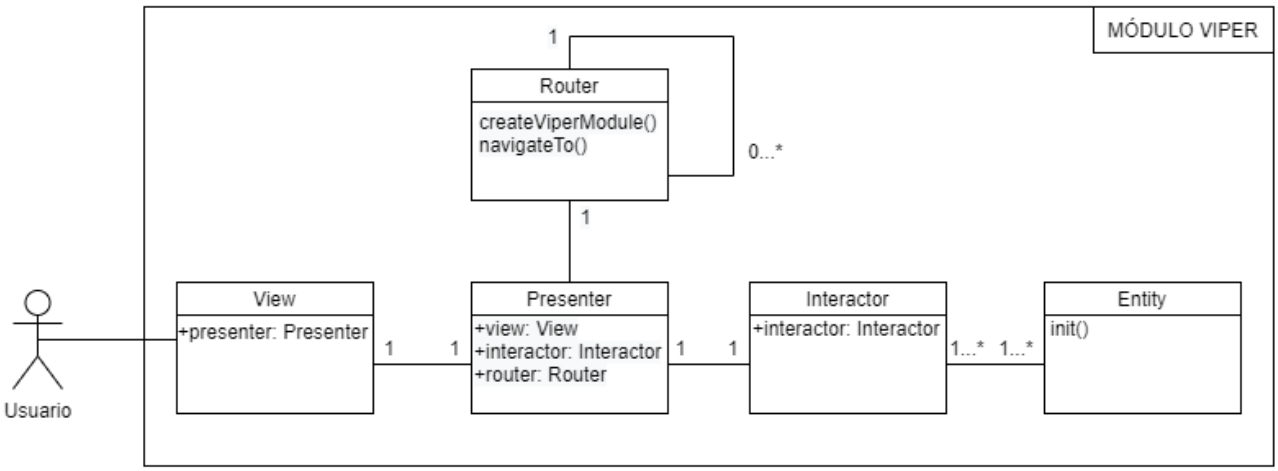


Figura 3.2: Diagrama de clases en VIPER

En la figura 3.2 se muestra un diagrama de clases[17] que incluye las relaciones existentes entre los distintos componentes de un módulo VIPER. La *View* únicamente mantiene una relación *uno a uno* con el *Presenter*. El *Interactor* mantiene una relación *muchos a muchos* con los *Entity* debido a que en un módulo *VIPER* un *Interactor* puede conocer varios *Entity* y un *Entity* puede ser conocido en distintos módulos *VIPER*. El componente *Presenter* mantiene una referencia con la *View*, el *Interactor* y el *Router*, esta última para desencadenar una navegación en el caso en el que fuera necesario. Por último, el *Router* mantiene una relación *ninguno a muchos* con otros componentes *Router* para realizar las navegaciones necesarias.

En la figura 3.3 se incluye un diagrama que dibuja los flujos de comunicación mas frecuentes en una arquitectura VIPER. El componente *View* únicamente debe mantener una referencia al *Presenter* para comunicarle los eventos que produzca el usuario. El componente *Presenter* es el que mantiene mayor número de referencias. Mantiene referencias con el *Router* para desencadenar una navegación si fuese necesario, con el *Interactor* para solicitar información y con la *View* para transmitirle información a

mostrar. El *Router* no mantiene ninguna referencia con ningún componente, solo tiene la potestad para navegar a otra vista. Por último, el *Interactor* mantiene una referencia con el *Presenter* para recibir órdenes y para transferirle la información obtenida de bases de datos o servicios web.

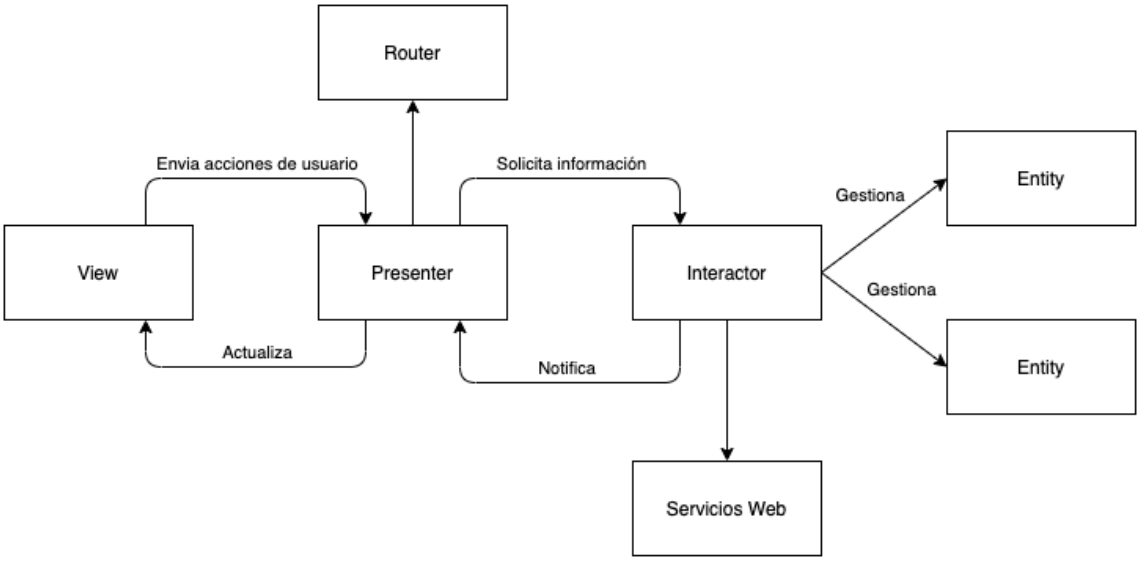


Figura 3.3: Diagrama incluyendo flujo del patrón VIPER

Desarrollar aplicaciones móviles usando el patrón VIPER nos ofrece una gran cantidad de ventajas. Algunas de las ventajas de aplicar este patrón en el desarrollo del proyecto Smart Mallorca son las siguientes:

- Simplificación de proyectos complejos.
- Distribución de responsabilidades clara entre los distintos módulos.
- Desacoplamiento de código que favorece la reutilización y la realización de pruebas unitarias.
- Aplicación de los principios SOLID:
  - Principio de Responsabilidad Única.
  - Principio de Abierto/Cerrado.
  - Principio de Sustitución de Liskov.
  - Principio de Segregación de la Interfaz.
  - Principio de Inversión de Dependencias

CAPÍTULO 4

METODOLOGÍA DE TRABAJO

En esta sección se describe la metodología de trabajo usada en IECISA. Se procederá a presentar el marco de trabajo ágil *Scrum* [8], junto a sus principales características, artefactos, reuniones y roles. Además, se explica el concepto de Inception de forma genérica y se define que es una historia de usuario y cómo se estiman, utilizando la técnica de Planning Poker.

Para finalizar la sección, se describe el marco tecnológico utilizado en el desarrollo de este trabajo y se mencionan los medios hardware utilizados.

4.1 FLUJO DE TRABAJO EN IECISA

Al inicio de cualquier proyecto se dispondrá de un pliego de condiciones, donde se encuentran detallados a alto nivel de abstracción las necesidades del cliente.

Una vez obtenido el pliego de condiciones, IECISA elabora una oferta, y si el cliente está de acuerdo con esa oferta, la adjudicación se hará efectiva. En el siguiente paso se lleva a cabo la firma del contrato por parte del departamento comercial de IECISA. Posteriormente se realizará el análisis, estimación y propuesta de ejecución (AEPE) para acordar las necesidades de personal, herramientas, tecnologías a utilizar o costes del proyecto. Una vez concluido el AEPE, se inicia el proyecto.

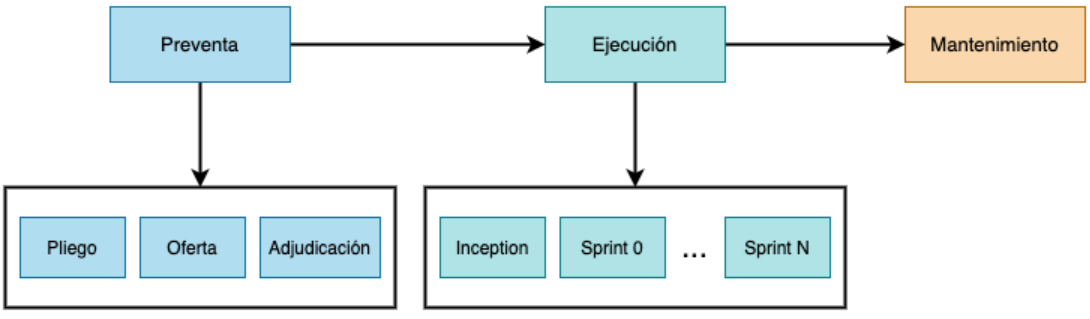


Figura 4.1: Ciclo de vida de un proyecto en IECISA

El diseño funcional y el diseño técnico de alto y bajo nivel pueden ser proporcionados por el cliente o realizados, como es el caso de Smart Mallorca, por IECISA. Esta documentación se realizará en paralelo con el desarrollo del proyecto.

Debido a que IECISA está certificada en un nivel 5 de madurez en CMMI, garantiza la aplicación de buenas prácticas de Ingeniería del Software para la mejora y evaluación de procesos para el desarrollo, mantenimiento y operación de sistemas de software.

4.2 SCRUM

Scrum es un marco de trabajo o framework que persigue como objetivo el de aplicar buenas prácticas para trabajar de forma colaborativa, fomentar el trabajo en equipo y obtener el mejor resultado posible. Adoptando *Scrum* a nuestra metodología de trabajo lo que se pretende conseguir es la entrega de valor en periodos cortos de tiempo, por ello se adopta una estrategia de desarrollo incremental.

El proyecto se divide en periodos cortos de tiempo que reciben el nombre de *Sprints*, y que pueden tener una duración de entre 1 y 4 semanas. La finalidad de cada *Sprints* es la de producir pequeños incrementos del producto final, lo que permite presentar resultados a los clientes cada poco tiempo y así favorecer la participación, comunicación y retroalimentación con el cliente.

4.2.1 Equipo y roles

En un equipo *Scrum* existen una serie de roles y responsabilidades que deben estar correctamente definidas, esos roles son *Scrum Master*, equipo de desarrollo y *Product Owner*.

- **Scrum Master**: Es la figura que dirige los equipos en la gestión ágil de proyectos. Su misión es la de alcanzar los objetivos de cada *Sprint* eliminando cualquier dificultad que pueda encontrarse en el camino y así poder alcanzar el incremento esperado. El *Scrum Master* es el responsable de que se sigan las prácticas y se adopten los valores descritos en el modelo Scrum. Las funciones principales de un *Scrum Master* son:

  - Negociar las condiciones con el cliente.
  - Incentivar al equipo de desarrollo.
  - Fomentar la autogestión del equipo.
  - Evitar intromisiones no esperadas y resolver los conflictos que impidan la correcta aplicación de Scrum.

- **Equipo de desarollo**: El equipo de desarrollo está integrado por profesionales que son los encargados de transformar las necesidades de los clientes expuestas por el *Product Owner* en la pila del producto, en un incremento del producto que pueda entregar valor al cliente. Los equipos de desarrollo tienen las siguientes características:

  - Son equipos auto-organizados, por lo que nadie, ni siquiera el Scrum Master, tendrá la potestad de dirigir al equipo de desarrollo en la realización de las tareas para conseguir un incremento de funcionalidad.
  - Son multifuncionales, es decir, cuentan con las habilidades necesarias para obtener un incremento de producto.

  - Es recomendable que no se reconzcan títulos dentro del equipo de desarrollo. Todos los profesionales son reconocidos como desarrolladores, independientemente del trabajo que realicen.
  - No se reconocen sub-equipos dentro del equipo de desarrollo.
  - Cada integrante del equipo puede estar especializado en una habilidad o en un área concreta pero la responsabilidad de los errores recae en el equipo entero.

- **Product Owner**: Es la persona más cercana al negocio y por lo tanto el responsable de tomar las decisiones relativas al producto. Es el encargado de establecer la prioridad de cada uno de los requisitos que componen la pila de producto o *Product Backlog*. Además, el *Product Owner* es el responsable de validar el *Sprint* comprobando para ellos si los objetivos propuestos al comienzo del *Sprint* se han cumplido.

  Muchas veces, el *Product Owner* es una de las personas que está estrechamente ligada a la creación del producto, y por lo tanto no pueden o prefieron no participar en el marco *Scrum*. Es por ello que surge una figura nueva, normalmente un profesional de la empresa, que se encarga de realizar las funciones del *Product Owner*. Este rol recibe el nombre de *Proxy Product Owner*, su función principal es la de actuar de mediador entre el cliente y el equipo de desarrollo. No se debe olvidar que el *Proxy Product Owner* no es el dueño del producto y por lo tanto esto puede restar efectividad al marco *Scrum*.

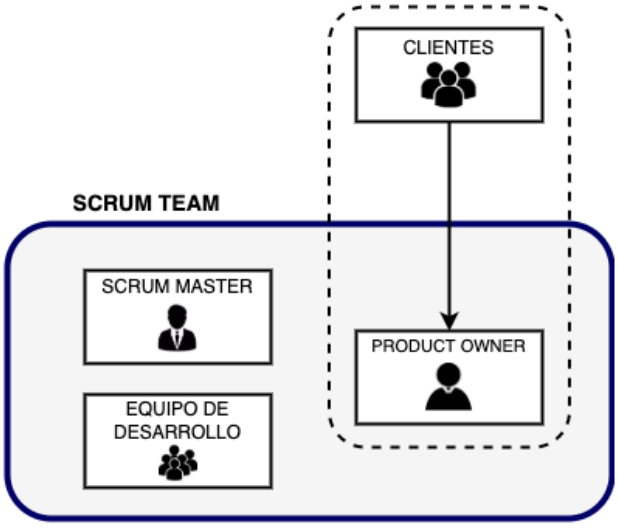


Figura 4.2: Roles en Scrum

4.2.2 Eventos de Scrum

Con el fin de mantener una cierta regularidad y minimizar la necesidad de realizar reuniones no definidas, en *Scrum* existen diferentes eventos predefinidos con una duración máxima que no debe ser sobrepasada y que contribuyen a cumplir las buenas prácticas del marco *Scrum*.

- **Sprint:** Es el evento más importante de *Scrum* entorno al cual giran el resto de eventos. Cada

Sprint es una iteracción dentro de un proyecto y abarca un periodo de tiempo de entre 1 y 4 semanas en el cual se crea un incremento de producto utilizable y potencialmente desplegable.

Cada *Sprint* contiene un *Sprint Planning*, una *Dialy Scrum* por cada día de trabajo, un *Sprint Review* y una *Sprint Retrospective*.

- **Sprint Planning:** En esta reunión, en la cual intervienen todos los miembros del *Scrum Team*, se planifica el trabajo a realizar durante el *Sprint*. El *Product Owner* presenta el conjunto de historias de usuario y es el equipo de desarrollo el que determina cúales de esas historias serán abordadas en el *Sprint*.

- **Daily Scrum:** Es una reunión que se realiza todos los días y que tiene una duración máxima de 15 minutos. En ella el equipo de desarrollo inspecciona el trabajo realizado en las 24 horas anteriores a la reunión y realiza una proyección acerca del trabajo a completar en las 24 horas posteriores. El equipo de desarrollo utiliza el *Daily Scrum* para evaluar el progreso hacia la consecución del objetivo de *Sprint*.

- **Sprint Review:** Al finalizar cada *Sprint* se lleva a cabo una revisión para inspeccionar el incremento y si fuese necesario, adaptar el *Product Backlog*. Se trata de una reunión informal en la cual, el *Scrum Team* y los interesados colaboran acerca de lo que se hizo durante el Sprint para determinar las siguientes acciones a tomar para optimizar el valor.

- **Sprint Retrospective:** Al finalizar cada *Sprint*, después de la *Sprint Review*, todo el equipo de Scrum muestra sus impresiones acerca del *Sprint* que acaba de finalizar. El principal objetivo de esta reunión es el de inspeccionar como fue el último *Sprint* en relación a las personas, relaciones, herramientas y tecnologías.

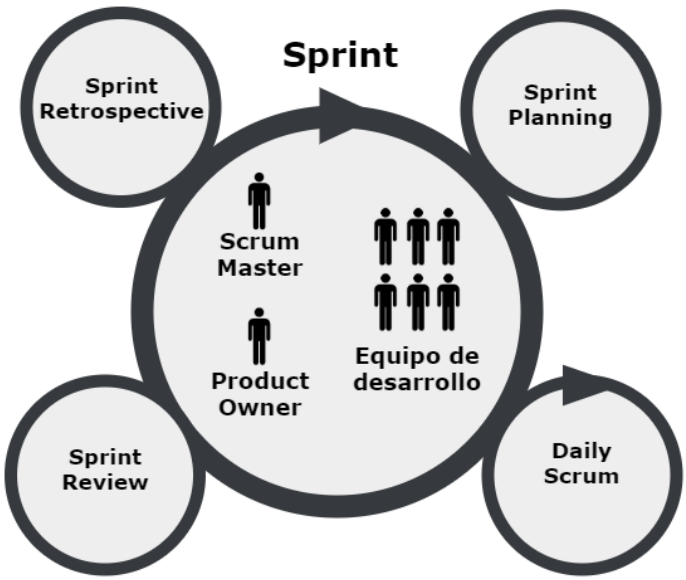


Figura 4.3: Eventos en Scrum

4.2.3 Artefactos de Scrum

- **Product Backlog:** La pila del producto o *Product Backlog* es una lista priorizada que enumera todas las características, funcionalidades, requisitos, mejoras y correciones necesarias para que el producto pueda estar completo. Se trata de un artefacto dinámico, debido a que se encuentra en constante evolución. El *Product Owner* es el encargado de gestionar, completar y priorizar esta pila.

- **Sprint Backlog:** Es el conjunto de los elementos del *Product Backlog* seleccionados para el *Sprint*. Esta pila incluye todo el trabajo que el equipo de desarrollo cree necesario para alcanzar un incremento en un *Sprint*.

- **Increment:** El *Increment* consiste en la suma de todos los elementos del *Sprint Backlog* que han sido terminados. A la conclusión del *Sprint*, el nuevo *Increment* deber estar terminado, es decir, debe estar en condiciones de ser utlizado, aunque el *Product Owner* decida no liberarlo.

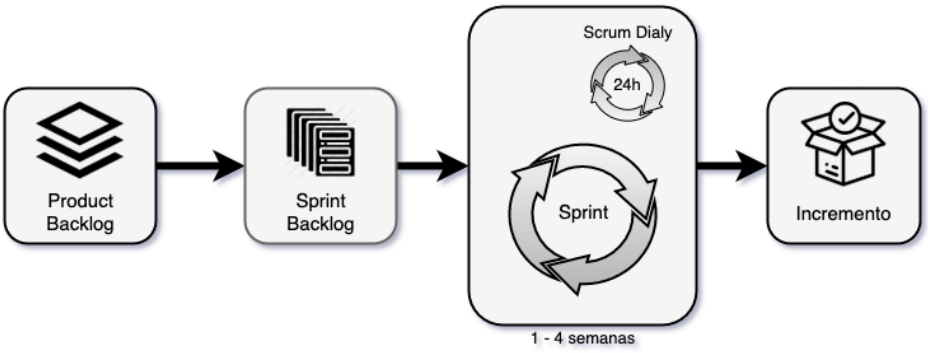


Figura 4.4: Transformación del Product Backlog en un incremento

4.2.4 Historias de usuario

Para satisfacer las necesidades de los clientes y usuarios es esencial que los requisitos estén bien claros y definidos. Gracias a las historias de usuario el cliente puede exponer su necesidades en un lenguaje cercano al del cliente final. Si los requisitos estuviesen redactados en un lenguaje más técnico, desarrolladores y *Stakeholders* alejarían sus posturas y se producirían malentendidos.

Tradicionalmente, las historias de usuario se reflejaban en pequeñas hojas de papel autoadhesivo, aunque con el paso del tiempo se ha trasladado la gestión de estas a herramientas software mucho más completas como *Jira*. En la figura 4.5 se incluye un ejemplo de historia de usuario.

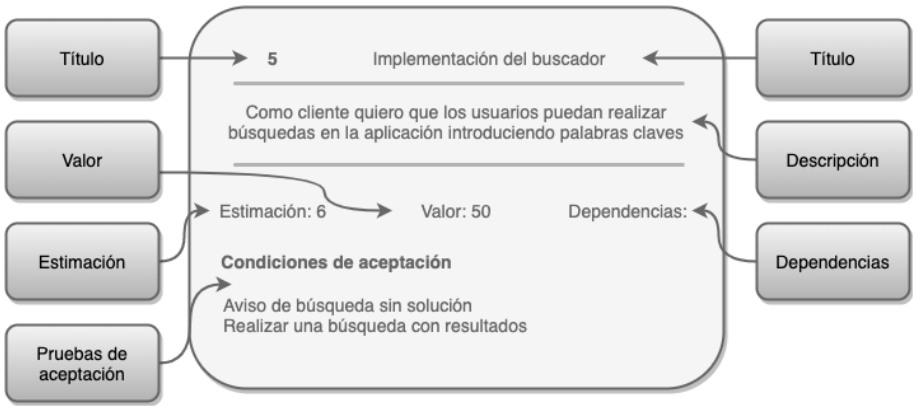


Figura 4.5: Ejemplo de historia de usuario

4.2.5 Planning Poker

A la hora de planificar las historias de usuario que se incluirán en un *Sprint*, es necesario calcular el tamaño o el esfuerzo que nos ocuparía resolver cada historia.

Para llevar a cabo esa estimación se utiliza la técnica *Planning Poker*. El objetivo de esta técnica es el de calcular en base a una estimación basada en el consenso, cuanto tiempo nos llevaría resolver una tarea de desarrollo de software. Para ello cada miembro del Equipo de Desarrollo haciendo uso de un mazo de cartas numeradas siguiendo la secuencia de *Fibonacci*, expone en base a su juicio el esfuerzo que le dedicaría a esa tarea.

En la figura 4.6 se muestran los pasos necesarios para realizar una estimación usando la técnica de *Planning Poker*:

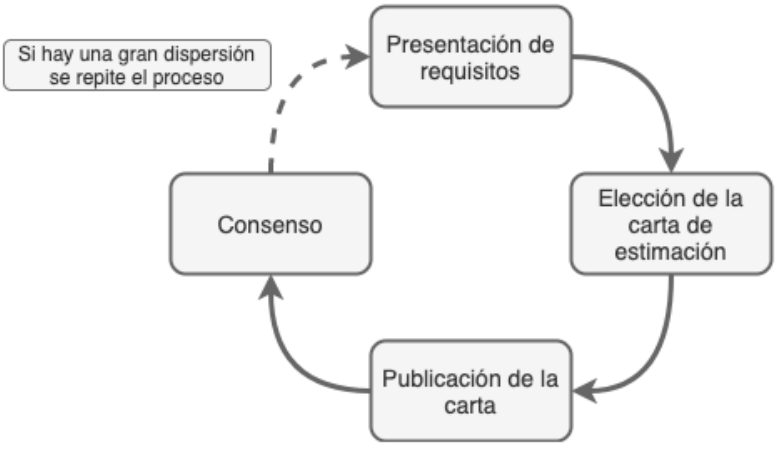


Figura 4.6: Proceso de estimación con Planning Poker

1. **Presentación de los requisitos:** En primer lugar, se presentan y se realiza una pequeña descripción de aquellas historias de usuario que se van a estimar y se procede a discutir aquellos detalles que no hayan quedado claros.

2. **Estimación:** Tras un primer periodo de discusión, cada miembro del equipo escoge de su mazo de cartas aquella que mejor represente su estimación de trabajo.

3. **Publicación:** Finalmente se publican todas las estimaciones, es decir, cada integrante del equipo muestra a la vez la carta seleccionada (esto es así para evitar que las estimaciones de unos modifiquen las de otros). Si existe una gran dispersión entre las estimaciones se vuelve a discutir el requisito y se vuelve a realizar la estimación.

4. **Consenso:** La existencia de dispersión en las estimación es síntoma de que la información acerca de las historias de usuarios no es completa o exacta. En el caso de una dispersión amplia se realiza una segunda ronda de estimación que permite aclarar aquellos puntos que no están claros. En la siguiente ronda de estimación la dispersión en las estimaciones debe ser mucho menor o incluso inexistente.

4.3 MARCO TECNOLÓGICO

En este apartado se enumeran y definen las tecnologías y herramientas utilizadas en el desarrollo del proyecto Smart Mallorca que de igual forma, serán utilizadas para el desarollo de este TFG. Estas herramientas están clasificadas en herramientas de documentación, planificación y seguimiento, herramientas de desarollo, herramientas de gestión de la configuración y herrramientas de comunicación.

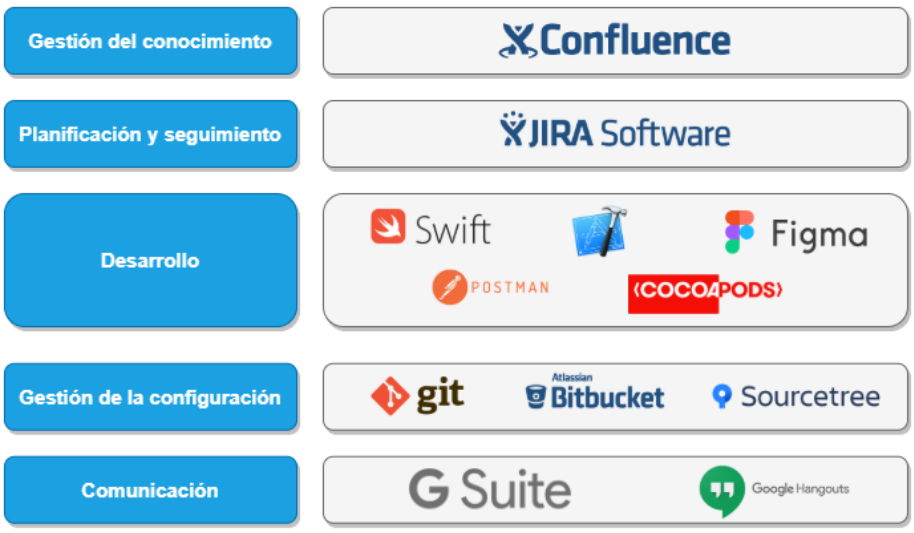


Figura 4.7: Marco tecnológico de Smart Mallorca

4.3.1 Herramientas de documentación, planificación y seguimiento

En este apartado se exponen las herramientas utilizadas para la gestión, planificación y gestión documental del proyecto.

4.3.1.1 Confluence

Confluence[6] es una herramienta software de colaboración desarrollada por *Atlassian* que permite a los equipos de desarrollo colaborar y compartir conocimientos de forma eficiente. En *Confluence*, el contenido se estructura y organiza mediante espacios, páginas y blogs en los cúales, cualquier miembro del equipo puede crear, modificar o eliminar contenido. Además de facilitar la gestión y reutilización de conocmiento a través de un único servicio, esta herramienta se puede utilizar para mantener los requisitos de un proyecto, bocetos de interfaces o informes de reuniones.

4.3.1.2 Jira

Jira[1] es una herramienta software desarrollada por *Atlassian* y utilizada para la gestión de proyectos. *Jira* permite gestionar las historias de usuario, tareas, errores o incidencias de un proyecto.

Dentro de IECISA, *Jira* es utlizada para planificar y gestionar los *Sprints* en *Scrum*. En cada *Sprint* se construye una pizarra para añadir todas las tareas a completar en el *Sprint* y gestionar el estado en el que se encuentren. Ademas, *Jira* permite la asignación de las tareas a los miembros del equipo de desarrollo, la creación de incidencias, la notificación de resolución de tareas o la gestión de riesgos.

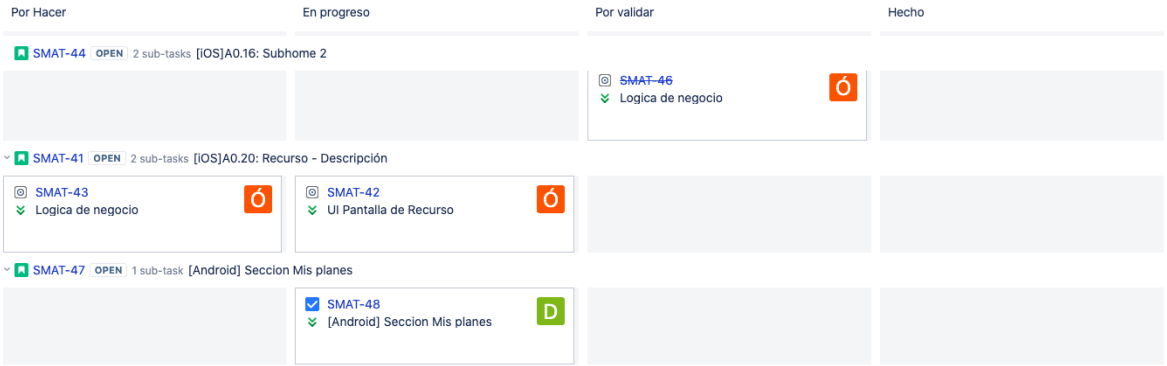


Figura 4.8: Extracto de la pizarra de Smart Mallorca

4.3.2 Herramientas de desarrollo

En este apartado se declaran las herramientas utilizadas para el desarrollo del proyecto.

4.3.2.1 Xcode 11

Xcode[23] es un entorno de desarrollo integrado (IDE) creado por Apple para macOS que contiene un conjunto de herramientas destinadas al desarrollo de software para macOS, iOS, watchOS y tvOS. Cuenta con una herramienta gráfica para crear intefaces denominada *Interface Builder*. Además, permite compilar código C, C++, Swift, Objetive-C, Java y AppleScript.

4.3.2.2 Postman

Postman [16] una herramienta que permite la creación de peticiones HTTP sobre APIs de una forma muy sencilla. Además, esta herramienta permite la construcción de APIs de aplicaciones. *Postman* también permite la escritura, monitorización y automatización de pruebas sobre los servicios

REST. Esta herramienta será utlizada en este proyecto para probar la comunicación y el estado de los diferentes servicios de Smart Mallorca.

4.3.2.3 CocoaPods

CocoaPods[5] es un administrador de dependencias a nivel de aplicación para Objective-C, Swift y cualquier otro lenguaje que se ejecute en el tiempo de ejecución de Objective-C, que proporciona un formato estándar para administrar bilbiotecas externas. *CocoaPods* se ejecuta a través de la línea de comandos. Para especificar dependecias de un proyecto en *Xcode* se utiliza un archivo de texto simple proporcionado por *CocoPods* denominado *Podfile*, dentro de este fichero el usuario puede indicar las dependecias que desea incluir en su proyecto, indicando para ello su nombre y versión. *CocoaPods* se encarga de recoger e instalar las dependencias de forma automática.

4.3.2.4 Figma

Figma[11] es una herramienta ejecutada en navegadores web y empleada para el diseño colaborativo de prototipos de interfaces de usuario. Una de las principales ventajas, a parte de la posibilidad de realizar diseños colaborativos, es que permite diseñar las interfaces con un preciso nivel de detalle. Además, se pueden establecer conexiones y puntos de acceso en el diseño para simular la navegación de los usuarios.

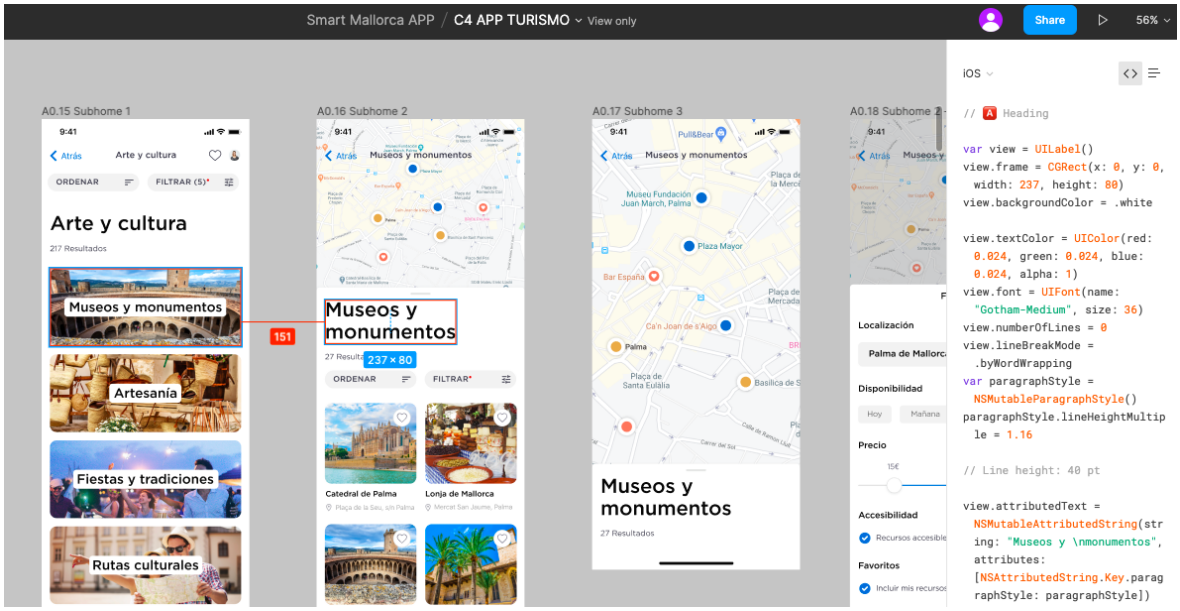


Figura 4.9: Muestra de la colección de interfaces de Smart Mallorca

4.3.3 Herramientas de gestión de la configuración

En este apartado se enumeran las herramientas y tecnologías empleadas para realizar la gestión de la configuración en el proyecto.

4.3.3.1 Git

Git [13] es un software de control de versiones distribuido (DVCS) pensando para la gestión de los cambios y la configuración de un producto software. *Git* permite gestionar y llevar un control de

un proyecto cuando este tiene una gran cantidad de archivos de código fuente.

Esta tecnología está basada en un sistema de trabajo con ramas que permite mantener un historial completo de versiones. Las ramas permiten tener una línea de progreso distinta de una la principal en la cúal realizar mejoras o cambios para posteriormente realizar una fusión con el proyecto principal.

4.3.3.2 Bitbucket

Bitbucket[4] es un servicio de alojamiento compartido basado en web y desarrolado por Atlassian, para código fuente y proyectos de desarrollo de software que utiliza como sistema de control versiones Git y Mercurial.

Bitbucket permite la creación de proyectos tanto de proyectos públicos como privados y está integrado con *Jira*, *Confluence* o *Bambo*. Cuenta con tres modelos de implementación como son Cloud, Bitbucket Server y Data Center.

4.3.3.3 Sourcetree

Sourcetree[19] es un cliente para Mac y Windows que proporciona una interfaz gráfica intuitiva para la gestión de repositorios Git y Mercurial. *Sourcetree* proporciona una estructuración de las diferentes ramas de un proyecto en forma de árbol que favorece la compresión y gestión de la estructura del repositorio.

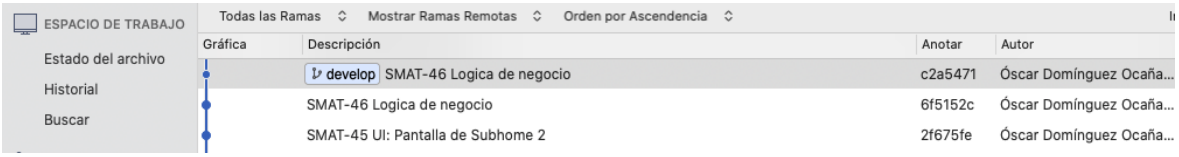


Figura 4.10: Muestra de la estructura de árbol en Sourcetree

4.3.4 Herramientas de comunicación

En este apartado se enumeran las herramientas empleadas para facilitar la comunicación durante el desarrollo del proyecto.

4.3.4.1 GSuite

GSuite [12] es un servicio proporcionado por Google que incluye numerosas aplicaciones para mensajería y comunicación que satisfacen las necesidades fundamentales de la empresa. Todas estas herramientas están hospedadas en la infraestructura de alta seguridad y disponibilidad de Google. Además, estas herramientas requieren una admninistración mínima. Particularmente, *GSuite* es bastante útil debido a que el equipo de desarrollo se encuentra deslocalizado durante el tiempo de desarrollo de este proyecto.

Algunas de las herramientas que forma parte de GSuite y que serán utilizadas durante el proyecto son:

- **Gmail:** Proporciona un servicio de correo electrónico bajo el dominio de la propia empresa.

- **Drive:** Proporciona un servicio para crear, modificar, compartir y almacenar documentos. Dentro del proyecto, esta herramienta es utlizada para compartir y almacenar documentos de requisitos y reuniones.

- **Calendar:** Servicio para la gestión de calendarios e instalaciones. Permite la gestión y programación de eventos y reuniones entre los miembros del equipo de desarrollo en las distinas salas de reuniones del centro de trabajo.

- **Chat:**. Servicio de mensajería instantánea individual y grupal que facilita la comunicación entre los distintos miembros del equipo de desarrollo.

4.3.4.2 Google Hangouts

Hangouts[14] es una aplicación de mensajería multiplataforma que, pese a estar incluida dentro de GSuite, merece una mención aparte por la importancia que cobró durante el desarrollo del proyecto debido a la situación provocada por el Coronavirus (Covid-19), la cual imposibilitaba las reuniones físicas en el centro de trabajo. Permite mantener conversaciones entre dos o más usuarios. Además, permite mantener videollamadas grupales entre los diferentes miembros del equipo de desarrollo. Debido a que las diferentes conversaciones se archivan en la nube, cualquier usuario podría acceder a las conversaciones desde cualquier dispositivo.

4.4 MEDIOS HARDWARE

El equipo hardware empleado para la realización de este TFG es un ordenador portatil MacBook Air: macOS Catalina versión 10.15.4; Procesador 1,6 GHz Intel Core i5; 8 GB 1600 Mhz DDR3; Gráficos Intel HD Graphics 6000 1536 MB.

Ademas de los emuladores que incluye Xcode[23] para probar la funcionalidad de la aplicación en desarrollo, el autor de este TFG cuenta con uno de los dispositivos móviles físicos que tiene el departamento de Movilidad para realizar comprobaciones:

- iPad Pro (iOS 12.1.3)

CAPÍTULO 5

RESULTADOS

En este capítulo se exponen los resultados obtenidos al aplicar el método de trabajo expuesto en el capítulo 4. Se aclara como se han construido progresivamente los distintos incrementos que han dado lugar a la consecución del objetivo principal de este TFG. Para facilitar la comprensión en la evolución del trabajo, el capítulo se organiza en secciones correspondientes con cada uno de los *Sprints* realizados.



Figura 5.1: Resumen desarrollo de cada Sprint

5.1 INCEPTION

En este apartado se explica de forma resumida el trabajo realizado en la reunión *Inception* para Smart Mallorca. El trabajo extendido se puede examinar en el anexo A.

La reunión tuvo una duración de 4 horas y entre los asistentes se encontraban el *Scrum Master*, el jefe del proyecto y los miembros del equipo de desarrollo. La reunión estuvo dividida en un total de cinco fases.

En la primera fase denominada *Vision*, de forma colaborativa y entre todos los asistenes se define el *Elevator Pitch* con el que se pretende obtener una idea clara del proyecto. Con esta actividad lo que se pretende es alinear la visión de todos los involucrados en el proyecto para así conseguir una idea clara del objetivo que se pretende conseguir con la realización de este proyecto.

En una segunda fase denominada *Project Community* se pretende identificar a todas las personas involucradas en el proyecto. Una temprana identificación y relación de los participantes en un proyecto es importante para alcanzar el éxito del proyecto.

En una tercera fase denominada *Restricciones* se identifican y anotan todas las restricciones a las que estará sometido el proyecto. También es posible identificar una serie de compromisos a adoptar por parte de los involucrados en el proyecto para su puesta en marcha.

En la cuarta fase denominada *Riesgos* se identifican los riesgos que podrían poner en peligro el proyecto. Una vez identificados se clasifican en una matriz de probabilidad/impacto y se crean acciones de mitigación o contingencia para aquellos riesgos en los que sea necesario. En la *Inception* de Smart Mallorca se identificaron un total de 4 posibles riesgo. Para dos de los riesgos se definieron un plan de contigencia y para los otros 2 se definió un plan de mitigación. Estos planes son los siguientes:

- Acciones de mitigación:
  - Realizar curso de aprendizaje de ARCGIS
  - Realizar curso de aprendizaje de ARKit / AR Core
- Acciones de contingencia:
  - Adaptar aplicación para turismo en Ciudad Real
  - Simular servicios temporalmente y adaptar preferencias

En la quinta y última fase denominada *Story Map*, se procedió a identificar todas las historias de usuario que compondrían el *Product Backlog* y que servirían como base para comenzar el de desarrollo del proyecto.

5.2 SPRINT 0

Tras la realización de la *Inception* y antes de comenzar con el desarollo de la aplicación, se realizó una instalación y configuración de los entornos de desarollo que serían utilizados. El autor recibió formación en materia de realidad aumentada, navegación y mapas. Además, procedió a construir el esqueleto de aplicación móvil basada en arquitectura VIPER que ha sido adaptada durante todo el desarrollo del proyecto.

En la figura 5.2 se incluye el diagrama de secuencia que refleja el flujo de interacción entre los distintos componentes de un módulo en VIPER. En concreto, esta figura modela la interacción entre los componentes de un modulo de acceso a la aplicación.

Como se puede apreciar, es la *View* quien recibe las interacciones por parte del usuario y posteriormente transmite al *Presenter* la necesidad de ejecutar una funcionalidad. El *Presenter* se lo transmite al *Interactor*, el cúal, empaqueta la información y la envía al servicio. Una vez recibida la respuesta, se la transmite al *Presenter* y este se encarga de generar el mensaje que visualizará el usuario y de desencadenar la navegación a la siguiente vista de la aplicación. La responsabilidad de la navegación a otra vista es del *Router*. Por último, la responsabilidad de construir el módulo VIPER de la pantalla a la que se navega es del componente *Router* de esa pantalla.

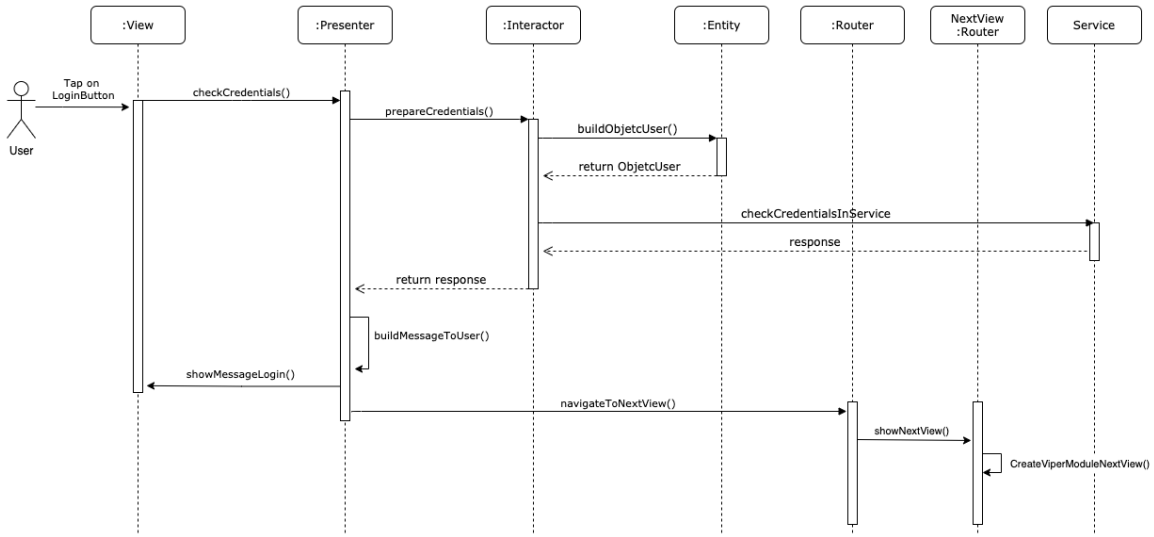


Figura 5.2: Diagrama de secuencia de esqueleto en VIPER

5.3 SPRINT 1

5.3.1 Objetivo

El objetivo establecido para el *Sprint 1* es el de conseguir el acceso y registro en la aplicación. Para ello se ha desarrollado la funcionalidad relacionada con la pantalla de carga de la aplicación, la pantalla *onBoarding* de la aplicación, el acceso con credenciales en la aplicación y a través de las redes sociales *Facebook* y *Twitter*; y por último el registro en la aplicación de Smart Mallorca.

5.3.2 Sprint Planning

En este Sprint las historias de usuario seleccionadas para desarollar e introducidas en el *Product Backlog* del Sprint 1 son las que se reflejan en la tabla 5.1.

Historias de usuario Sprint 1
iOS H0: Splash Screen
iOS H1: onBoarding
iOS H2: Login
iOS H3: Registro

Tabla 5.1: Sprint Backlog (Sprint 1)

5.3.3 Desarrollo por historias de usuario

En este apartado se indican las historias de usuario desarrolladas y sus correspondientes tareas junto a los resultados obtenidos.

5.3.3.1 H0: SplashScreen

En la tabla 5.2 se muestra la tarea que se ha llevado a cabo para satisfascer la historia de usuario. Al ser una historia de usuario que solo implica maquetado de pantallas no se han incluido más tareas relacionadas con la lógica de negocio o la funcionalidad.

Tarea	Estimación
iOS: UI Splash Screen	1h

Tabla 5.2: Tareas de H0: Splash Screen

Esta historia de usuario consiste en elaborar una vista que aparece en el momento en el que se inicia la aplicación y se mantiene durante el tiempo que tarda la aplicación en mostrar la pantalla de onBoarding, que es la siguiente. Con Xcode, la construcción de esta vista no requiere mucho esfuerzo debido a que permite su maquetación de forma independiente y exclusiva del resto de vistas de la aplicación. El resultado obtenido es el que se muestra en la figura 5.3.



Figura 5.3: SplashScreen de Smart Mallorca

5.3.3.2 H1: onBoarding

En la tabla 5.3 se muestran las tareas que se han llevado a cabo para satisfascer la historia de usuario junto a su estimación . Se han incluido dos tareas, una para la maquetación de la vista y otra para desarrollar la lógica de esa pantalla, incuyendo las acciones de navegación que se desencadenan al pulsar sobre alguno de los botones.

Tarea	Estimación
iOS: UI onBoarding	3h
iOS: Negocio onBoarding	6h

Tabla 5.3: Tareas de H1: onBoarding

El objetivo de esta historia de usuario es la elaboración de la pantalla inicial de la aplicación de Smart Mallorca, en ella, el usuario visualiza un *Slider*[1] con la información más relevante de la aplicación. Desde esta pantalla el usuario podrá acceder a la vista de registro para crear una cuenta o la vista de identificación de usuarios si cuenta con credenciales.

[1]Elemento que se coloca generalmente en la página de inicio y que consisten en un pase de diapositivas, que pueden estar animadas o ser imágenes fijas.

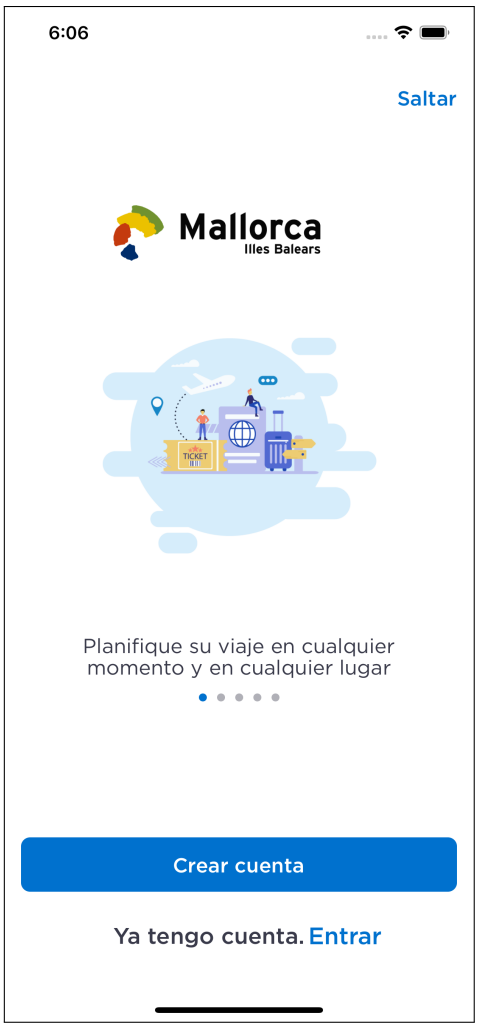


Figura 5.4: onBoarding de Smart Mallorca

5.3.3.3 H2: Login

En la tabla 5.4 se muestran las tareas que se han llevado a cabo para satisfascer la historia de usuario y sus estimaciones. Esta historia de usuario incluye dos tareas, una para realizar la maquetación de la vista y otra para implementar la lógica de navegación y comunicación con el servicio de autenticación de Smart Mallorca.

Tarea	Estimación
iOS: UI Loging	4h
iOS: Negocio Login	9h

Tabla 5.4: Tareas de H2: Login

Esta historia de usuario consiste en una vista (5.5) de acceso a la aplicación dónde el usuario puede introducir sus credenciales (obtenidas anteriormente) para acceder al contenido de la aplicación. En el campo *Constraseña* se incluye un icono con forma de ojo para ofrecer al usuario la posibilidad de ocultar o mostrar su contraseña. Si el usuario selecciona el botón *Entrar* los datos serán enviados al servidor de Smart Mallorca a través de un servicio habilitado para tal efecto, en ellos se comprobarán si son correctos y en función de la respuesta recibida se permitirá el acceso del usuario al contenido.

Por último, esta vista incluye dos botones más, uno para recuperar la contraseña, que en el momento del desarrollo de esta historia no era funcional, y otro para navegar a la vista de registro de Smart Mallorca.

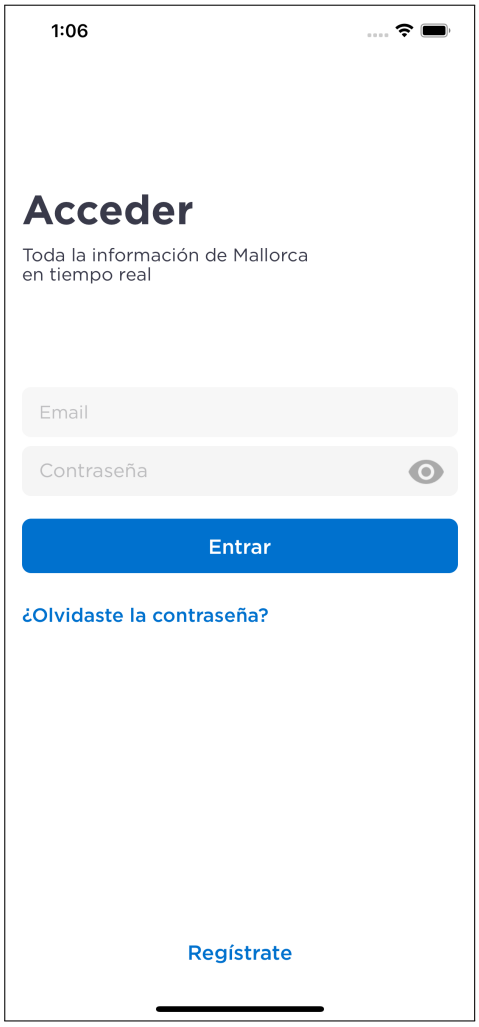


Figura 5.5: Acceso de Smart Mallorca

5.3.3.4 H3: Registro

En la tabla 5.5 se muestran las tareas que se han llevado a cabo para satisfascer la historia de usuario junto a su estimación . Se han incluido dos tareas, una para la maquetación de la vista y otra para desarrollar la lógica de navegación de esta pantalla y la implementación de la comunicación a través del SDK de Facebook y el SDK de Twitter. Debido a que los servicios de registro de usuario de Smart Mallorca no estaban disponibles no se han implementado.

Tarea	Estimación
iOS: UI Registro	3h
iOS: Negocio Registro	10h

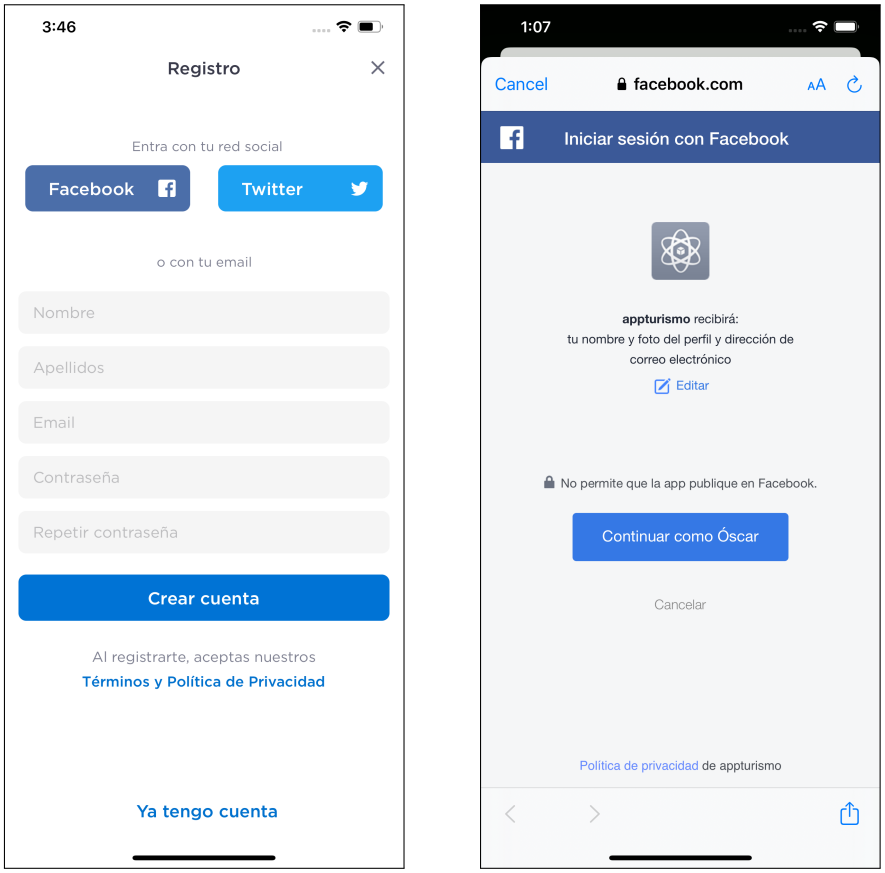
Tabla 5.5: Tareas de H3: Registro

El resultado obtenido de esta historia de usuario consiste en una vista (5.6) en la que el usuario

puede introducir sus datos para registrarse en Smart Mallorca. Adicionalmente se incluyen 2 botones en la parte superior de la vista con los que el usuario puede registrarse en la aplicación con su perfil de Facebook o con su usuario de Twitter. Para registrarse con Facebook se hace uso del SDK de Facebook.

Cuando el usuario pulsa sobre el botón de Facebook se abre una pestaña del navegador donde puede introducir sus datos de acceso de Facebook, si los credenciales son correctos el usuario accederá a la vista principal de la aplicación, en cambio si son erróneos se le notificará con un mensaje emergente.

De la misma forma, si el usuario desea acceder con su perfil de Twitter, puede hacerlo pulsando en el botón de Twitter. Haciendo uso del SDK de Twitter para Swift se gestionará si el usuario y la contraseña introducidos son correctos y en función del resultado se permitirá el acceso al menú principal o se le notificará al usuario que los datos introducidos no son correctos.



(a) Registro en Smart Mallorca **(b)** Registro a través de Facebook

Figura 5.6: Pantallas de registro en Smart Mallorca

5.3.4 Sprint Review

Una vez finalizado el *Sprint 1* se realizó una reunión de revisión para mostrar los resultados obtenidos. Se obtuvo que el 100 % del trabajo planificado fue completado. No se encontraron defectos.

5.3.5 Sprint Retrospective

Al concluir el desarrollo, todo el equipo de Movilidad participó en una reunión de retrospectiva para exponer sus sensaciones respecto de todos los proyectos en los que participan. Debido a las circunstancian del momento y a que todos los componentes del equipo de Movilidad nos encontrábamos teletrabajando se utilizó la herramienta *GoReflect* para realizar de forma online la reunión.

De esta forma cada componente del equipo expuso su opinión sobre lo que había funcionado y lo que no este *Sprint* y se determinó una acción de mejora.

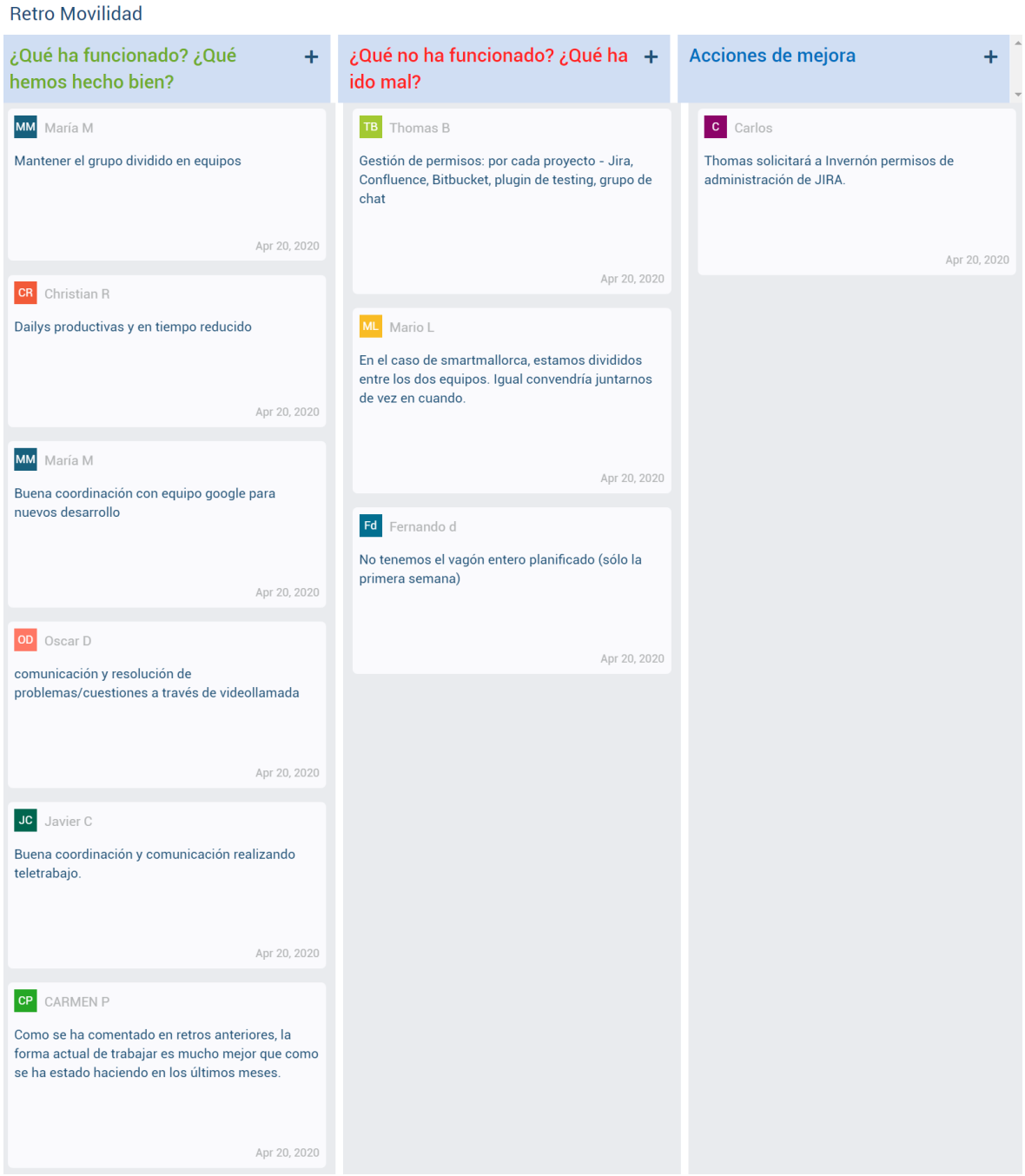


Figura 5.7: Resumen retrospectiva Sprint 1

5.4 SPRINT 2

5.4.1 Objetivo

El objetivo que se ha establecido en el *Sprint 2* es el de desarrollar toda la funcionalidad de Realidad Aumentada de Smart Mallorca. Este objetivo incluye la construcción de una pantalla, que es accesible desde el menú lateral de la aplicación, que implementa a través de la cámara del dispositivo móvil la proyección de los recursos turísticos que rodean al usuario y el despliege de la descripción del recurso si el usuario interacciona con uno de ellos a través de la cámara.

5.4.2 Sprint Planning

En este Sprint las historias de usuario seleccionadas para desarollar e introducidas en el *Product Backlog* del Sprint 2 son las que se reflejan en la tabla 5.6. Para este *Sprint* se han incluido las historias de usuario relacionadas con el ménu de navegación lateral y con la Realidad Aumentada.

Historias de usuario Sprint 2
iOS H4: Menú lateral de navegación
iOS H5: Encuadre de cámara negativo (RA)
iOS H6: Encuadre de cámara positivo (RA)
iOS H7: Contenido Realidad Aumentada (RA)

Tabla 5.6: Sprint Backlog (Sprint 2)

5.4.3 Desarrollo por historias de usuario

En este apartado se indican las historias de usuario que componen el *Sprint Backlog* de este *Sprint* junto a sus correspondientes tareas y estimaciones.

5.4.3.1 H4: Menú lateral de navegación

En la tabla 5.7 se muestran las tareas que el autor ha tenido que llevar a cabo para realizar esta historia de usuario junto a sus estimaciones. Se han incluido dos tareas, una para maquetar la pantalla del menú lateral incluyendo todas las opciones y otra para implementar toda la lógica de la vista, incluyendo el acceso a través de un gesto de deslizamiento lateral de izquierda a derecha.

Tarea	Estimación
iOS: UI Interfaz de Usuario	3h
iOS: Lógica de negocio	5h

Tabla 5.7: Tareas de H4: Menú lateral de navegación

El resultado de esta historia de usuario es una pantalla navegable desde un botón superior del menú principal o a través de un gesto de desplazamiento lateral de izquierda a derecha desde la misma pantalla mencionada. La pantalla del menú de navegación se presentará con una animación de desplazamiento siguiendo la dirección izquierda-derecha. Una vez dispuesta sobre la pantalla del dispositivo del usuario aparecerán un conjunto de opciones relevantes que estarán agrupadas en

esta vista para facilitar su acceso por parte del usuario. En ella se incluye una opción de Realidad Aumentada para facilitar el acceso a esa funcionalidad al usuario.

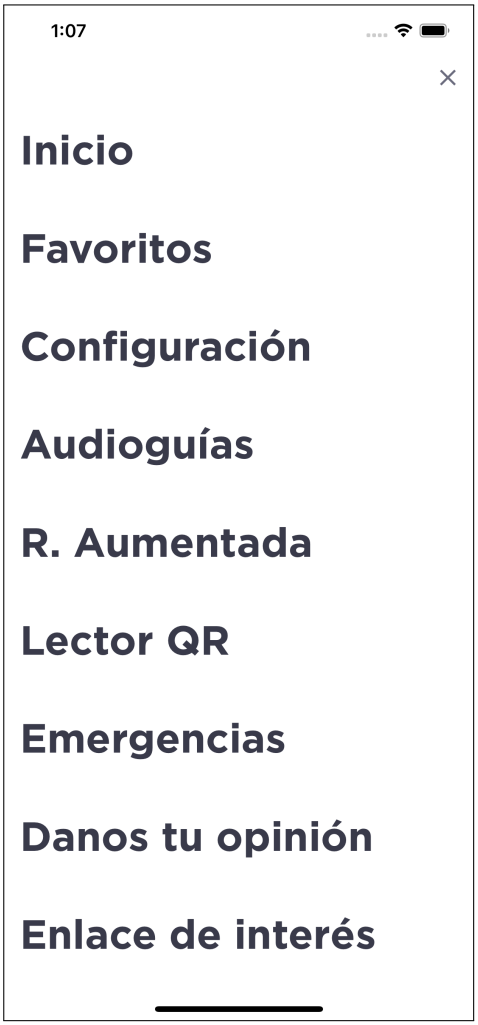


Figura 5.8: Menu lateral de Smart Mallorca

5.4.3.2 H5: Encuadre de cámara negativo (RA)

En la tabla 5.8 se muestran las tas tareas que el autor ha desarrollado para esta historia de usuario y sus estimaciones.

Tarea	Estimación
iOS: Interfaz de Usuario	5h
iOS: Lógica de negocio	15h

Tabla 5.8: Tareas de H5: Encuadre de cámara negativo (RA)

Como resultado de esta historia de usuario se ha obtenido la vista de inicial que implementa la funcionalidad de Realidad Aumentada. Esta vista es accesible desde el menú lateral mencionado en la historia de usuario anterior y en ella el usuario puede visualizar lo que le rodea haciendo uso de la cámara de su dispositivo. En el momento en el que despliega la cámara le aparecerá una ventana

como la que se muestra en la figura 5.9 donde el usuario visualizará el entorno y un mensaje donde se le indica que puede mover la cámara a su alrededor para buscar los recursos turísticos. Además podrá activar la linterna del dispositivo si así lo requiere.



Figura 5.9: Pantalla inicial de realidad aumentada

5.4.3.3 H6: Encuadre de cámara positivo (RA)

En la tabla 5.9 se muestran las tas tareas que el autor ha desarrollado para satisfacer esta historia de usuario y sus estimaciones.

Tarea	Estimación
iOS: Interfaz de Usuario	8h
iOS: Lógica de negocio	12h

Tabla 5.9: Tareas de H6: Encuadre de cámara positivo (RA)

Como resultado de esta historia de usuario se ha añadido funcionalidad a la vista obtenida en la historia de usuario anterior. Esta funcionalidad adicional consiste en proyectar en la cámara del dispositivo los recursos turísticos que rodean al usuario.

El elemento que representa al recurso turístico consiste en un botón con un fondo azul sobre el que se mostrará el número de identificación del recurso turístico. En base a la localización del recurso turístico (conocidad de antemanto), la localización del usuario, la orientación del dispositivo y la distancia que separa al usuario del recurso turístico se proyectará el botón con un tamaño proporcional, teniendo los que se encuentran más cerca al usuario un tamaño mayor que los que se encuentran más alejados. Para acceder a la posición del usuario es necesario acceder a los servicios de localización y GPS del dispositivo.



Figura 5.10: Encuadre positivo de recursos turísticos

5.4.3.4 H7: Contenido Realidad Aumentada (RA)

En la tabla 5.10 se muestran las tas tareas que se han desarrollado para satisfacer esta historia de usuario junto a sus estimaciones.

De nuevo, para esta historia de usuario no se ha creado una nueva vista, si no que sobre la ya existente se ha añadido funcionalidad. Esta funcionalidad consiste en la asociación de una acción provocada por la interacción del usuario con alguno de los botones dispuestos en la pantalla, es decir,

Tarea	Estimación
iOS: Interfaz de Usuario	6h
iOS: Lógica de negocio	12h

Tabla 5.10: Tareas de H7: Contenido Realidad Aumentada (RA)

si el usuario pulsa alguno de los botones que están proyectados en la pantalla de su dispositivo, se mostrará una vista con la información del recurso turístico. Esta información incluye el nombre del recurso turístico junto a su descripción. Si el usuario quiere dejar de visualizar la información del recurso turístico solo debe pulsar en cualquier punto de la pantalla que no sea el recurso seleccionado y la pantalla con la descripción del recurso desaparecerá.



Figura 5.11: Información de un recurso turístico

5.4.4 Sprint Review

Una vez finalizado el *Sprint 2* se realizó una reunión de revisión para mostrar los resultados obtenidos. Se obtuvo que el 100 % del trabajo planificado fue completado. Se acordó modificar el color del marco de encuadre de la cámara en la Realidad Aumentada para que se mantuviese únicamente de color blanco.

5.4.5 Sprint Retrospective

A la conclusión del *Sprint 2*, todo el equipo de Movilidad participó en una reunión de retrospectiva para exponer sus sensaciones respecto de todos los proyectos en los que participan. Debido a que todos los componentes del equipo de Movilidad nos encontrábamos teletrabajando se utilizó la herramienta *GoReflect* para realizar de forma online la reunión.

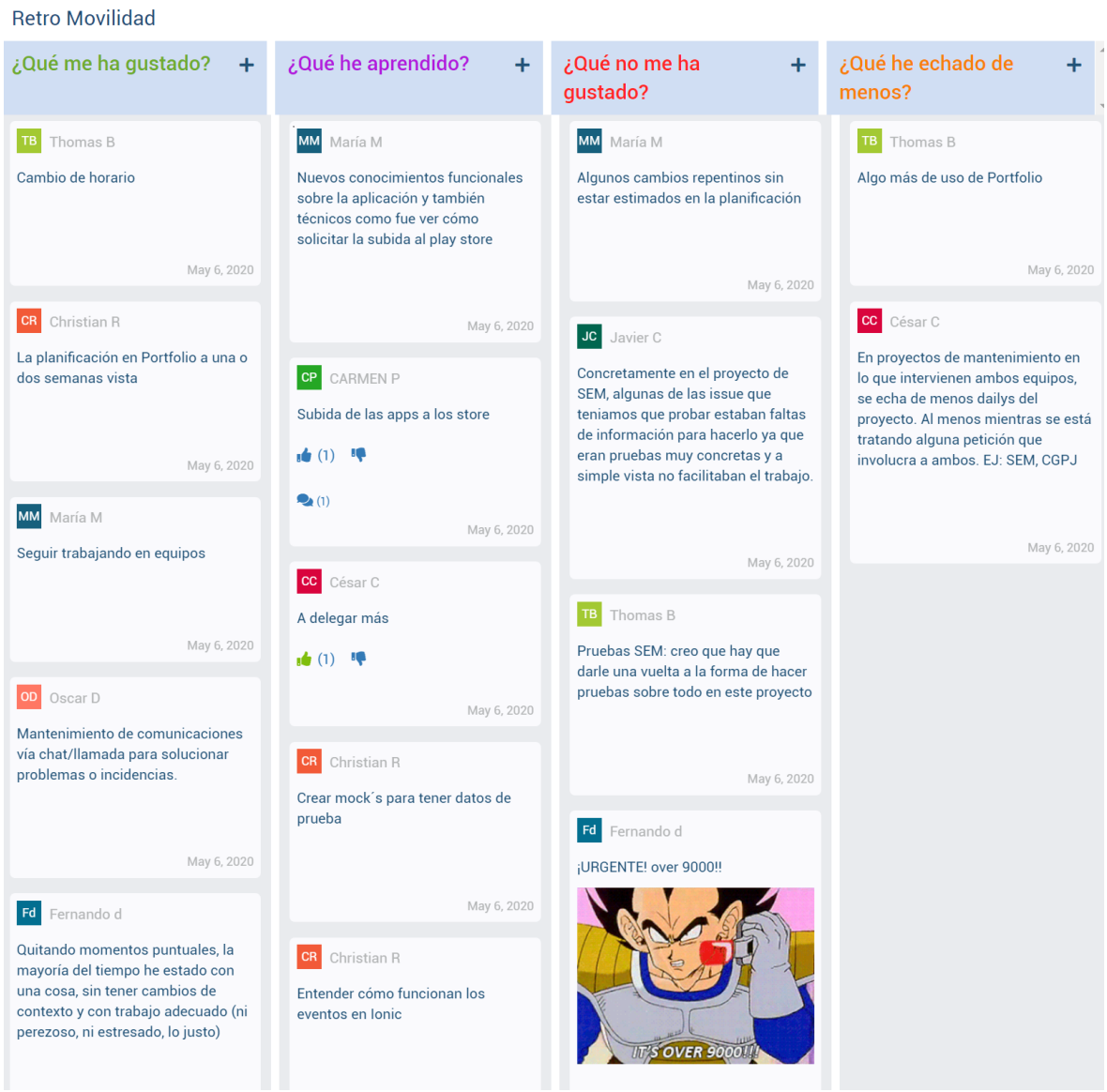


Figura 5.12: Resumen retrospectiva Sprint 2

5.5 SPRINT 3

5.5.1 Objetivo

El objetivo establecido en el *Sprint 3* es el desarrollo de la vista del menú principal de la aplicación y un menú de navegación inferior. Esta vista incluye un buscador de contenido de la aplicación, una vista con información meteorológica y una agrupación de colecciones con todos los recursos

turísticos de la isla de Mallorca agrupados por secciones.

5.5.2 Sprint Planning

En este Sprint se ha seleccionado una única historia para introducir en el *Sprint Backlog* del *Sprint 3*. Esta historia de usuario está relacionada con la pantalla principal de la aplicación también denominada Home, en ella los usuarios podrán visualizar de forma agrupada todos los recursos turísticos de la isla de Mallorca.

Historias de usuario Sprint 3
iOS H8: Home

Tabla 5.11: Sprint Backlog (Sprint 3)

5.5.3 Desarrollo por historias de usuario

En este apartado se indican las historias de usuario desarrolladas y sus correspondientes tareas junto a los resultados obtenidos.

5.5.3.1 H8: Home

En la tabla 5.12 se muestran las tareas que el autor ha tenido que llevar a cabo para realizar esta historia de usuario junto a sus estimaciones. Se han incluido tres tareas, una para implementar el acceso a la pantalla Home desde la pantalla de Login, otra para maquetar esta vista y una última para implementar la lógica de negocio de la vista incluyendo la conexión con los servicios de Smart Mallorca que en el momento de la realización de este *Sprint* se encontraban disponibles.

Tarea	Estimación
iOS: Navegación: Acceso a la Home	5h
iOS: UI Pantalla Home	16h
iOS: Lógica de Negocio	12h

Tabla 5.12: Tareas de H8: Home

El resultado de esta historia de usuario es una pantalla accesible desde la pantalla de Login de la aplicación. En esta pantalla los usuarios visualizarán los recursos turísticos de Mallorca agrupados por categorías (Destinos, Arte y cultura..) y por secciones (Sierra de Tramuntana, Raiguer...). Además, podrán realizar búsquedas de contenido desde un buscador situado en la parte superior y visualizar la información metereológica.

La vista está compuesta con una serie de componentes que paso a detallar a continuación:

- **ScrollView**: Es el componente que engloba al resto y proveé de la funcionalidad para poder deslizar la pantalla de forma vertical.

- **TableView**: Es el componente que estructura y dispone la información de los recursos ordenados por categorías. Cada categoría de la aplicación está dispuesta en una celda del componente *TableView*.

- **CollectionView**: Dentro de cada celda del *TableView* se construye una colección de secciones Este componente permite el usuario deslizar las secciones de cada categoría de forma horizontal en la pantalla. El *CollectionView* a su vez, está compuesto de *CollectionViewCell* que se corresponde con una sección de la aplicación de la que se indica una imagen, su nombre y el número de recursos que contiene.

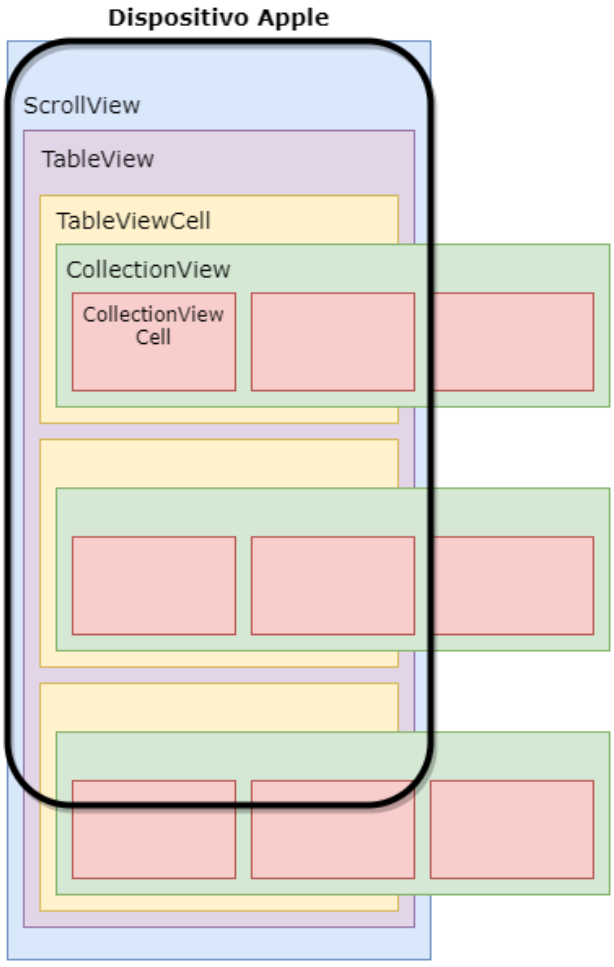


Figura 5.13: Composición pantalla Home

Toda la información de los recursos turísticos es recibida desde los servicio de Smart Mallorca. En el momento de la carga de la pantalla se realizan tres peticiones GET al servicio de Smart Mallorca que devuelve toda la información disponible de los recursos. Las respuestas del servicio son recibidas de forma síncrona, de forma que hasta que no se reciben las tres respuestas en un tiempo determinado no se carga la información en la tabla de categorías.

```
title : "Ocio y diversión"
childs
  0
    title : "Cuevas"
    imgUrl : "https://devsmart.osp.iecisa.net/c/document_library/get_file?uuid=ed086ea4-cadc-045c-45ec-b911ed64bb1a&groupId=37102"
    numRes : 5
    subChilds
      0
        title : "Coves de Campanet"
        imgUrl : "https://devsmart.osp.iecisa.net/c/document_library/get_file?uuid=89cf70be-ad8c-f648-b3cf-1cfa38a2bfe5&groupId=37102"
        address : "Camí de les Coves, S/N - 07310, Campanet"
        location : "{"latitude":39.79286289160394,"longitude":2.967136431061952}"
        classPK : 132779
```


Figura 5.14: Estructura JSON de un recurso turístico

En la figura 5.15 se muestra la pantalla construida en este *Sprint*.

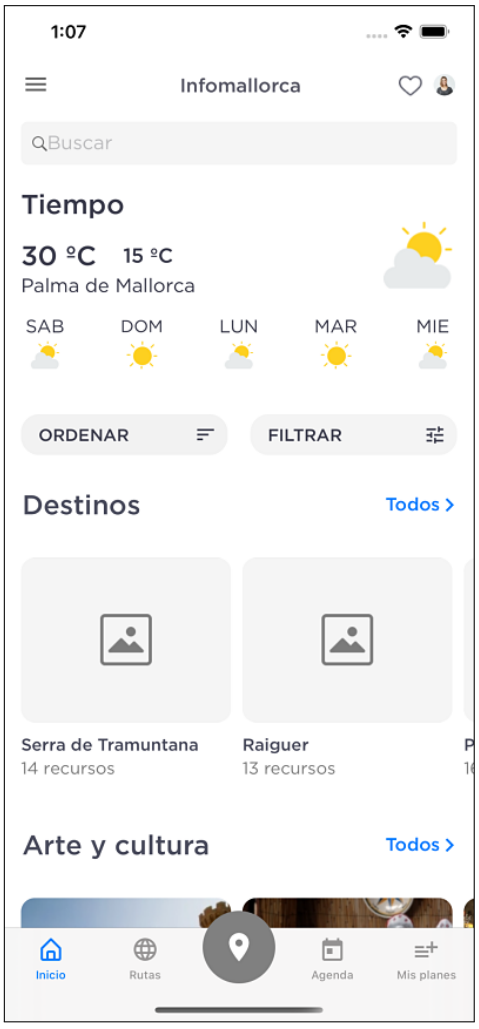


Figura 5.15: Pantalla Home Smart Mallorca

5.5.4 Sprint Review

Una vez finalizado el *Sprint 3* se realizó una reunión de revisión para mostrar los resultados obtenidos. Se obtuvo que el 100 % del trabajo planificado fue completado. Se acordó añadir un acceso al menú lateral desde la pantalla Home en la parte superior izquierda.

5.5.5 Sprint Retrospective

A la conclusión del *Sprint 3* se realizó una reunión telemática de retrospectiva con todos los componentes del equipo de Movilidad utilizando la herramienta *GoReflect* y usando la técnica Mad, Sad, Glad, indicando en la columna Glad aquello que ha ido bien, en la columna Sad lo que nos ha decepcionado y en la columna Mad lo que no ha ido bien o no ha funcionado en absoluto.

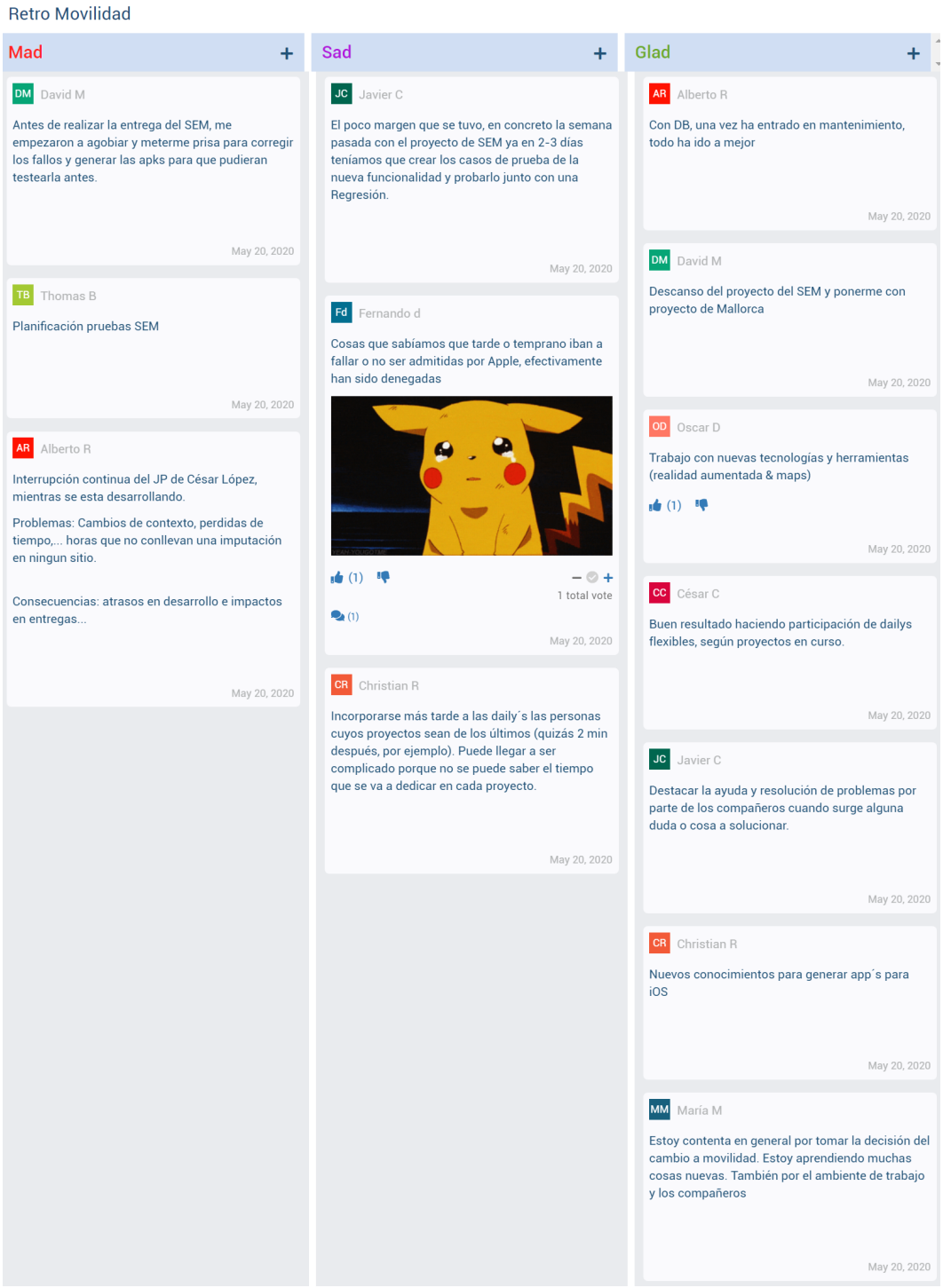


Figura 5.16: Resumen retrospectiva Sprint 3

5.6 SPRINT 4

5.6.1 Objetivo

El objetivo establecido para el *Sprint 4* es el de desarrollar las pantallas de visualización de los recursos turísticos agrupados por categorías y secciones, además de la pantalla de información de un recurso turístico concreto.

En total, las pantallas desarrolladas en este *Sprint* son 3, la primera que permite visualizar los recursos turísticos de una sección, la segunda que muestra los recursos turísticos de una categoría dentro de una sección y los situa en un mapa y la tercera que muestra toda la información disponible de un recurso turístico. Dentro de la pantalla en la que se visualiza toda la información de un recurso turístico se incluyen algunos componentes como un mapa para visualizar la posición del recurso turístico, un gráfico para conocer la ocupación del recurso, un reproductor para visualizar contenido videográfico o un reproductor de audio para las audioguías de un recurso.

5.6.2 Sprint Planning

En este Sprint se han seleccionado tres historias de usuario para introducir en el *Sprint Backlog* del *Sprint 4*. Estas historias de usuario están relacionadas con la visualización de los recursos turísticos de una categoría concreta, con los recursos de una sección concreta dentro de una categoria y con la descripción de un recurso concreto.

Historias de usuario Sprint 4
iOS H9: SubHome 1
iOS H10: SubHome 2
iOS H11: Recurso - Descripción

Tabla 5.13: Sprint Backlog (Sprint 4)

5.6.3 Desarrollo por historias de usuario

En este apartado se indican las historias de usuario desarrolladas y sus correspondientes tareas junto a los resultados obtenidos.

5.6.3.1 H9: SubHome 1

En la tabla 5.14 se muestran tas tareas que se han desarrollado para satisfacer esta historia de usuario junto a sus estimaciones.

Tarea	Estimación
iOS: UI Pantalla de SubHome 1	7h
iOS: Lógica de negocio SubHome 1	10h

Tabla 5.14: Tareas de H9: SubHome 1

Como resultado de esta historia de usuario se ha desarrollado una vista de la aplicación de Smart Mallorca que permite visualizar en una tabla las categorías contenidas en una sección. Con ella, el usuario podrá visualizar la imagen más representativa y el título de las secciones de una categoría. Si el usuario pulsa alguna de las secciones será redirigido a la siguiente vista en la que puede visualizar los recursos turísticos de una sección en forma de colección y en un mapa.



Figura 5.17: subHome 1 de Smart Mallorca

5.6.3.2 H10: SubHome 2

En la tabla 5.15 se muestran las tareas que se han desarrollado para satisfacer esta historia de usuario junto a sus estimaciones.

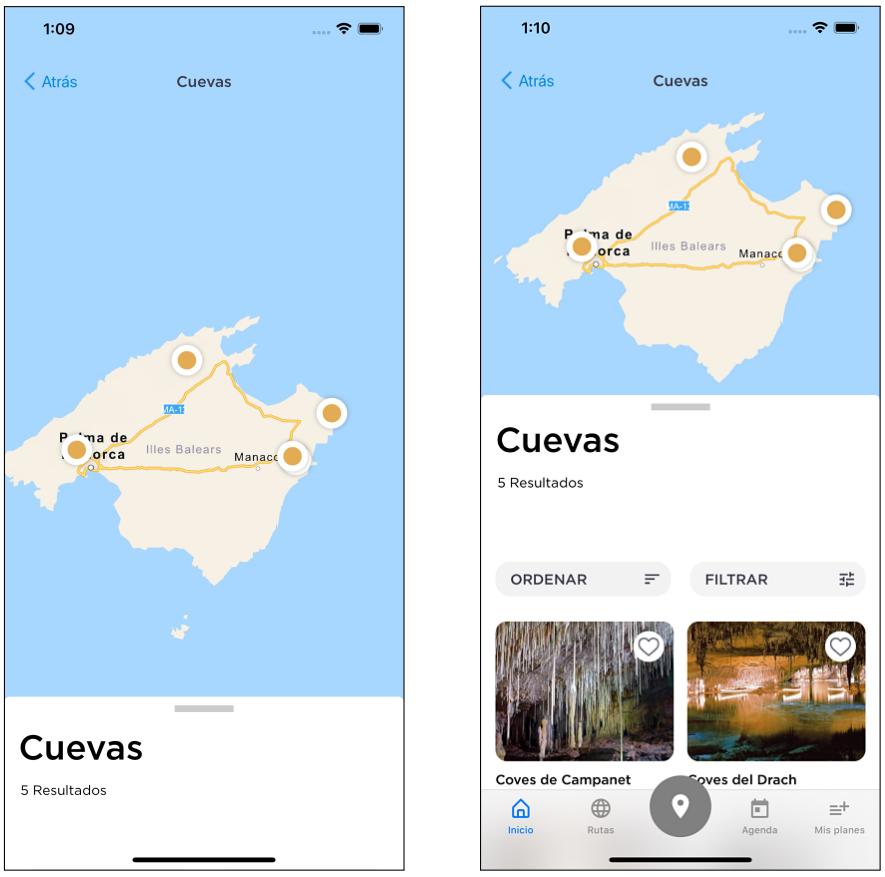
Tarea	Estimación
iOS: UI Pantalla de SubHome 2	16h
iOS: Lógica de negocio SubHome 2	8h

Tabla 5.15: Tareas de H10: SubHome 2

El resultado de esta historia de usuario es una vista que muestra y situa los recursos turísticos de una sección concreta en el mapa. Navegando desde la pantalla anterior, la de SubHome 1, el usuario

puede visualizar los recursos turísticos de una sección concreta.

Para ello se ha construido un mapa que distribuye los puntos en el mapa dependiendo de su posición geográfica recibida del servicio de Smart Mallorca y ajusta el marco del mapa para incluir todos los puntos en la vista inicial. En un principio, la vista inferior aparece contraída para que el usuario pueda visualizar el mapa, pero si el usuario desea puede desplazar la vista de forma vertical hacía la parte superior de la pantalla. Si el usuario despliegla la vista, se visualizará la colección de recursos de una sección y el menú inferior pasará de estar oculto a estar visible.



(a) SubHome 2 con mapa expandido **(b)** SubHome 2 con mapa contraído

Figura 5.18: Pantallas de SubHome 2 en Smart Mallorca

5.6.3.3 H11: Recurso - Descripción

En la tabla 5.16 se muestran las tareas que se han desarrollado para satisfacer esta historia de usuario junto a sus estimaciones.

Tarea	Estimación
iOS: UI Interfaz de Usuario	24h
iOS: Lógica de negocio	14h

Tabla 5.16: Tareas de H11: Recurso - Descripción

Como resultado de esta historia de usuario se ha obtenido una pantalla que permite el usuario

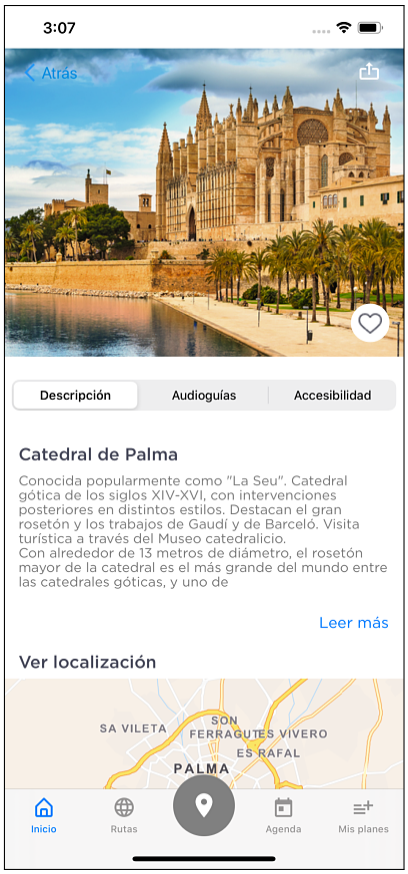
visualizar toda la información disponible de un recurso turístico. En ella, el usuario visualizará la imagen del recurso y un control que permite alternar entre tres vistas con diferentes tipos de información.

La primera, la vista de descripción, incluye una descripción del recurso, un mapa para visualizar el recurso, un componente para conocer el número de personas que ocupan un recurso en momento determinado, un reproductor de video para reproducir contenido audiovisual relacionado con el recurso y más información relevante como pueden ser los horarios, la líneas de comunicación con el transporte público o el contenido relacionado. En la segunda vista, se incluye una tabla con un conjunto de audioguías relacionadas con el recurso. Además, es posible reproducir esas audioguías haciendo uso de un reproductor de audio implementado. Por último, la tercera vista incluye información de accesibilidad del recurso turístico junto a una descripción completa de ese recurso.

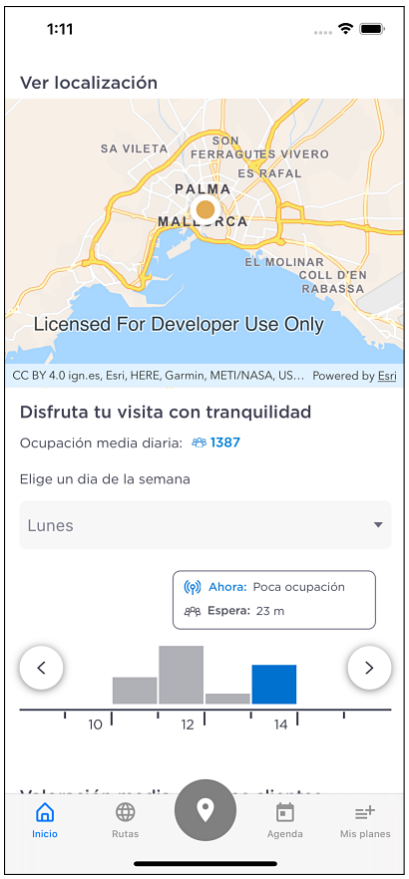


Figura 5.19: Gestión de vistas en pantalla de descripción de recurso

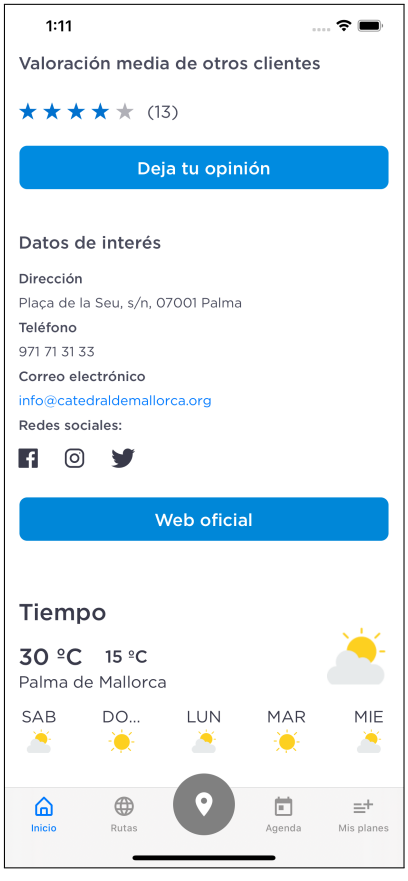
Entrando en detalle, la pestaña de descripción del recurso incluye una porción inicial de la descripción del recurso turístico quu puede ser leída completamente si se pulsa en el botón *Leer más*. Seguidamente aparece un mapa, en el que se situa el recurso, usando para ello la posición geográfica recibida del recurso. El siguiente componente incluido es un componente que estructura y conforma un gráfico para visualizar la información de ocupación de un recurso, es adaptable al número de franjas horarias del recurso y al número de ocupación en cada una de esas franjas. Siguiendo con la vista aparece diferentes componentes con información relevante del recurso. Por último, es posible visualizar un reproductor de video que contendrá contenido audiovisual del recurso turístico, el reproductor incluye controles para pausar y reproducir el contenido, una barra de progreso para conocer la fracción del video que ha sido reproducida y un temporizador con el tiempo transcurrido y el tiempo total del contenido.



(a) Descripción de un recurso



(b) Posición y ocupación de un recurso



(c) Información del recurso

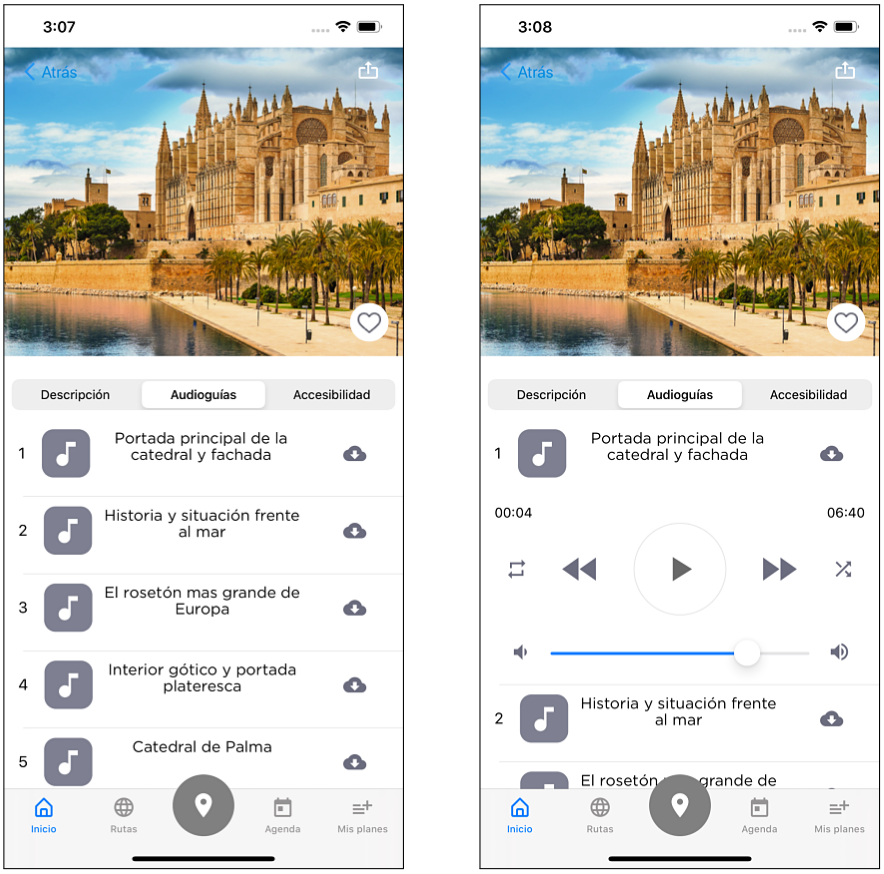


(d) Reproductor de video de un recurso

Figura 5.20: Pestaña descripción de un recurso en Smart Mallorca

Al posicionar el control sobre la opción audioguías se carga la vista que contiene una tabla con todas las audioguías relacionadas con el recurso turístico. Cada celda de la tabla representa una audioguía de la que muestra el nombre. Si el usuario pulsa sobre algunas de las celdas de la tabla de audioguías se desplegará un reproductor de audio.

El reproductor de audio está compuesto por un botón central con el que es posible pausar y reanudar la reproducción de la pista de audio. En los extremos superiores del reproductor se encuentrar dos temporizadores, el de izquierda indica el tiempo escuchado de la pista y el de la derecha el tiempo total de duración de la pista. A la derecha e izquierda del botón central se encuentran dos botones que permiten retrasar o adelantar la pista diez segundos. Por último, en la parte inferior del reproductor se encuentra una barra de progreso que permite modificar el volumen de la audioguía.



(a) Tabla de audioguías **(b)** Audiguía de un recurso expandida

Figura 5.21: Vista de audioguías de un recurso turístico

Por último, si el usuario posiciona el control sobre la opción accesibilidad, visualizará la vista de accesibilidad, que incluye la descripción completa del recurso turístico y una lista de contenido relacionado con la accesibilidad y la adaptación de los accesos del recurso turístico.



Figura 5.22: onBoarding de Smart Mallorca

5.6.4 Sprint Review

Una vez finalizado el *Sprint 4* se realizó una reunión de revisión para mostrar los resultados obtenidos. Se obtuvo que el 100 % del trabajo planificado fue completado. Se acordó modificar una característica del reproductor de video del recurso para que los controles del reproductor no desaparecieran por primera vez hasta que el usuario no pulsase en el interior de la vista del reproductor.

5.6.5 Sprint Retrospective

A la conclusión del *Sprint 4*, todo el equipo de Movilidad participó en la reunión de retrospectiva para exponer sus sensaciones respecto de todos los proyectos en los que participan. Debido a que todos los componentes del equipo de Movilidad nos encontrábamos teletrabajando se utilizó la herramienta *GoReflect* para realizar de forma online la reunión.

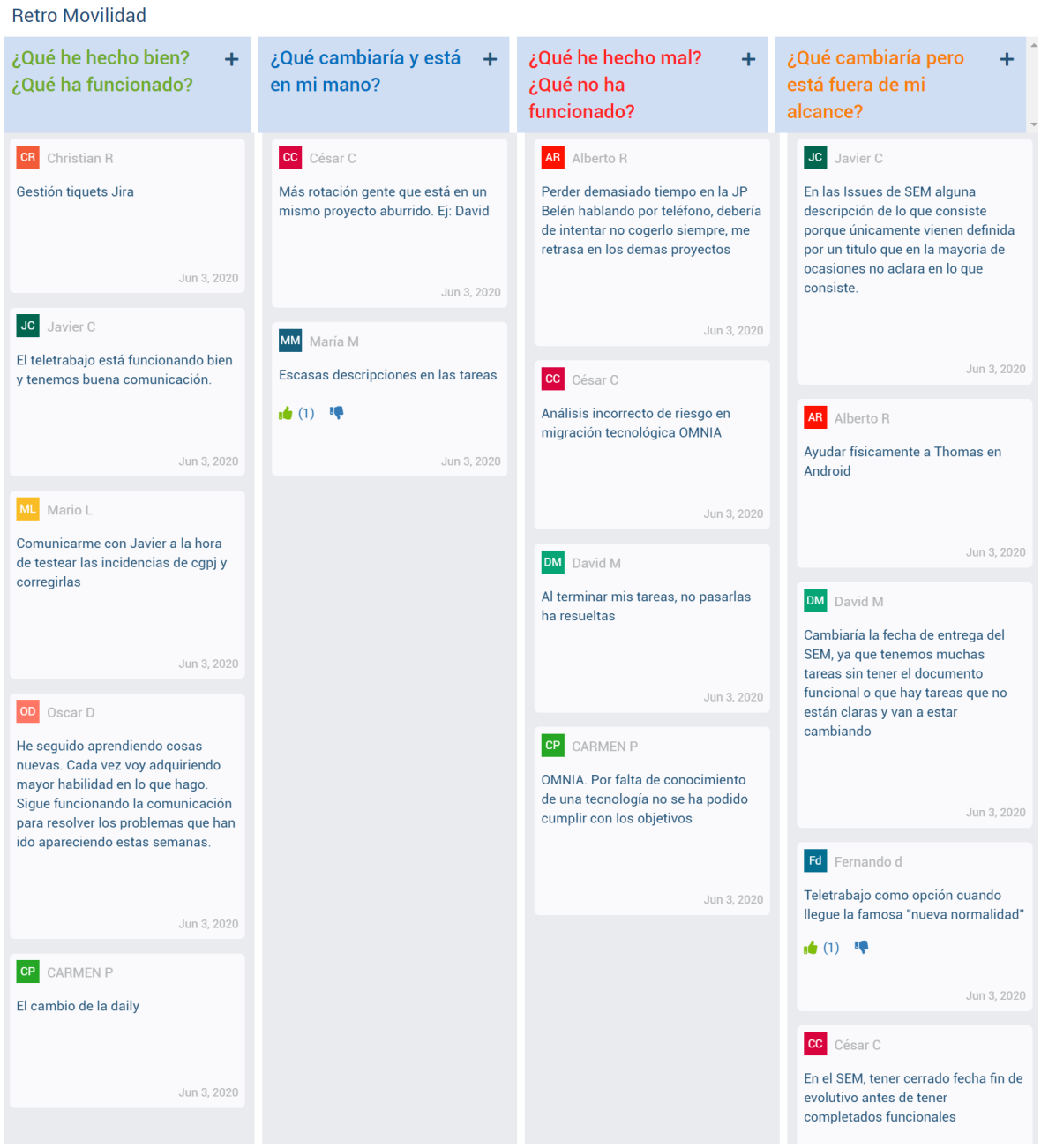


Figura 5.23: Resumen retrospectiva Sprint 4

5.7 SPRINT 5

5.7.1 Objetivo

El objetivo establecido para el *Sprint 5* es el desarrollo del lector de códigos QR, la pantalla para mostrar los recursos cercanos al usuario en base a su ubicación actual, la búsqueda de recursos turísticos en el mapa por texto o por voz, y las pantallas de registro en Smart Mallorca, que permitan al usuario personalizar sus preferencias en cuanto a la recepción de notificaciones y el envio de su posición geográfica.

5.7.2 Sprint Planning

En este Sprint se han seleccionado tres historias de usuario para introducir en el *Sprint Backlog* del *Sprint 4*. Estas historias de usuario están relacionadas con la visualización de los recursos turísticos de una categoría concreta, con los recursos de una sección concreta dentro de una categoría y con la descripción de un recurso concreto. En la tabla 5.17 se muestran las historias de usuario abordadas en este *Sprint*.

Historias de usuario Sprint 5
iOS H12: Pantalla de escaneado QR
iOS H13: Preferencias registro
iOS H14: Mapa de recursos cercanos

Tabla 5.17: Sprint Backlog (Sprint 5)

5.7.3 Desarrollo por historias de usuario

En este apartado se indican las historias de usuario desarrolladas y sus correspondientes tareas junto a los resultados obtenidos.

5.7.3.1 H12: Pantalla de escaneado QR

En la tabla 5.18 se muestran tas tareas que se han desarrollado para satisfacer esta historia de usuario junto a sus estimaciones.

Tarea	Estimación
iOS: Interfaz de usuario: Pantalla	5h
iOS: Acceso desde menú lateral	4.5h
iOS: Lógica de negocio de escaneado QR	6h

Tabla 5.18: Tareas de H12: Pantalla de escaneado QR

Como resultado de esta historia de usuario se ha desarrollado una pantalla de la aplicación que permite, haciendo uso de la cámara del dispositivo, el escaneado de códigos QR. Esta pantalla, que es accesible desde el menú lateral, carga el contenido asociado a un determinado código QR en el navegador del dispositivo. Para un correcto escaneado del código QR, es necesario que este se encuentre encuadrado dentro del marco con los bordes blancos que incluye la vista.



Figura 5.24: Lector de códigos QR

5.7.3.2 H13: Preferencias registro

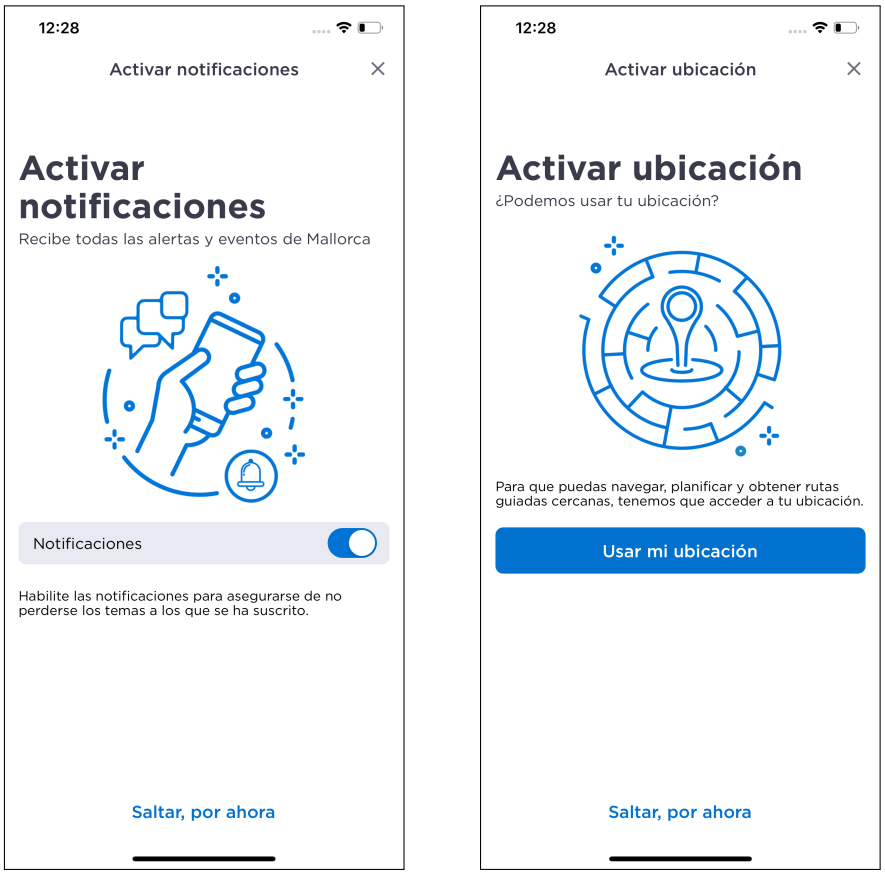
En la tabla 5.19 se muestran tas tareas que se han desarrollado para satisfacer esta historia de usuario junto a sus estimaciones.

Tarea	Estimación
iOS: Interfaz de usuario notificaciones	3h
iOS: Lógica de negocio notificaciones	4h
iOS: Interfaz de usuario ubicación	2h
iOS: Lógica de negocio ubicación	2h

Tabla 5.19: Tareas de H13: Preferencias registro

El resultado de esta historia de usuario consiste en 2 pantallas que permiten al usuario definir sus preferencias en relación a la visualización de notificaciones push y la ubicación. Estas pantallas aparecen cuando un usuario realiza un registro en la aplicación Smart Mallorca. Tanto si el usuario acepta o deniega la visualización de notificaciones y la activación de la ubicación, sus preferencias quedan registradas. Es posible que el usuario deniege en el momento del registro la activación de la ubicación pero si más adelante desea utilizar la funcionalidad de Realidad Aumentada o mapas sería

necesario que aceptase su uso.



(a) Activar notificaciones **(b)** Activar ubicacióna

Figura 5.25: Opciones de registro en Smart Mallorca

5.7.3.3 H14: Mapa de recursos cercanos

En la tabla 5.20 se muestran tas tareas que se han llevado a cabo para satisfacer esta historia de usuario junto a sus estimaciones.

Tarea	Estimación
iOS: Interfaz de usuario mapa	8h
iOS: Lógica de negocio mapa	12h
iOS: Buscador y localización	12h

Tabla 5.20: Tareas de H14: Mapa de recursos cercanos

El resultado de esta historia de usuario consiste en una pantalla que incluye un mapa en el que se puede visualizar la posición geográfica del usuario en base a su localización. Además, si cerca de la posición del usuario se encuentra algún recurso turístico, este será proyectado en el mapa. La vista incluye un buscador con el cúal es posible introducir el nombre de algún recurso para comprobar si existiese. Si el usuario introduce el nombre de algún recurso que existe, automáticamente el mapa cambiará el punto de vista hacia la posición de ese recurso como se muestra en la figura 5.27. También es posible realizar la búsqueda de contenido con la voz del usuario haciendo uso del microfono del dispositivo.

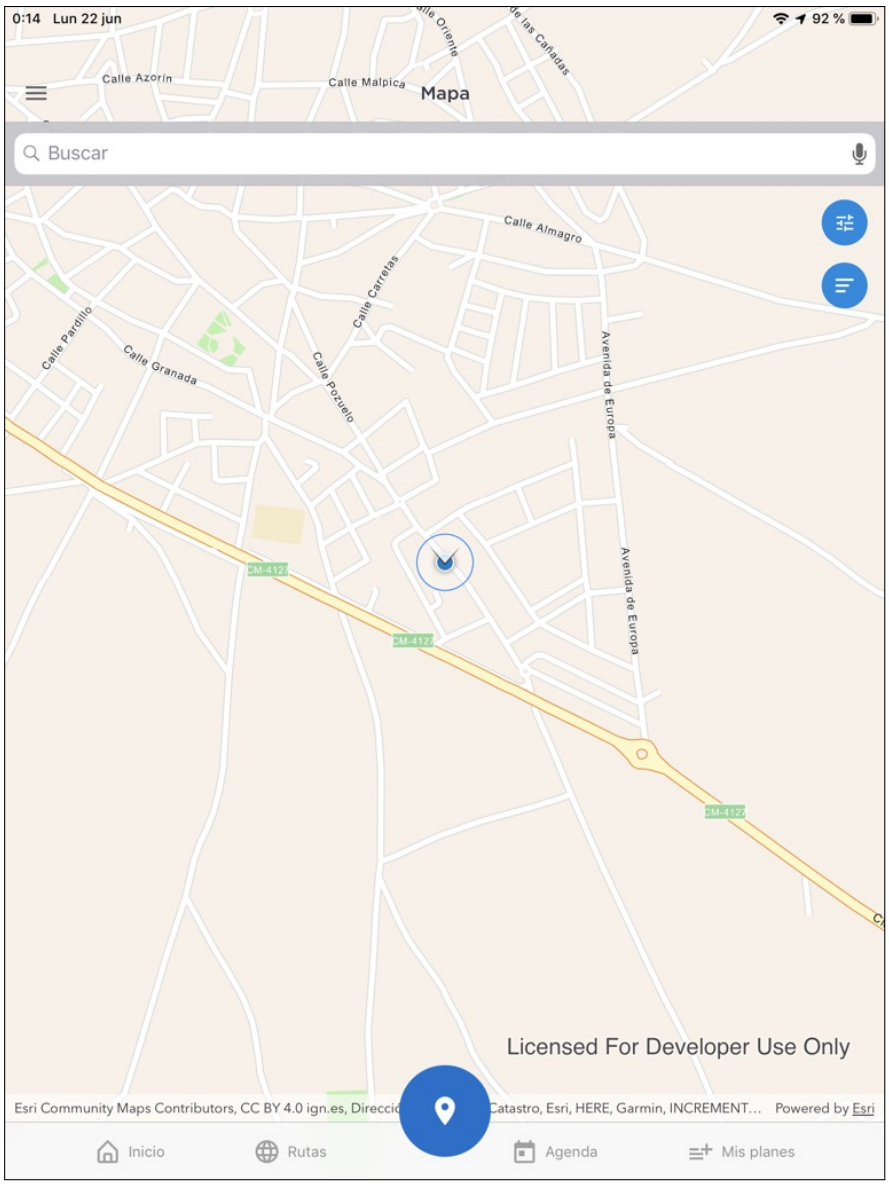


Figura 5.26: Ubicación actual en mapa

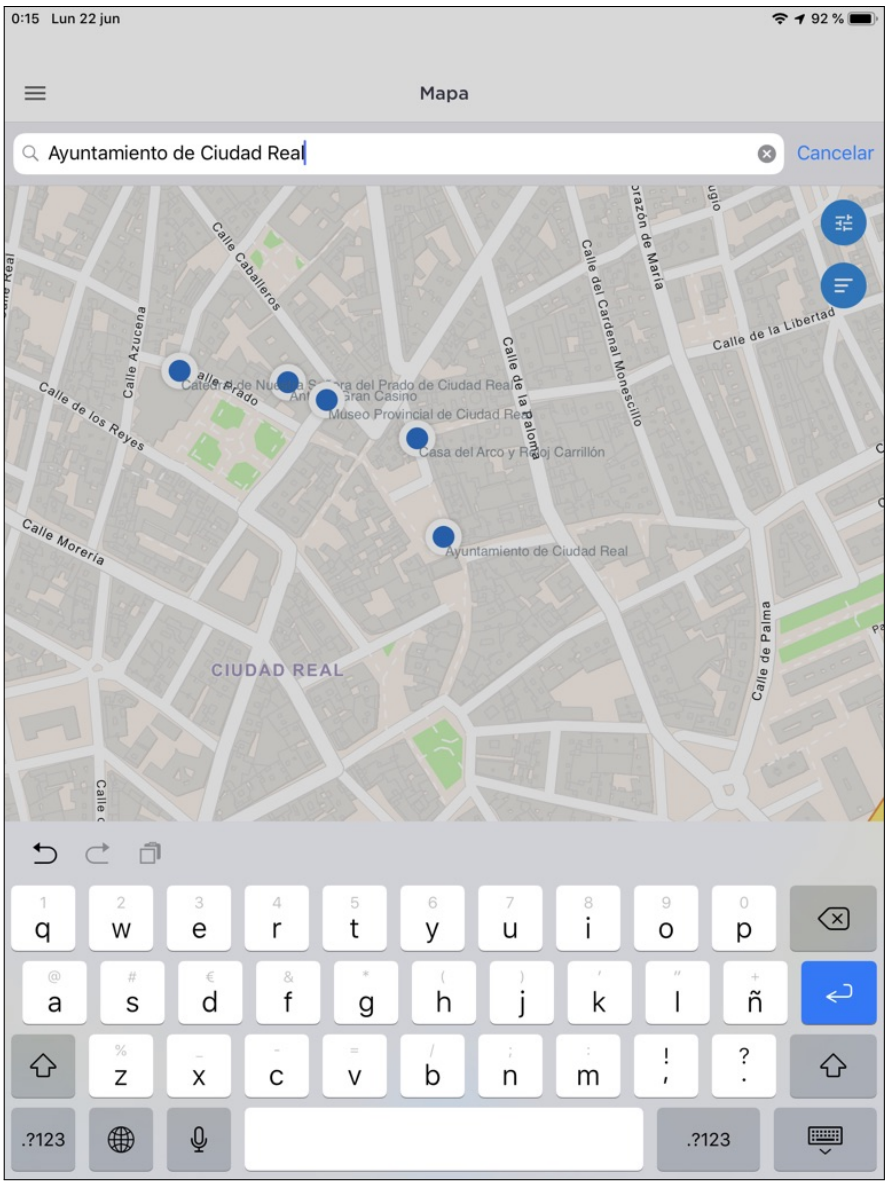


Figura 5.27: Resultados de búsqueda en mapa

5.7.4 Sprint Review

Una vez finalizado el *Sprint 5* se realizó una reunión de revisión para mostrar los resultados obtenidos. Se obtuvo que el 100 % del trabajo planificado fue completado. Se acordó añadir un componente que permitiese al usuario identificar si la búsqueda mediante voz está activada o desactivada.

5.7.5 Sprint Retrospective

A la conclusión del *Sprint 5*, todo el equipo de Movilidad participó en una reunión de retrospectiva para exponer sus sensaciones en relación a los proyectos en los que participaron durante este *Sprint*. Debido a que todos los componentes del equipo de Movilidad nos encontrábamos teletrabajando se utilizó la herramienta *Jamboard* para realizar de forma online la reunión.

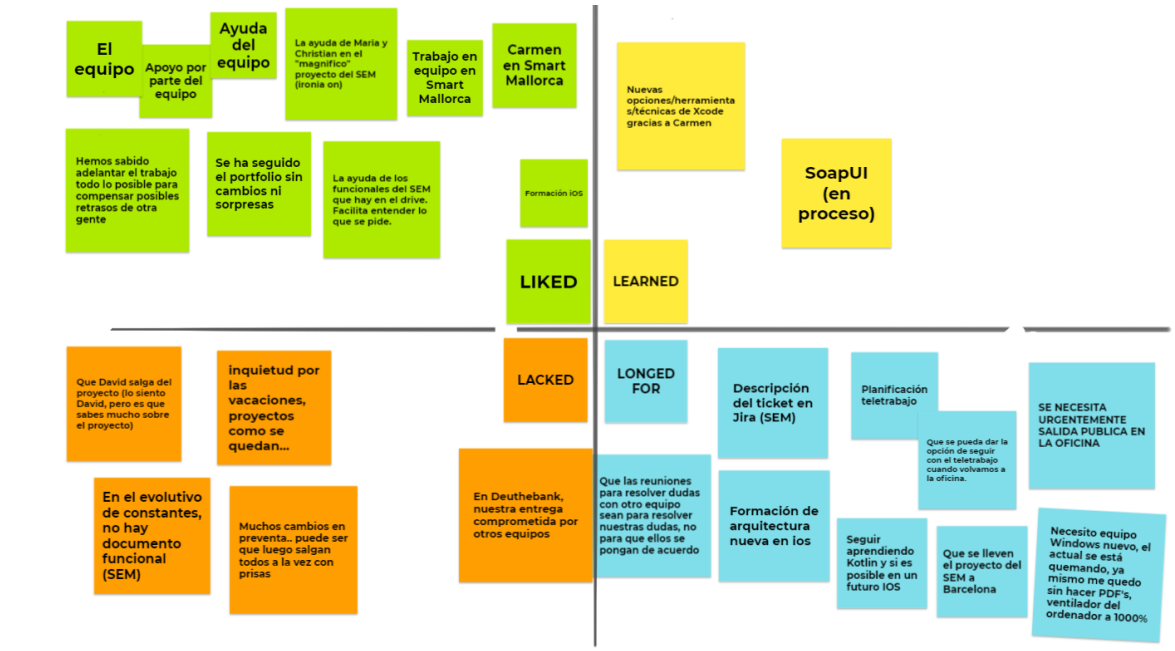


Figura 5.28: Resumen retrospectiva Sprint 5

5.8 INVESTIGACIÓN

En este apartado se explican y se exponen las tareas de investigación que el autor tuvo que llevar a cabo antes de la realización del proyecto.

5.8.1 Introducción

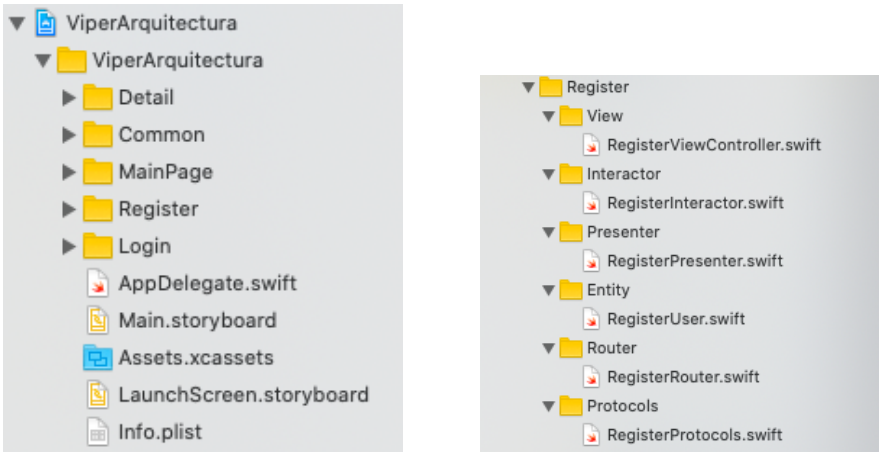
En las etapas anteriores al comienzo del proyecto, el autor tuvo que realizar una labor de investigación en cuestiones de vital importancia para su desarrollo. Durante este tiempo el autor realizó un serie de estudios sobre la arquitectura VIPER, que fue la elegida para desarrollar el proyecto debido a que ofrecía una facil estructuración en proyectos de gran tamaño. El objetivo final de esta labor, sería la de desarrollar una arquitectura base que sirviese como base o esqueleto para el desarrollo posterior.

Adicionalmente, el autor realizó una labor de investigación en el framework de Realidad Aumentada ARkit[3] para su posterior implementación en el proyecto de Smart Mallorca. Esta investigación se llevó a cabo como medida de mitigación frente al riesgo *"Desconocimiento Framework Realidad Aumentada"* identificado en la *Inception*. De la misma forma se dedicó un periodo de tiempo al estudio del framework de navegación y mapas ArcGIS[2] como acción de mitigación al riesgo *"Desconoci-*

miento Framework Mapas"

5.8.2 Arquitectura VIPER

Para la construcción de la arquitectura base siguiendo el patrón VIPER se elaboró un proyecto con distintos módulos que simulasen la autenticación y el registro de una aplicación móvil para iPhone. En la figura 5.29 se muestra la estructura del proyecto y la organización de un módulo VIPER.



(a) Estructura proyecto base VIPER **(b)** Estructura módulo VIPER

Figura 5.29: Proyecto esqueleto VIPER

Tomando como referencia el módulo *Register* del proyecto, se explicará como se estructura un módulo VIPER.

Cada componente del módulo implementa una interfaz que indica que funciones y parámetros debe implementar cada clase para que puedan ser accesibles desde una clase externa. El componente encargado de definir estas interfaz está contenido en la carpeta *Protocols* que aunque no se trate de un módulo propio de la arquitectura VIPER, es importante definirlo para mantener una buena organización.

Como se muestra en la figura 5.30, cada clase implementa una serie de parámetros y funciones. El componente *View* implementado a través de la interfaz *RegisterViewProtocol* mantiene una única referencia al *Presenter* al que deberá comunicar las acciones del usuario.

El componente *Presenter* está representado a través del protocolo *RegisterPresenterProtocol* y mantiene referencias con el *View*, para pasarle datos para mostrar al usuario, con el *Router* para comunicarle una acción de navegación entre vistas y el *Interactor* para solicitar llamadas a servicios. El componente *Interactor* mantiene una única referencia con el *Presenter* para comunicarle los resultados de las llamadas a los servicios.

Por último, el componente *Router* encargado de la navegación entre vistas y la creación del módulo VIPER no mantiene ninguna referencia porque no es necesario, unicamente implementa los métodos para realizar las transicciones hacia otras vistas.

```
import UIKit

protocol RegisterPresenterProtocol : class{

    var router: RegisterRouterProtocol? {get set}
    var view: RegisterViewProtocol? {get set}
    var interactor: RegisterInteractorProtocol? {get set}

    func changeViewToLogin(view: UIViewController)
    func checkRegister(nombre: String?, apellidos: String?, usuario: String?, correo: String?, contraseña: String?)
}

protocol RegisterViewProtocol : class{
    var presenter: RegisterPresenterProtocol? {get set}

    func showMessage(title: String?,message :String?)
}

protocol RegisterRouterProtocol : class {
    func pushToLogin(view: UIViewController)
    func createRegisterModule(with Register: RegisterViewController)
}

protocol RegisterInteractorProtocol: class{
    var presenter: RegisterPresenterProtocol? {get set}

    func checkRegister(nombre: String?, apellidos:String?,usuario:String?,correo:String?,contraseña:String?)
    -> [String: String]
}
```

Figura 5.30: Protocolos de comunicación VIPER

El componente *Router* que implementa el protocolo *RegisterRouterProtocol* es el primer componente en ser instanciado a la hora crear el módulo *Register* debido a que el método *createRegisterModule* es ejecutado desde otro *Router* de otro módulo VIPER. El método *createRegisterModule* es el encargado de instanciar y enlazar todas las referencias que existen en un módulo VIPER. Por último, la función*pushToLogin* implementa la funcionalidad para descartar la vista de Registro este módulo, en caso de que el usuario desencadene la acción para volver a la pantalla anterior.

```
import UIKit

class RegisterRouter: RegisterRouterProtocol {

    class func createRegisterModule(with Register: RegisterViewController) {
        let presenter = RegisterPresenter()
        Register.presenter = presenter
        Register.presenter?.view = Register
        Register.presenter?.router = RegisterRouter()
        Register.presenter?.interactor = RegisterInteractor()
        Register.presenter?.interactor?.presenter = presenter
    }


    func pushToLogin(view: UIViewController) {

        view.navigationController?.popViewController(animated: true)
        view.dismiss(animated: true, completion: nil)

    }

}
```

Figura 5.31: Módulo Router en VIPER

El componente *View* implementa el protocolo *RegisterViewControllerProtocol* y desde la referencia que mantiene al componente *Presenter* es posible comunicar que el usuario ha desencadenado una acción desde la vista. Para ello, realiza una llamada al método *checkRegister* definido en la interfaz del *Presenter* pasándole la información obtenida de los campos de texto.

```
import UIKit

class RegisterViewController: UIViewController, RegisterViewProtocol {

    @IBOutlet weak var nombreTF: UITextField!
    @IBOutlet weak var apellidosTF: UITextField!
    @IBOutlet weak var usuarioTF: UITextField!
    @IBOutlet weak var correoElectronicoTF: UITextField!
    @IBOutlet weak var contraseñaTF: UITextField!
    @IBOutlet weak var btnRegistrarse: UIButton!

    var presenter : RegisterPresenterProtocol?

    override func viewDidLoad() {
        super.viewDidLoad()
    }
    @IBAction func btnRegister(_ sender: Any) {
        presenter?.checkRegister(nombre: nombreTF.text,
                                 apellidos: apellidosTF.text,
                                 usuario: usuarioTF.text,
                                 correo: correoElectronicoTF.text,
                                 contraseña: contraseñaTF.text)
    }
```

Figura 5.32: Módulo View en VIPER

El componente *Presenter* que implementa el protocolo *RegisterPresenterProtocol* realiza una llamada a la función *checkRegister* del *Interactor*. Desde esta clase se produciría la navegación a otra vista si el usuario o los resultados provenientes del *Interactor* lo indicasen.

```
import UIKit

class RegisterPresenter: RegisterPresenterProtocol{

    var router: RegisterRouterProtocol?
    weak var view: RegisterViewProtocol?
    var interactor: RegisterInteractorProtocol?

    func checkRegister(nombre: String?, apellidos: String?, usuario: String?, correo: String?, contraseña: String?){
        let response = interactor?.checkRegister(nombre: nombre, apellidos: apellidos, usuario: usuario, correo: correo, contraseña: contraseña)

        if response?["Type"] == "Success"{
            router?.pushToLogin(view: view as! UIViewController)
        }else{
            view?.showMessage(title: response?["Title"], message: response?["Content"])
        }
    }
```

Figura 5.33: Módulo Presenter en VIPER

Por último, el componente *Interactor* implementa el protocolo *RegisterInteractorProtocol* y mantiene una referencia al *Presenter* para comunicarle los resultados obtenidos de la llamada al servicio de registro simulado. Esta clase es la encargada de encapsular los datos recibidos y formar el objeto que se debe enviar al servidor.

```
import UIKit

class RegisterInteractor: RegisterInteractorProtocol {

    weak var presenter: RegisterPresenterProtocol?

    func checkRegister(nombre: String?, apellidos:String?,usuario:String?,correo:String?,contraseña:String?)
        -> [String: String]{

        let usuarioRegistro = RegisterUser(nombre: nombre,apellidos: apellidos, usuario: usuario, correo: correo, contraseña: contraseña)

        /* Simulacion llamada a servicio*/
        let response = RegisterService.checkRegisterService(registerUser: usuarioRegistro)
        return response
    }
}
```

Figura 5.34: Módulo Interactor en VIPER

5.8.3 Realidad Aumentada

Para implementar la funcionalidad de Realidad Aumentada en la aplicación de Smart Mallorca, fue necesario realizar una labor de investigación y formación en el framework de ARKit[3]. Con esta labor el autor construyó una serie de prototipos que implementaban la tecnología de Realidad Aumentada en dispositivos iOS. Es por ello, que para reflejar la labor de investigación realizada, en este apartado se explica como conformar una vista que implemente Realidad Aumentada usando el lenguaje Swift.

Con ARKit todas las vistas deben recibir un fichero de configuración, en la figura 5.35 se muestra este fichero de configuración denominado *configuration* con el cual, es posible habilitar la disposición del eje de coordenadas 3D de una vista que implemente Realidad Aumentada o indicar que la detección de planos sea visible para el usuario. Este fichero es añadido al objeto *SceneView* que es el objeto que mantiene la referencia a la pantalla de realidad aumentada.

```
override func viewWillAppear(_ animated: Bool) {
    super.viewWillAppear(animated)

    // Create a session configuration
    let configuration = ARWorldTrackingConfiguration()

    // Run the view's session
    sceneView.session.run(configuration)
}

override func viewWillDisappear(_ animated: Bool) {
    super.viewWillDisappear(animated)

    // Pause the view's session
    sceneView.session.pause()
}
```

Figura 5.35: Configuración pantalla de Realidad Aumentada

Seguidamente, en la función *viewDidLoad* que es la que se ejecuta al cargar cualquier vista en una aplicación móvil programada en Swift, se crea una objeto de tipo *SCNScene*, el cual será la vista padre que contendrá en su interior a todos los nodos a mostrar en la cámara del dispositivo. Posteriormente, se ejecuta la función *createGloble* que devuelve 2 nodos, que serán añadidos a la escena.

```
import UIKit
import SceneKit
import ARKit

class ViewController: UIViewController, ARSCNViewDelegate {

    @IBOutlet var sceneView: ARSCNView!

    override func viewDidLoad() {
        super.viewDidLoad()

        // Set the view's delegate
        sceneView.delegate = self

        // Show statistics such as fps and timing information
        sceneView.showsStatistics = true

        // Create a new scene
        let scene = SCNScene()
        let position = SCNVector3(0,0,2)
        let globe = createGlobe(at: position)
        scene.rootNode.addChildNode(globe)

        // Set the scene to the view
        sceneView.scene = scene
    }
```

Figura 5.36: Creación de escena SCNScene

En la función *CreateGlobe*, que recibe por parámetro una coordenada en tres dimensiones, se crean dos nodos. El primer nodo contiene en su interior una esfera de 30 centímetros de radio como indica la clase *SCNSphere* y su contorno estará relleno de una imagen anteriormente cargada. Esta esfera debe ser añadida a un objeto *SCNNode* que será el objeto hijo que recibirá la *SCNScene* antes declarada. La posición del primer nodo creado será la recibida por parámetro.

Para la segunda esfera construida, el procedimiento es el mismo, pero la posición será una posición definida mediante el objeto *SCNVector3* utilizado para crear una posición de tres dimensiones. Como se aprecia, este segundo nodo es añadido como nodo hijo (otherSphereNode) del nodo padre (sphereNode), por lo tanto la posición indicada para el nodo *otherSphereNode* tendrá como origen de referencia la posición del nodo *SphereNode*.

```
func createGlobe(at position: SCNVector3) -> SCNNode{
    let sphere = SCNSphere(radius: 0.3)
    sphere.firstMaterial?.diffuse.contents = UIImage(named: "art.scnassets/earth.jpeg")
    let sphereNode = SCNNode(geometry: sphere)
    sphereNode.position = position

    let otherSphere = SCNSphere(radius: 0.07)
    otherSphere.firstMaterial?.diffuse.contents = UIImage(named: "art.scnassets/tierra2.jpg")
    let otherSphereNode = SCNNode(geometry: otherSphere)
    otherSphereNode.position = SCNVector3(0.3, 0.3, 0.2)
    sphereNode.addChildNode(otherSphereNode)

    return sphereNode
}
```

Figura 5.37: Creación de nodos SCNNode

5.8.4 Navegación y mapas

Para labor de investigación sobre mapas y navegación llevada a cabo, se ha seguido la misma metodología de trabajo que en la investigación sobre Realidad Aumentada. Se ha llevado a cabo una investigación sobre el framework de mapas ArcGIS[2] realizando para ello una serie de prototipos que incluían funcionalidad que posteriormente debería ser implementada en la aplicación Smart Mallorca. Algunos de los prototipos llevados a cabo incluían disposición de mapas, navegación en mapas, geolocalización, construcción de figuras sobre mapas... En este apartado se procede a explica como conformar una vista de navegación a través de mapas que incluyen figuras.

En primer lugar es necesario importar el SDK ArcGIS para poder acceder a toda la funcionalidad que proveé. En esta clase se incluye una referencia a la vista de la aplicación, de tipo *AGSMapView* sobre la que se trabajará posteriormente. Además, se incluye un objeto de tipo *AGSGraphicsOverlay* que permitirá la proyección de figuras sobre el mapa. En la clase *ViewDidLoad* se realiza una llamada a los métodos *setupMap* y *createGraphics* que construirán el mapa y las figuras respectivamente.

```
import UIKit
import ArcGIS

class ViewController: UIViewController {

    @IBOutlet var mapView: AGSMapView!
    private var graphicsOverlay: AGSGraphicsOverlay!

    override func viewDidLoad() {
        super.viewDidLoad()
        setupMap()
        createGraphics()

    }
```

Figura 5.38: Declaración de variables AGSMapView y AGSGraphicsOverlay

En segundo lugar, con la función *setupMap* se realiza la creación del mapa, indicándole para ello, el tipo de mapa (basemapType), un punto inicial (latitude y longitude) y un nivel de detalle.

```
private func setupMap() {
       mapView.map = AGSMap(basemapType: .navigationVector, latitude: 34.09042, longitude: -118.71511,
           levelOfDetail: 12)
   }
```

Figura 5.39: Configuración del mapa

En tercer lugar, en la figura 5.40 se muestra como se realiza una llamada a los métodos que inicializan y crean la capa de figuras geométricas y las figuras geométricas respectivamente.

```
private func createGraphics(){
    createGraphicsOverlay()
    createPointGraphics()
    createPolylaneGraphics()
}
```

Figura 5.40: Llamada para creación de figuras en el mapa

En cuarto lugar, como se muestra en la figura 5.41, se crea un objeto de tipo *AGSGraphicsOverlay*, el cual albergará las figuras geométricas. Este objeto se añade como capa adicional del objeto *AGSMapView*.

```
private func createGraphicsOverlay(){
    graphicsOverlay = AGSGraphicsOverlay()
    self.mapView.graphicsOverlays.add(graphicsOverlay!)
}
```

Figura 5.41: Adición de capa de gráficos al mapa

Por último, se crean las figuras geométricas que se proyectarán sobre el mapa. Para crear un punto de referencia en el mapa se usa el objeto de tipo *AGSPoint*, en él se indica la latitud y la longitud, suficiente para determinar una posición en un mapa. Además, se incluye una referencia para indicar que se usará el sistema geodésico de coordenadas geográficas. El objeto *AGSSimpleMarkerSymbol* es utilizado para indicar la representación gráfica del punto. Por último, se crea el objeto *AGSGraphic* que contiene tanto la posición del punto como su representación y se incluye en el objeto *AGSGraphicsOverlay*.

Para crear una línea se utiliza el objeto *AGSPolylaneGraphics* al cual se le introduce dos puntos geográficos de tipo *AGSPoint* que indican los extremos de la línea. Después, se crea un objeto de tipo *AGSSimpleLineSymbol* para indicar el estilo de la línea. Por último, se crea un objeto de tipo *AGSGraphics* que contendrá la linea definida anteriormente y será añadido al objeto *AGSGraphicsOverlay*.

```
/*Crear un punto en el mapa*/
private func createPointGraphics(){
    let point = AGSPoint(x: -118.69333917997633, y: 34.032793670122885,
        spatialReference:AGSSpatialReference.wgs84())

    let pointSymobl = AGSSimpleMarkerSymbol(style: .circle, color: .brown, size: 10.0)
    pointSymobl.outline = AGSSimpleLineSymbol(style: .solid, color: .blue, width: 2.0)

    let pointGraphic = AGSGraphic(geometry: point, symbol: pointSymobl, attributes: nil)
    graphicsOverlay.graphics.add(pointGraphic)
}

/*Crear una linea en el mapa*/

private func createPolylaneGraphics(){
    let polyline = AGSPolyline(points: [
        AGSPoint(x: -118.67999016098526, y: 34.035828839974684, spatialReference: AGSSpatialReference.wgs84()),
        AGSPoint(x: -118.65702911071331, y: 34.07649252525452, spatialReference: AGSSpatialReference.wgs84())
    ])
    let polylineSymbol = AGSSimpleLineSymbol(style: .solid, color: .blue, width: 3.0)
    let polylineGraphic = AGSGraphic(geometry: polyline, symbol: polylineSymbol, attributes: nil)
    graphicsOverlay.graphics.add(polylineGraphic)
}
```

Figura 5.42: Creación de figuras

CAPÍTULO 6

CONCLUSIONES

En este capítulo se analiza el cumplimiento de los objetivos propuestos en el Capítulo 2, se exponen las conclusiones obtenidas al finalizar el TFG, se indica el trabajo futuro que se podría llevar a cabo para completar la aplicación móvil y se muestra la evolución de los conocimientos del autor en las tecnologías y herramientas empleadas durante el desarrollo del proyecto. Por último, el autor expone su opinión personal.

6.1 CONCLUSIONES

Gracias a esta aplicación móvil, la isla de Mallorca podrá ser explorada de forma interactiva y los usuarios podrán interactuar con todos los recursos turísticos distribuidos por la isla. La realización de este proyecto permite a los usuarios, planificar y diseñar sus viajes con antelación a la visita de la isla sin necesidad de acudir a una oficina física.

A nivel personal, es necesario destacar el conocimiento adquirido en tecnologías punteras durante el desarrollo de este proyecto tanto globales como el desarrollo de aplicaciones móviles para iOS como aspectos más concretos como la Realidad Aumentada.

6.2 CUMPLIMIENTO DE OBJETIVOS

El objetivo principal establecido en el Capítulo 2 "desarrollo de una aplicación móvil para iOS siguiendo el patrón de arquitectura VIPER y desarrollado en Swift que facilite a los usuarios el acceso a la información turística de la isla de Mallorca desde su dispositivo móvil, para que el turista pueda planificar su visita a la isla de Mallorca", ha sido completado con el desarrollo de la aplicación móvil **Smart Mallorca: App turimo**. La consecución del objetivo ha sido posible gracias a los siguientes motivos:

- Aplicación de la metodología de trabajo en IECISA.
- Haber hecho frente a los riesgos identificados durante la *Inception*.
- Trabajo en equipo.

También se establecieron un conjunto de objetivos parciales que se pueden observar en la tabla 6.1:

Objetivo	Estado	Evidencia
Desarrollo del módulo de Realidad Aumentada, que incluye la funcionalidad que permitirá a los usuarios localizar y obtener información de los recursos turísticos disponibles en la isla de Mallorca a través de Realidad Aumentada. Esta función permitirá a los usuarios localizar desde la cámara de su dispositivo móvil los recursos turísticos más próximos a su localización geográfica y en base a la latitud, longitud y altitud de dichos recursos.	Completado	Sprint 2: H5: Encuadre de cámara negativo (RA), H6: Encuadre de cámara positivo (RA), H7: Contenido Realidad Aumentada (RA)
Desarrollar el módulo de gestión de los recursos turísticos en los mapas (búsqueda, visualización, filtrado, rutas).	Parcialmente	Sprint 4: H10: SubHome 2, H11: Recurso - Descripción
Desarrollar el módulo que incluye las pantallas principales de la aplicación para la representación en el menú principal de los recursos turísticos, actividades turísticas, eventos, planes de viajes o expereriencias disponibles en la isla de Mallorca. Estos recursos estarán ordenados en base a tipologías de usuarios u ociotipos.	Completado	Sprint 3: H8: Home, H9: SubHome 1, H10: SubHome 2, H11: Recurso - Descripción
Desarrollar el módulo de gestión del acceso y registro a la aplicación turística.	Completado	Sprint 1: H2: Login, H3: Registro
Llevar a cabo en las pruebas necesarias en dispositivos físicos reales para comprobar el correcto funcionamiento de los distintos módulos.	Completado	Pruebas y demostraciones realizas durante las reuniones de revisión

Tabla 6.1: Objetivos específicos cumplidos

6.3 DESTREZAS ADQUIRIDAS

En el Capítulo 2 se detallaban los niveles de conocimientos del autor en las herramientas y tecnologías que durante el desarrollo del proyecto ha debido utilizar o poner en práctica.

Para la elaboración de las gráficas se ha utilizado un nivel de gradación de inexistente, bajo, medio o alto, donde inexistente se corresponde con el desconocimiento total acerca de la herramienta o tecnología, bajo con un conocimiento o manejo bajo de la herramienta, medio con un nivel aceptable de manejo o conocimiento y alto con un conocimiento eficaz y productivo de la herramienta. Cada herramienta se encuentra dividia en diferentes grupos en función de su utilidad dentro del desarrollo del proyecto de Smart Mallorca.

En las figuras 6.1 y 6.2 se puede observar el gráfico correspondiente a las capacidades del autor en el momento del inicio del desarrollo de este TFG y las capacidades del autor en el momento de la finalización del TFG respectivamente.

Como se puede apreciar en la comparación entre las figuras 6.1 y 6.2, se ha producido un crecimiento en todas las herramientas y tecnologías utilizadas durante el desarrollo del proyecto. Esto es

debido a que al inicio del proyecto el conocimiento sobre estas herramientas y tecnologías por parte del autor eran nulo o inexistente.

Concretamente, en las herramientas de documentación, planificación y seguimiento, el autor contaba con cierto nivel de manejo debido a que realizó un uso de ellas en etapas anteriores a la del proyecto.

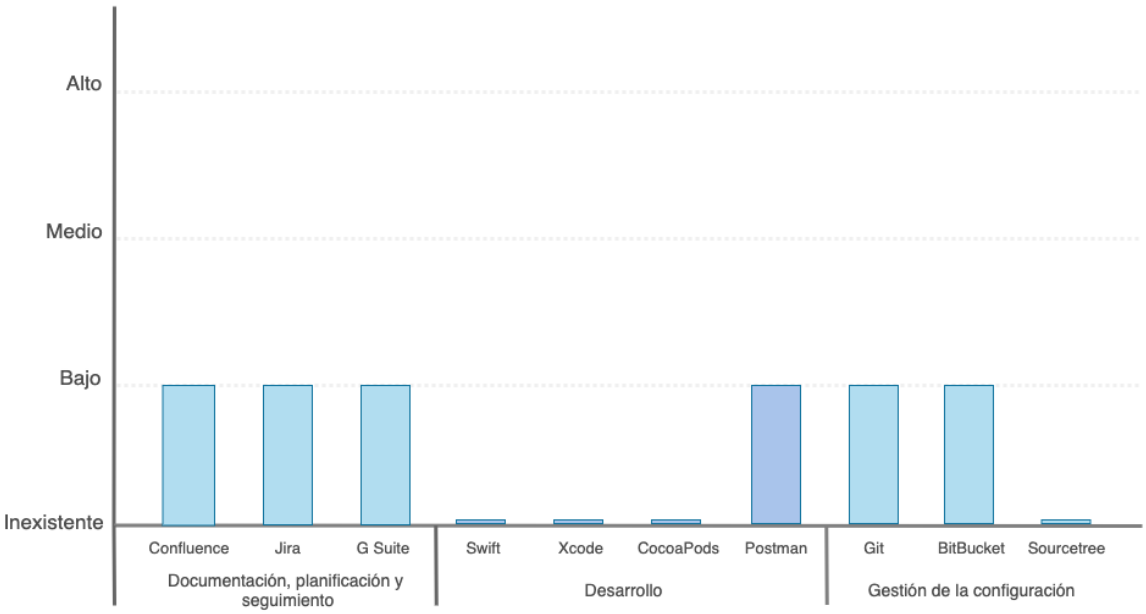


Figura 6.1: Competencias del autor al inicio del TFG

Comparando las herramientas y tecnologías de desarrollo se aprecia un notable aumento de nivel debido a su utilización diaria durante el desarrollo del proyecto. El autor ha adquirido conocimientos tanto en el lenguaje de programación Swift[20], como en el entorno de desarrollo Xcode 11[23], y en el administrador de dependencias CocoaPods[5], sin olvidar la herramienta Postman[16] de la que ya contaba con ciertos conocimientos que ha conseguido aumentar hasta un nivel superior. A parte de aumentar sus conocimientos en el lenguaje Swift, el autor ha adquirido conocimiento en el framework ARKit[3] para Realidad Aumentada y el SDK de ArcGIS[2] de iOS.

Por último, en el apartado de gestión de la configuración, el autor ha aumentado sus conocimientos tanto en el software de control de versiones Git[13], como en el servicio de alojamiento de repositorios BitBucket[4] y en la herramienta Sourcetree[19], utlizada como cliente para manejar los repositorios Git.

En definitiva, gracias al trabajo diario realizado ha sido posible adquirir un nivel aceptable de conocimientos en herramientas de documentación, planificación y seguimiento, en herramientas y tecnologías de desarrollo de aplicaciones móviles y herramientas para la gestión de la configuración.

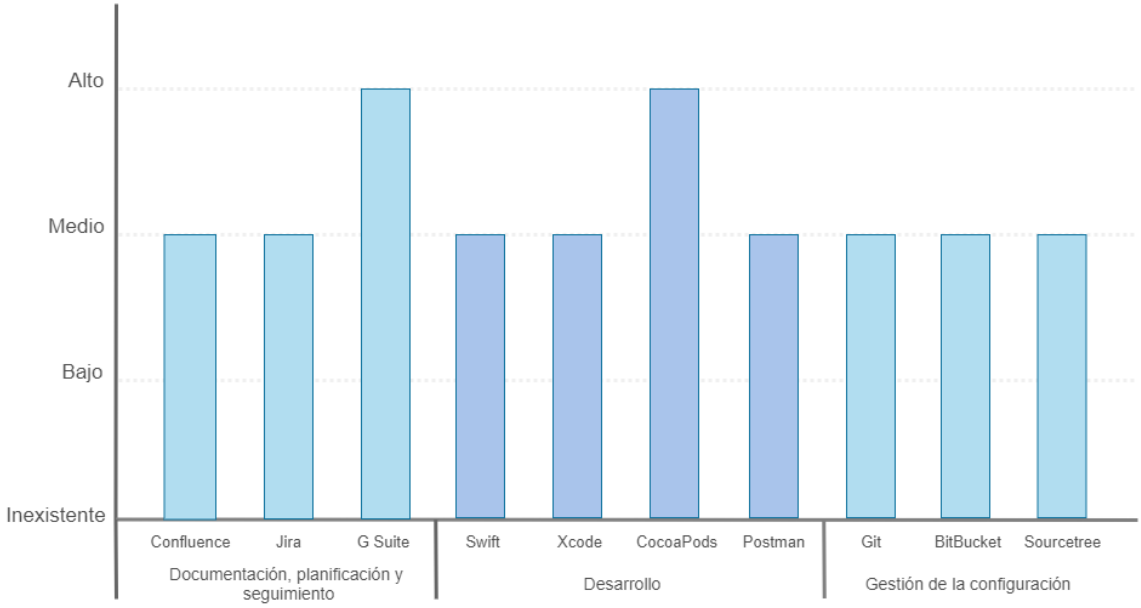


Figura 6.2: Competencias del autor a la finalización del TFG

6.4 COMPETENCIAS ADQUIRIDAS

En la tabla 6.2 se indican las competencias y habilidades adquiridas con la práctica junto a una descripción que permite justificar su relación con los conocimientos y competencias adquiridos en los estudios universitarios.

6.5 TRABAJO FUTURO

Una vez obtenida una primera versión de Smart Mallorca durante del desarrollo del TFG, se proponen algunas mejoras para modificar o ampliar la funcionalidad de la aplicación móvil. Estas propuestas de mejora son:

- En el momento de la finalización de este TFG, la aplicación se encuentra integrada con los servicios básicos de Smart Mallorca, por lo que es posbile plantear que en el futuro la aplicación esté integrada con todos los servicios restantes para ofrecer una operativa completa.

- Estudiar la posiblidad de incluir una opción de autenticación haciendo uso de huella dactilar.

- Construir un entorno de integración continua para las revisiones de la aplicación, despliege y ejecución de pruebas automatizadas.

- Estudiar la posibilidad de incluir una proyección automática de los puntos turísticos a través de realidad aumentada cúando estos aparecen en la pantalla del usuario y se encuentran dentro del marco con bordes blancos que está presente en esta vista. De esta forma, la información de los recursos turísticos podría ser visualizada sin necesidad de pulsar sobre el punto.

Competencia	Justificación
Capacidad para desarrollar, mantener y evaluar servicios y sistemas software que satisfagan todos los requisitos del usuario y se comporten de forma fiable y eficiente, sean asequibles de desarrollar y mantener y cumplan normas de calidad, aplicando las teorías, principios, métodos y prácticas de la Ingeniería del Software.	Esta competencia se ve reflejada en la necesidad de que la aplicación móvil a desarrollar cumpla con los requisitos de calidad esperados por el cliente. Para facilitar el cumplimiento de las necesidades de los clientes se seguirán las buenas prácticas de Ingeniería del Software.
Capacidad para valorar las necesidades del cliente y especificar los requisitos software para satisfacer estas necesidades, reconciliando objetivos en conflicto mediante la búsqueda de compromisos aceptables dentro de las limitaciones derivadas del coste, del tiempo, de la existencia de sistemas ya desarrollados y de las propias organizaciones.	Esta competencia se ve cumplida siguiendo el marco de trabajo Scrum junto a sus eventos y reuniones que facilitan la gestión de las necesidades de los clientes. Haciendo uso del marco Scrum es posible identificar las necesidades de los clientes y traducirlas en forma de requisitos.
Capacidad de dar solución a problemas de integración en función de las estrategias, estándares y tecnologías disponibles.	Esta competencia se ve reflejada con la necesidad de integrar elementos propios de la aplicación móvil con servicios externos a través de estándares de comunicación.
Capacidad de identificar y analizar problemas y diseñar, desarrollar, implementar, verificar y documentar soluciones software sobre la base de un conocimiento adecuado de las teorías, modelos y técnicas actuales.	Esta competencia se ve reflejada en la necesidad de aplicar teorías, modelos y técnicas apropiadas para analizar y dar una solución adecuada a la necesidad que origina este proyecto.
Capacidad de identificar, evaluar y gestionar los riesgos potenciales asociados que pudieran presentarse.	Esta competencia se ve reflejada y adquirida desde el inicio del proyecto en el cual se realizar una reunión Inception para identificar, analizar y valorar los posibles riesgos existentes en el proyecto.
Capacidad para diseñar soluciones apropiadas en uno o más dominios de aplicación utilizando métodos de la ingeniería del software que integren aspectos éticos, sociales, legales y económicos.	Esta competencia se ve reflejada en la necesidad de gestionar únicamente los datos mínimos necesarios para la identificación de los usuarios en la aplicación.

Tabla 6.2: Competencias adquiridas

6.6 OPINIÓN PERSONAL

El desarrollo de este TFG en un entorno profesional ha supuesto todo un reto para el autor debido a la magnitud y complejidad del proyecto al que se enfrentaba. Este proyecto le ha permitido poner en práctica muchos de los conocimientos adquiridos durante el Grado en Ingeniería Informática y además, ha sido posible adquirir muchos más conocimientos en tecnologías nuevas y emergentes gracias al convenio FORTE entre la Escuela Superior de Informática de Ciudad Real e IEICSA. Además, he podido experimentar de primera mano lo que es trabajar en un entorno real y profesional.

El resultado obtenido ha producido en el autor una gran satisfacción, por tanto agradece tanto a la Escuela Superior de Informática de Ciudad Real como a la empresa IECISA la oportunidad brindada.

BIBLIOGRAFÍA

[1] *«Jira».«Atlassian». [En línea].* URL: https://www.atlassian.com/es/software/jira.

[2] *ArcGIS Runtime SDK for iOS. [En línea].* URL: https://developers.arcgis.com/ios/latest/.

[3] *ARKit | Apple Developer Documentation. [En línea].* URL: https://developer.apple.com/documentation/arkit.

[4] *Bitbucket. «Atlassian». [En línea].* URL: https://bitbucket.org/.

[5] *CocoaPods | Dependency Manager. [En línea].* URL: https://cocoapods.org/.

[6] *Confluence. «Atlassian». [En línea].* URL: https://www.atlassian.com/software/confluence.

[7] *Consejo Insular de Mallorca. [En línea].* URL: https://www.conselldemallorca.cat/.

[8] *Escuela Superior de Informática - UCLM «Programa FORTE». [En línea].* URL: https://esi.uclm.es/categories/forte/nodes/convenios-forte.

[9] *Escuela Superior de Informática. [En línea].* URL: https://esi.uclm.es/.

[10] *Facebook SDK for iOS. [En línea].* URL: https://developers.facebook.com/docs/ios/.

[11] *Figma [En línea].* URL: https://www.figma.com/.

[12] *G Suite. [En línea].* URL: https://gsuite.google.com/intl/es/.

[13] *Git. [En línea].* URL: https://git-scm.com/.

[14] *Hangouts. [En línea].* URL: https://hangouts.google.com/.

[15] *Informática El Corte Inglés S.A. [En línea].* URL: https://www.iecisa.com/es.

[16] *Postman. | The Collaboration Platform for API Development [En línea].* URL: https://www.postman.com/.

[17] James Rumbaugh. *El lenguaje unificado de modelado. Manual de referencia.* 2007.

[18] Ken Schwaber y Jeff Sutherland. *The Scrum Guide.* 2017.

[19] *Sourcetree. «Atlassian». [En línea].* URL: https://www.sourcetreeapp.com/.

[20] *Swift Apple. [En línea].* URL: https://www.apple.com/es/swift/.

[21] *Turismo Ciudad Real. [En línea].* URL: https://turismo.ciudadreal.es/.

[22] *Twitter Developers. [En línea].* URL: https://developer.twitter.com/en.

[23] *Xcode | Apple Developer. [En línea].* URL: https://developer.apple.com/xcode/.

ANEXOS

ANEXO A

INCEPTION SMART MALLORCA APP TURISMO

A.1 INTRODUCCIÓN

Agile Inception, o también conocido como *Inception Desk*, es una sesión colaborativa con un conjunto de técnicas orientadas a alinear a todas las personas involucradas en un proyecto.

El objetivo de estas técnicas es el de reducir al máximo las incertidumbres que puedan surgir, identificando para ello los riesgos más evidentes y unificando la visión que tienen los implicados sobre el proyecto. Esto permite reducir al mínimo la falta de consenso, los problemas de comunicación o la ambigüedad. Las fases de una *Inception* y las preguntas que se pretenden responder se muestran a continuación.

A.2 ELEVATOR PITCH

En primer lugar y de forma colaborativa, todos los presentes deben establecer el *Elevator Pitch* del producto, utilizando para ello una plantilla. Todo el equipo debe conocer la motivación por la cual se quiere llevar a cabo este proyecto.

Si tuviéramos 45 segundos para vender nuestra idea en lo que dura una conversación de ascensor, ¿Que diríamos? Esta idea debe guiar al equipo a la hora de rellenar el *Elevator Pitch.*

Nos hemos centrado en el valor real del producto, estableciendo las necesidades y los beneficios claves que nos aportaría la aplicación en contraposición a los medios que ya existen. Esta técnica nos ha ayudado a alinear al equipo de IECISA, alumno y tutor de la UCLM en cuanto al trabajo que se va a realizar.

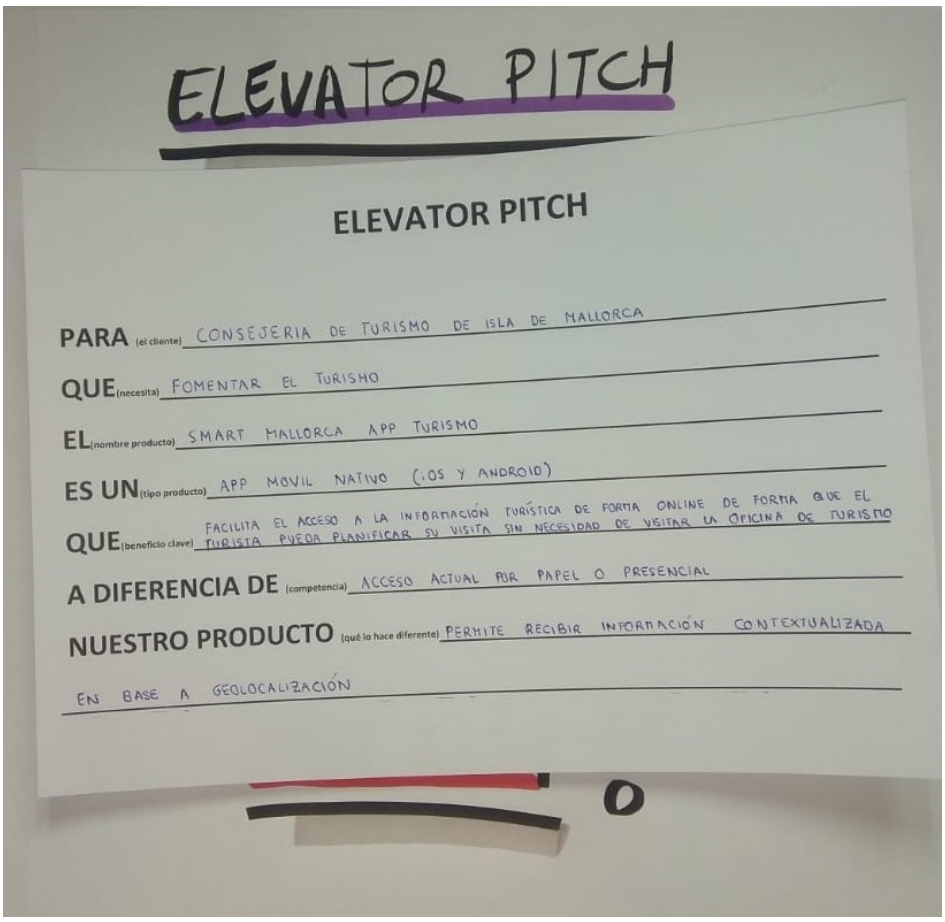


Figura A.1: Elevator Pitch

A.3 PROJECT COMUNNITY

Para la correcta consecución del objetivo es necesario que todos los participantes se conozcan. Es por ello que en este punto de la *Inception* se procede a identificar a todas las personas involucradas en el proyecto. Esto favorecerá la comunicación entre los miembros del proyecto y permitirá conocer la función que desempeñará cada integrante.

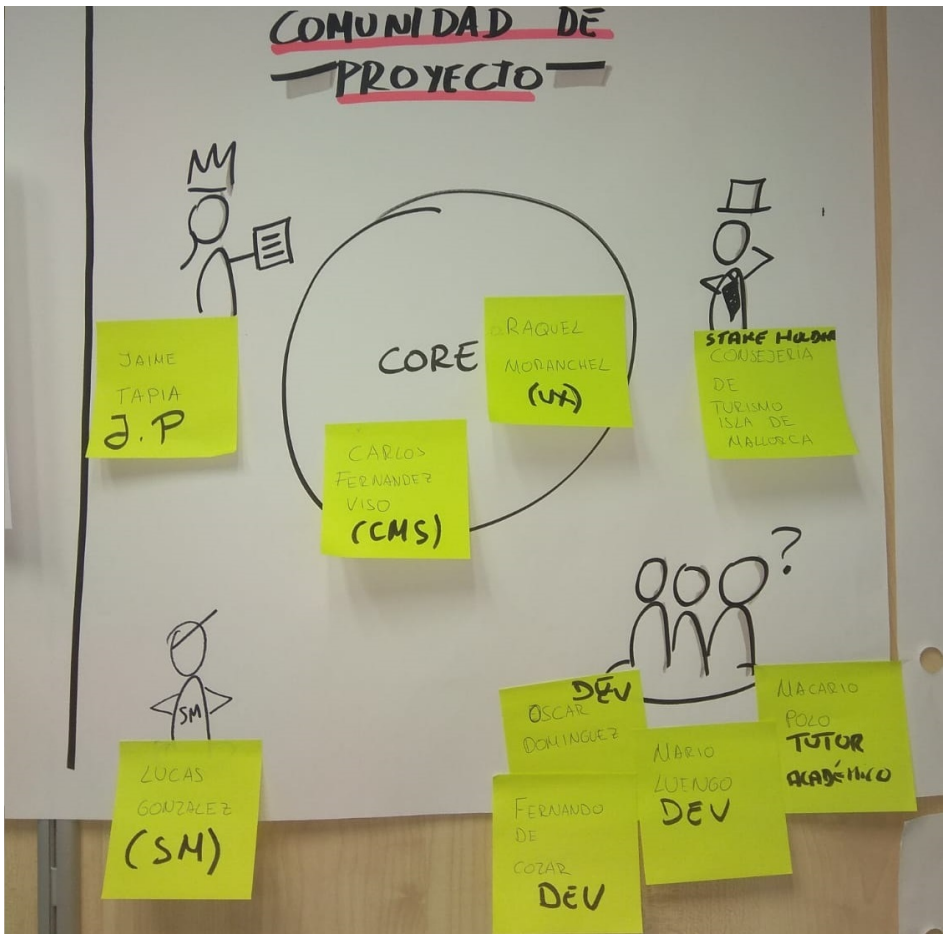


Figura A.2: Project Comunnity de Smart Mallorca

A.4 RESTRICCIONES

En este punto se procede a identificar las restricciones a las que estará sometido el proyecto.

En primer lugar se identifican restricciones relacionadas con el tiempo y fechas límites.

- Entrega Anteproyecto tutor académico (rellenar formulario) (23/3/2020)
- Reunión de seguimiento (20/3/2020)
- Entrega primera versión TFG tutor académico (18/6/2020)
- Entrega TFG (8/7/2020)
- Entrega anteproyecto (8/4/2020)
- Revisiones de trabajo realizado (cada 2 semanas)
- Entrega presentación (8/7/2020)
- Defensa (20/7/2020 - 24/7/2020)

En segundo lugar, se acuerda que a la finalización de cada Sprint de desarollo se producirá una entrega que incluirá un incremento software o una memoria.

En tercer lugar, se enumeran las restricciones de tipo tecnológico relacionadas con la arquitectura e infraestructura a utilizar.

- GIT
- Kotlin
- Android 6 (funcionalidad limitada)
- Android 8
- Swfit 5
- iOS 11
- arKIT
- ARGis
- Jira + Confluence

En cuarto lugar, se acuerda que el proyecto se realizará bajo el marco de trabajo Scrum. Se realizarán Sprints de 2 semanas.

Por último, se acuerda un conjunto de compromisos o primeros pasos que se deben llevar a cabo para la puesta en marcha del proyecto.

- Solicitar información de acceso a back end (CMS)
- Solicitar interfaces publicadas en Figma.
- Formación en las librerías de mapas (ARCGIS) y realidad aumentada (ARKit / AR Core)
- Información sobre el TFG (formularios, etc.):

A.5 RIESGOS

Una vez identificadas las restricciones a las que se verá sometido el proyecto, se deben identificar los riesgos. Cada riesgo cuenta con una probabilidad de ocurrir y con un impacto sobre el proyecto.

Se han identificado riesgos bajo la pregunta de: ¿Qué podría pasar que pudiera poner en peligro el proyecto? Una vez identificados los riesgos, estos se han clasificado en una matriz de probabilidad / impacto y se han creado acciones de mitigación y contingencia a las que se deberá dar seguimiento.

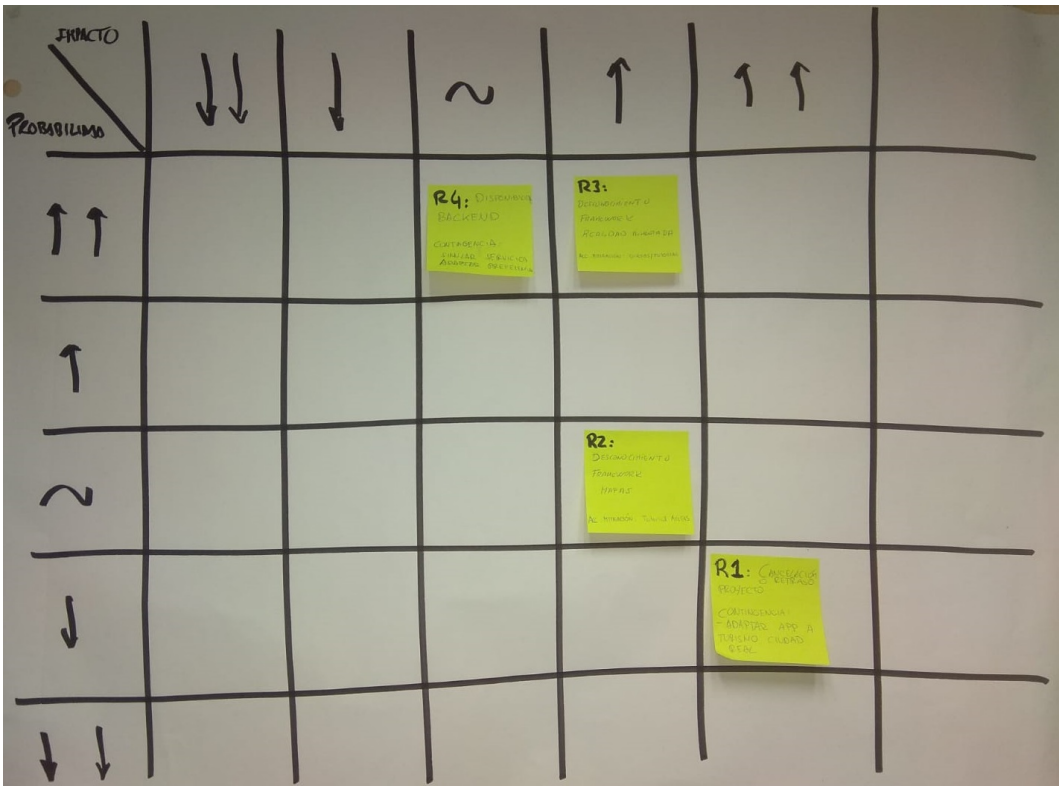


Figura A.3: Riesgos de Smart Mallorca

Para cada uno de los riesgos identificados en la figura anterior se ha propuesto una acción de mitigación o contigencia.

Riesgo	Probabilidad	Impacto	Acción de mitigación	Acción de contigencia
Cancelación o retraso del proyecto	Baja	Alto		Adaptar aplicación para turismo en Ciudad Real
Desconocimiento Framework Mapas	Media	Medio-alto	Realizar curso de aprendizaje de ARCGIS	
Desconocimiento Framework Realidad Aumentada	Alta	Medio-alto	Realizar curso de aprendizaje de ARKit / AR Core	
Disponibilidad Back-End	Alta	Medio		Simular servicios temporalmente y adaptar preferencias

Tabla A.1: Clasificación de riesgos y acciones a tomar

A.6 STORY MAP

En este punto, se procede a identificar todas las historias de usuario que componen el *Product Backlog* del proyecto y que servirán de base para comenzar el desarrollo. Posteriormente serán des-

glosadas en tareas que guiarán el desarollo. En esta tarea están involucrados todos los participantes en la *Inception*. Cabe destacar que el *Product Backlog* no se trata de un documento estático y por lo tanto es susceptible de sufrir modificaciones durante el desarollo.

Login/Registro	Recursos	Realidad Aumentada	Mapas
HU1. Crear cuenta o retraso del proyecto	HU6. Buscar recursos	HU16. Mostrar recursos a través de realidad aumentada	HU18. Ver recursos cercanos en un mapa
HU2. Acceder a la APP	HU7. Ver tiempo meteorológico	HU17. Mostrar detalle del recurso a través de realidad aumentada	HU19. Búsqueda de recursos en un mapa
HU3. Login via facebook	HU8. Ordenar recursos		HU20. Filtrar recursos en el mapa
HU4: Login via Twitter	HU9. Mostrar recursos por categorías		HU21. Visualizar la distancia entre recursos
HU5: Recuperar contraseña.	HU10. Ver recursos de una categoría en el mapa		HU22. Iniciar ruta
	HU11. Filtrar recursos		HU23. Mostrar alertas y de tiempo
	HU12. Detalle de un recurso		
	HU13. Audioguía de un recurso		
	HU14. Descarga de guía de un recurso		
	HU15. Añadir recurso a favoritos.		

Tabla A.2: Historias de usuario identificadas en la *Inception*

ANEXO B

REALIDAD AUMENTADA EN SMART MALLORCA: APP TURISMO

En el siguiente Anexo se procede a exponer en que consiste la realidad aumentada (RA) y cómo esta tecnología es aplicada en el proyecto Smart Mallorca: App turismo para permitir al usuario simular una experiencia gráfica con los recursos turísticos que componen la isla de Mallorca.

B.1 INTRODUCCIÓN

La realidad aumentada (RA) es el término usado para identificar al conjunto de tecnologías que permiten al usuario visualizar el mundo real a través de un dispositivo tecnológico (la cámara de un móvil o una tablet) con información gráfica añadida adicionalmente.

La primera vez que se acuñó este concepto fue en el año 1901, el escritor **Frank Baum** imaginó una gafas electrónicas para visuazliar información de las personas que se tenía delante.

En el año 1957, el cinematógrafo **Morton Heiling** realizó la primera implementación tecnológica basada en realidad aumentada. Pretendía crear una experiencia multisensorial a los usuarios utilizando para ellos elementos visuales, sonoros y olfativos, el experimiento recibió el nombre de *Sensorama*.

Años mas tarde, en 1973, el artista informático **Myron W. Krueger** pretendía crear un artilugio que mezclase cámaras de vídeo con una sistema de proyección. Su objetivo era que los usuarios pudiesen interactuar de forma interactiva con el entorno, respondiendo a los movimientos de los usuarios por medio de sombras y movimientos.

Es en el año 2016, adquiere una gran importancia. Esto fue debido al éxito de la aplicación móvil *Pokemon Go*[1], un videojuego que consistía en buscar y capturar diferentes criaturas recorriendo las calles de las ciudades e interactuando con el entorno y que llegó a alcanzar la gigantesca cifra de 45 millones de usuarios diarios activos.

[1] https://pokemongolive.com/es/



Figura B.1: Realidad Aumentada en Pokemon Go

Aunque pueda parecer una tecnología con muchos años de antigüedad, la Realidad Aumentada es en la actualidad una tecnología todavía emergente y necesitada de una gran cantidad de avances tecnológicos en computación y visualización para alcanzar una madurez adecuada.

B.1.1 Características de la Realidad Aumentada

Las características que hacen que la Realidad Aumentada sea una de las tecnologías con mayor futuro y con mayor índice de crecimiento son:

- **Combinación del mundo real y el mundo virtual**: Haciendo uso de realidad aumentada es posible interactuar con el mundo real a través de elementos gráficos del mundo virtual. De esta forma se obtienen experiencias mucho más completas que no serían posibles sin esta tecnología.

- **Dependiente del contexto**: La información adicional que el usuario puede percibir tiene relación directa con la información real que está recibiendo. Por tanto, el usuario podrá observar el entorno real con información añadida, obteniendo así un retrato de la realidad mucho mas completo.

- **Interactivo en tiempo real**: La Realidad Aumentada interactúa con el usuario en tiempo real, es decir, cualquier acción que provoque un usuario en el dispositivo con el que está interactuando la Realidad Aumentada, tiene su repercusión en la escena recreada con Realidad Aumentada.

- **Tridimensional**: La información virtual que se puede mostrar con Realidad Aumentada es dependiente del entorno físico que lo rodea, por lo tanto es posible adaptar el mundo virtual al entorno físico teniendo en cuenta la relación que existe con este a través de distancias, posicionamiento u orientación.

B.1.2 Tipos de la realidad aumentada

Dependiendo de los componentes del mundo real con lo que se interaccione y del objetivo final que se quiera obtener, los tipos de Realidad Aumentada identificables son:

- **Espacios**: Debido a la constante evolución a la que está sometida la tecnología de realidad aumentada, existen en la actualidad dispositivos que nos permiten reconocer estancias, habitaciones, o capas físicas del entorno como pueden ser suelos, paredes o techos. Con esta información es posible diseñar y generar mapas en tres dimensiones en los que proyectar componentes visuales que faciliten las labores de los usuarios. Un ejemplo claro de este tipo de realidad aumentada podría ser la creación de una ruta en un museo que indicase al usuario por donde debería moverse.

- **Imágenes**: Anteriormente y en la actualidad en menor medida, se utlizaban los denominados *Códgos QR* como apoyo sobre el que colocar contenido de Realidad Aumentada. En la actualidad, gracias a las técnicas de clasifiación de imágenes, es posible proyectar contenido de realidad aumentada sobre cualquier elemento, como pueden ser una tarjeta de visita de una empresa o un mapa turístico.

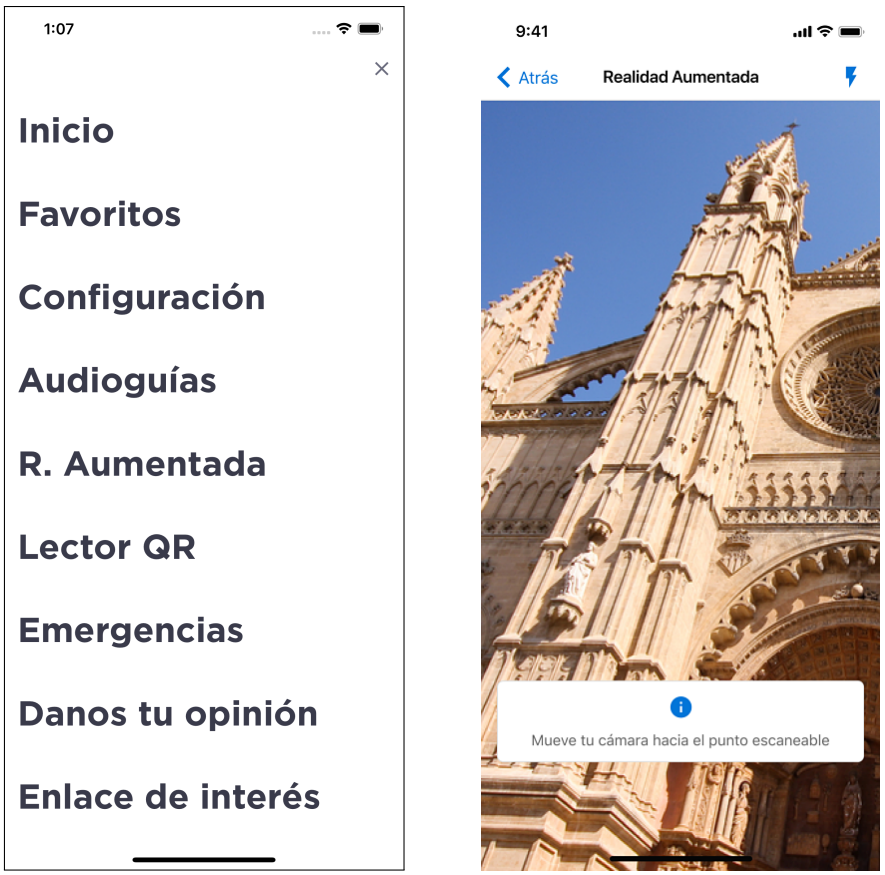
- **Lugares**: Conociendo las coordenadas geográficas de un lugar concreto es posible visualizar contenido de Realidad Aumentada en base a ese punto geográfico que suele recibir el nombre de punto de interés. Es posible proyectar elementos visuales jugando con la perspectiva, las distancias, la altura o los ángulos de inclinación que se pueden obtener de los sensores GPS o las brújulas digitales de un dispositivo móvil. Este tipo de Realidad Aumentada es el que será puesto en práctica en este proyecto.

B.2 REALIDAD AUMENTADA EN SMART MALLORCA

Llegados a este punto, hablaremos de la implementación de esta tecnología en la aplicación Smart Mallorca. Con ella el usuario podrá localizar a través de la cámara de su dispositivo móvil (iPhone o iPad) los recursos turísticos que están distribuidos por toda la isla.

El usuario podrá activar esta funcionalidad accediendo desde el menú lateral de la aplicación, que se encuentra disponible en la ventana *Home* de la aplicación. Una vez accedido, se mostrará en pantalla la cámara del dispositivo, la cúal estará en condiciones de proyectar los elementos de Realidad Aumentada. Aparecerá un mensaje en pantalla similar al que se muestra en la figura B.2a. En ese mismo instante el usuario estará en condiciones de proyectar los recursos turísticos a través de Realidad Aumentada. Además, de forma adicional, el usuario podrá activar la iluminación de su dispositivo móvil en caso de que la iluminación en el mundo real no fuese adecuada.

Las imágenes que se muestran en la figura B.2 se corresponden con los prototipos diseñados para la versión de Smart Mallorca en un dispositivo iPhone.



(a) Menú lateral de acceso a realidad aumentada **(b)** Pantalla inicial de realidad aumentada

Figura B.2: Acceso a realidad aumentada en Smart Mallorca

En el momento que el usuario acceda a la cámara de su dispositivo móvil tendrá en su mano la posibilidad de localizar los puntos turísticos a través de ella. Si el usuario está enfocando hacia un recurso turístico entonces, en la pantalla del dispositivo será proyectada una figura cuadrada que indique que en ese lugar se encuentra un punto de interés (PDI).

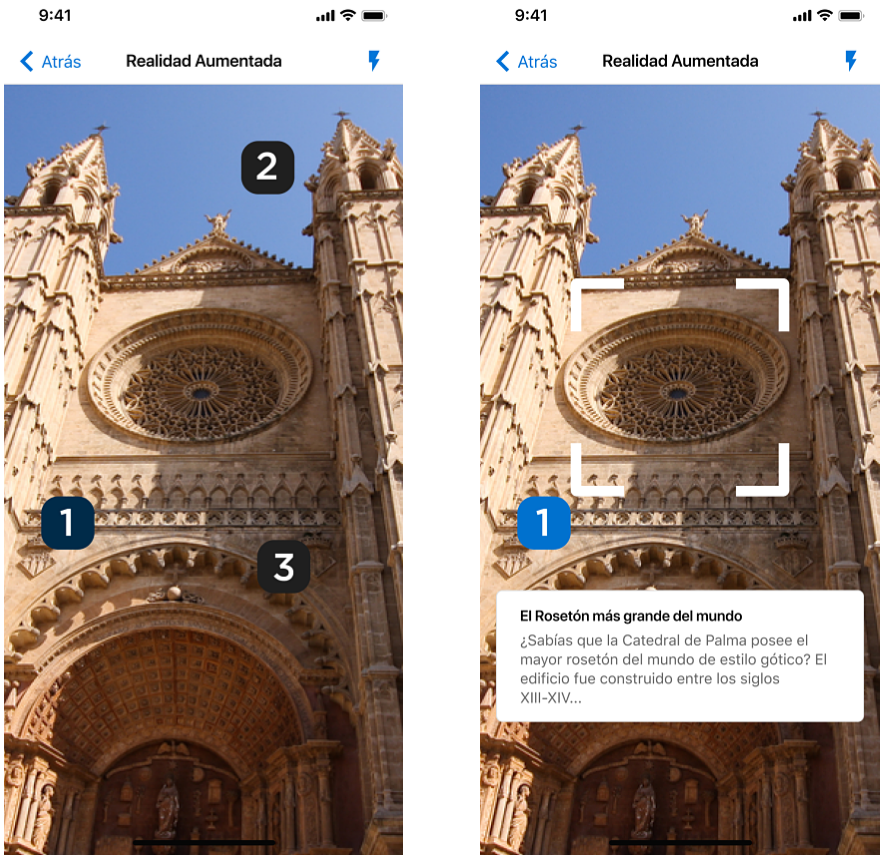
Para comprender como ARKit[3], que será la librería usada para implementar esta funcionalidad en iOS, coloca y proyecta los puntos turísticos a través de la cámara del usuario es necesario trabajar con la siguiente información:

- Todos los recursos de la isla de Mallorca son recibidos como resultado de realizar una petición HTTP al servidor de Smart Mallorca. Por cada recurso turístico, la información que facilita su gestión a través de realidad aumentada es:

  - **Latitud:** Representación en grados decimales de la posición de un recurso turístico respecto del ecuador.

  - **Longitud:** Representación en grados decimales de la posición de un recurso turístico respecto del meridiano 0° o meridiano de Greenwich.

- **Altura:** Distancia a la que se encuentra el recurso del suelo. Es importante no confundir esta medida con la altitud que representa la distancia desde el nivel del mar hasta el recurso.



(a) Encuadre positivo en Smart Mallorca **(b)** Información de un recurso turístico

Figura B.3: Proyección de recursos turísticos a través de realidad aumentada

- Gracias al acelerómetro, al giroscopio de tres ejes y al barómetro con el que cuentan los dispositivos iPhone y iPad es posible determinar una serie de mediciones que nos permiten ajustar la localización del usuario respecto de los recursos turísticos. Estas medidas son:

  - **Posición geográfica:** Indica la posición geográfica del usuario en grados decimales. Incluye dos componentes, la componente respecto del ecuador (latitud) y la componente respecto del meridiano de Greenwich (longitud)

  - **Altura:** Distancia a la que se encuentra el usuario respecto del suelo. No debe ser confundida con la altitud al nivel del mar.

  - **Altitud:** Altura del usuario respecto del nivel del mar

  - **Orientación:** Orientación del usuario respecto de los puntos cardinales. Se mide en grados y el 0º representa el Norte.

– **Ángulo de inclinación:** Indica el grado de inclinación del dispositivo.

B.3 IMPLEMENTACIÓN DE REALIDAD AUMENTADA EN SMART MALLORCA

En este apartado se explica el desarrollo llevado a cabo para implementar la funcionalidad de Realidad Aumentada en Smart Mallorca. Para la explicación de la vista de Realidad Aumentada se seguirá un orden lógico para favorecer la compresión.

En primer lugar, como se ha comentado anteriormente, el primer método que se ejecuta al cargar una vista es el método *ViewDidLoad*, en él se realizan las llamadas a los métodos que construyen la vista.

```
override func viewDidLoad() {
    super.viewDidLoad()
    createTabBar()
    setupAR()
    composeARViews()
}
```

Figura B.4: Función viewDidLoad en vista de Realidad Aumentada

El primer método llamado en la función anterior es el *createTabBar*. Esta construye la barra de navegación superior en la que se situan un botón para volvér al menú lateral y un botón para activar la linterna del dispositivo.

```
func createTabBar(){
    navigationBar.backgroundColor = UIColor.white
    navigationBar.delegate = self

    let navItem = UINavigationItem()
    navItem.title = "Realidad Aumentada"

    buttonBack = UIButton(type: UIButton.ButtonType.system)
    buttonBack.setImage(UIImage(named: "Back"), for: UIControl.State.normal)
    buttonBack.setTitle("Atrás", for: .normal)
    buttonBack.addTarget(self, action: #selector(tapBackButton), for: UIControl.Event.touchUpInside)

    buttonlight = UIButton(type: UIButton.ButtonType.custom)
    let origImage = UIImage(named: "Flash")
    let tintedImage = origImage?.withRenderingMode(.alwaysTemplate)
    buttonlight.setImage(tintedImage, for: .normal)
    buttonlight.setImage(tintedImage, for: UIControl.State.normal)
    buttonlight.tintColor = UIColor(displayP3Red: 108/255, green: 157/255, blue: 209/255, alpha: 1)
    buttonlight.addTarget(self, action: #selector(tapFlashLightButton), for: UIControl.Event.touchUpInside)

    let barButton = UIBarButtonItem(customView: buttonlight)
    let barButtonBack = UIBarButtonItem(customView: buttonBack)

    navItem.rightBarButtonItem = barButton
    navItem.leftBarButtonItem = barButtonBack

    navigationBar.items = [navItem]

}
```

Figura B.5: Barra de navegación superior en vista de Realidad Aumentada

Seguidamente se llama al método *setupAR* con el cual se realiza la configuración de la vista de Realidad Aumentada. Es importante reseñar que el objeto *ARLandmarker* es la piedra angular sobre la que se conformará toda la funcionalidad de la Realidad Aumentada porque sobre él se ajustarán todos los aspectos de configuración.

Sobre el objeto *landmarker* se define la distancia máxima a la que serán visibles los recursos turísticos, la escala máxima del punto que representa el recurso turístico, la tasa de refresco del GPS, que es de 30 segundos para comprobar si se han producido cambios en la posición del usuario y una opción para permitir la superposición de los puntos.

```
func setupAR(){
    landmarker = ARLandmarker(view: ARSKView(), scene: InteractiveScene(), locationManager: CLLocationManager())
    landmarker.delegate = self
    landmarker.maximumVisibleDistance = 120

    landmarker.minViewScale = 0.5
    landmarker.maxViewScaleDistance = 75

    landmarker.worldRecenteringThreshold = 30
    landmarker.locationManager.desiredAccuracy = kCLLocationAccuracyBestForNavigation

    landmarker.overlappingLandmarksStrategy = .showAll

}
```

Figura B.6: Configuración funcionalidad Realidad Aumentada

El último método invocado en la función*ViewDidLoad* es *composeARViews* que su única función es la de añadir todas las vistas necesarias a la pantalla.

```
func composeARViews(){

    viewCamara.addSubview(landmarker.view)
    viewCamara.addSubview(viewInformacion)
    viewCamara.addSubview(encuadreCamara)
    viewCamara.addSubview(viewInformacionPunto)
}
```

Figura B.7: Composición de vista en Realidad Aumentada

La función que aparece en la figura B.8 es la encargada de recibir y configurar la posición geográfica del usuario. Para ello, se accede al parámetro *location* incluido en el objeto *LocationManager* del componente principal de nusetra vista, el objeto *landmarker*. Una vez recibida la posición se añade como punto de referencia de la escena. Además, esta función recibe todos los recursos turísticos y los posiciona en la escena en función de su latitud, longitud y altitud.

```
func displayLandmarks(viewModel: ListLandmarks.FetchLandmarks.ViewModel) {
    for landmark in viewModel.landmarks {
        let user = landmarker.locationManager.location!
        let markView = reusableMarker
        let location = CLLocation(coordinate: landmark.location.coordinate, altitude: (landmark.location.altitude + user.altitude + 5),
            horizontalAccuracy: 1, verticalAccuracy: 1, timestamp: Date())
        markView.set(idPunto: landmark.idPunto, name: landmark.name, detail: format(distance: user.distance(from: landmark.location)))
        landmarker.addLandmark(userInfo: [landmarkKey: landmark], view: markView, at: location, completion: nil)
    }
}
```

Figura B.8: Mapeo de información de recurso turístico en escena

En la figura B.9 está representado el protocolo delegado que debe ser implementado para mantener un correcto funcionamiento de la vista de realidad aumentada. Incluye la definición de unos métodos que deben ser implementados por el programador.

El primer método desencadena una petición al componente *Presenter* para obtener los recursos turísticos. Esta llamada desencadena la petición por parte del *Interactor* para recibir los recursos turísticos y añadirlos a la escena a través del método definido en la figura B.8.

El segundo método gestiona la carga de la información del recurso turístico en la vista con la información del mismo. Se obtiene el título y la información y una vez obtenidos se hacen visibles.

El tercer método de este protocolo es el encargado de actualizar la posición y la proyección del recurso turístico en la vista de Realidad Aumentada cuando el usuario cambia de posición.

El cuarto y último método se lanza exclusivamente cuando se produce un error.

```
extension EncuadreCamaraViewController: ARLandmarkerDelegate {

    func landmarkDisplayerIsReady() {
       presenter?.fetchLandmarksPresenter(request: ListLandmarks.FetchLandmarks.Request())
    }

    func landmarkDisplayer(_ landmarkDisplayer: ARLandmarker, didTap landmark: ARLandmark) {
        guard let index = landmark.model?.index else {
            return
        }
        let info = presenter?.showLandmark(withIndex: index)
        guard let title = info?.title, let description = info?.description else {
            return
        }


        setView(view: viewInformacionPunto, hidden: false)
        tfTitle.text = title
        tfInformacionPunto.text = description

    }

    func landmarkDisplayer(_ landmarkDisplayer: ARLandmarker, willUpdate landmark: ARLandmark, for location: CLLocation) -> UIView? {
        guard let model = landmark.model else {
            return nil
        }
        let markView = reusableMarker
        markView.set(idPunto: model.idPunto, name: model.name, detail: format(distance: location.distance(from: landmark.location)))
        return markView
    }

    func landmarkDisplayer(_ landmarkDisplayer: ARLandmarker, didFailWithError error: Error) {
        print("Failed! Error: \(error)")
    }
}
```

Figura B.9: Protocolo funcionamiento Realidad Aumentada

Por último, el método que aparece en la figura B.10 es el encargado de gestionar el toque del usuario en la pantalla. Este método se ejecuta cada vez que el usuario pulsa la pantalla y en él se comprueba si en el toque de pantalla está incluido un objeto de tipo *SKNode*, que es el que conforma los nodos que representan los recursos turísticos. Si es la primera vez desde que se inició la pantalla que el usuario toca la pantalla, desaparece el panel inferior informativo. Además, si hay una vista con descripción de un recurso mostrándose y el usuario pulsa fuera de ella, esta desaparece.

```
override func touchesBegan(_ touches: Set<UITouch>, with event: UIEvent?) {
    if (!primerToque){
        primerToque = true
        setView(view: viewInformacion, hidden: true)
    }
    for touch in touches {
        let location = touch.location(in: landmarker.scene.scene!)
        let node : SKNode = landmarker.scene.scene!.atPoint(location)
        if node.name != nil  {
            return
        }else{
            setView(view: viewInformacionPunto, hidden: true)
        }
    }
}
```

Figura B.10: Gestión de toques en pantalla

B.4 DEMOSTRACIÓN REALIDAD AUMENTADA

Una vez finalizado el desarrollo de la pantalla de Realidad Aumentada, el autor realizó un video demostración. Para ello simuló unos recursos turísticos en la ciudad de Ciudad Real incluyendo nombre, descripción e imagen de ellos. En las siguientes imágenes se exponen los resultados. Para el desarrollo de esta demostración se utilizó el iPad Pro con el que contaba el autor.

En la imagen de la figura B.11 aparece la pantalla de la aplicación en el momento en el que no es posible visualizar recursos turísticos.



Figura B.11: Pantalla inicial de la demostración en Ciudad Real

En la figura B.12 aparece la pantalla cuando en ella se proyecta un recurso turístico, en este caso el Ayuntamiento de Ciudad Real.



Figura B.12: Recurso turístico proyectado en Ciudad Real

En las figuras B.13 y B.14 aparecen capturas de la demostración en las que el usuario ha pulsado sobre los puntos que representan los recursos turísticos. En ese momento aparece una vista en la parte inferior de la pantalla con la que el usuario puede visualizar toda la información relacionada con el recurso.



Figura B.13: Descripción del recurso 1 en Ciudad Real



Figura B.14: Descripción del recurso 2 en Ciudad Real